

JavaScript: End to End

A Complete Guide from First Line to Production

This book takes you from "what is a variable" all the way to building, testing, and shipping real JavaScript applications in the browser and on Node.js. Every chapter pairs explanation with runnable code. The matching code for chapter `NN-topic.md` lives in `examples/NN-topic.js` (or a subfolder for multi-file examples) — open it, run it with `node`, and experiment.

How to use this book

```
node examples/03-variables-and-data-types.js
```

Every example file runs standalone with plain Node.js (no build step, no dependencies) unless a chapter says otherwise (DOM chapters need a browser console instead).

Table of Contents

Part I — Foundations

1. [Introduction to JavaScript](#)
2. [Setting Up Your Environment](#)
3. [Variables and Data Types](#)
4. [Operators](#)
5. [Control Flow](#)
6. [Loops and Iteration](#)
7. [Functions](#)

Part II — Core Data Structures 8. [Arrays](#) 9. [Objects](#) 10. [Strings](#) 11. [Numbers and Math](#) 12. [Dates and Time](#) 13. [Destructuring, Spread & Rest](#)

Part III — The Engine Room 14. [Scope and Closures](#) 15. [The `this` Keyword](#) 16. [Prototypes and Inheritance](#) 17. [Classes](#) 18. [Error Handling](#) 19. [Higher-Order Functions & Functional Programming](#) 20. [Iterators and Generators](#)

Part IV — Asynchronous JavaScript 21. [Callbacks and the Event Loop](#) 22. [Promises](#) 23. [Async/Await](#)

Part V — Working With the Platform 24. [Modules](#) 25. [JSON and Web Storage](#) 26. [Regular Expressions](#) 27. [The DOM](#) 28. [Events](#) 29. [Fetch, AJAX, and HTTP](#)

Part VI — Node.js and the Ecosystem 30. [Node.js Fundamentals](#) 31. [npm and Package Management](#) 32. [Testing JavaScript](#)

Part VII — Writing Better JavaScript 33. [Design Patterns](#) 34. [Performance and Best Practices](#) 35. [Building Real Projects](#) 36. [Appendix: Cheat Sheet](#)

Chapter 1 — Introduction to JavaScript

1.1 What is JavaScript?

JavaScript (JS) is a high-level, interpreted (or just-in-time compiled), dynamically-typed programming language. It was created in 1995 by Brendan Eich in just ten days while he was working at Netscape, and it has since become the only programming language that runs natively in every web browser on Earth. What started as a small scripting language for adding interactivity to web pages ("make the button blink") has grown into one of the most widely used programming languages in the world, powering:

- **Front-end web development** — every interactive website you use (React, Vue, Angular, Svelte, or plain vanilla JS)
- **Back-end development** — via Node.js, JS powers APIs, servers, and microservices (Express, Fastify, NestJS)
- **Mobile apps** — React Native, Ionic, NativeScript
- **Desktop apps** — Electron (VS Code, Slack, Discord are all Electron apps)
- **Games** — Phaser, Three.js, Babylon.js
- **Machine learning** — TensorFlow.js
- **Embedded / IoT** — Johnny-Five, Espruino
- **Command-line tools** — npm itself is written in JS

The single most important fact to internalize about JavaScript is this: **it is the only language that runs natively in web browsers**. You can compile other languages *to* JavaScript (TypeScript, ClojureScript, WebAssembly-adjacent tooling), but the browser itself only understands HTML, CSS, and JavaScript.

1.2 JavaScript vs. Java

Despite the name, JavaScript has almost nothing to do with Java. The name was a marketing decision made by Netscape in 1995 to ride the popularity of Java at the time. JavaScript's actual design was inspired by Scheme (functions as first-class values), Self (prototype-based objects), and a C-like syntax borrowed from Java to look familiar to programmers.

	JavaScript	Java
Typing	Dynamic	Static
Execution	Interpreted / JIT-compiled	Compiled to bytecode, run on JVM
Object model	Prototype-based	Class-based

	JavaScript	Java
Primary environment	Browser, Node.js	JVM
Concurrency model	Single-threaded, event loop	Multi-threaded

1.3 ECMAScript: the specification behind the language

JavaScript is an *implementation* of a specification called **ECMAScript** (ES), maintained by Ecma International's TC39 committee. Different JavaScript engines (V8 in Chrome and Node.js, SpiderMonkey in Firefox, JavaScriptCore in Safari) all implement the ECMAScript specification, plus some engine-specific extras.

Major milestones you'll hear referenced constantly:

- **ES5 (2009)** — `strict mode`, `Array.prototype.map/filter/reduce`, `JSON` object, `Object.create`
- **ES6 / ES2015** — the biggest single update ever: `let / const`, arrow functions, classes, template literals, destructuring, promises, modules, generators, default/rest parameters
- **ES2016 and onward (yearly releases)** — `Array.prototype.includes`, `async / await` (ES2017), object spread (ES2018), `Array.flat / flatMap` (ES2019), optional chaining `?.` and nullish coalescing `??` (ES2020), `Array.at`, top-level `await` (ES2022), array grouping and more (ES2023+)

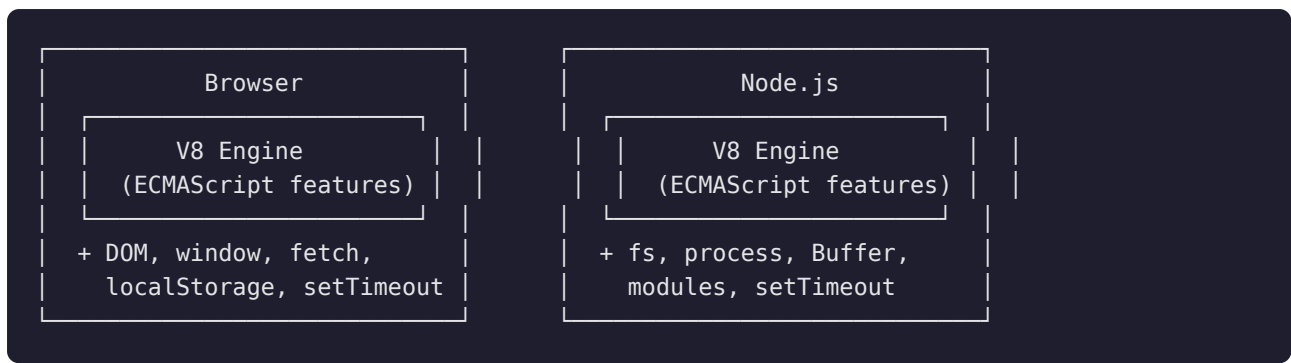
TC39 now ships a new version every year, so "ES6" and "modern JavaScript" are often used loosely to mean "anything from ES2015 onward." This book teaches modern JavaScript throughout and calls out which version introduced a feature when it matters.

1.4 How JavaScript runs: engines, runtimes, and hosts

A common point of confusion: **JavaScript the language** is separate from **the environment that runs it**.

- **Engine** — the program that parses and executes JS code (e.g., V8, SpiderMonkey). The engine implements ECMAScript: variables, functions, objects, the event loop's microtask queue, etc.
- **Runtime / Host environment** — the engine embedded inside a larger program that adds extra capabilities the language itself doesn't define:
 - **Browsers** add the DOM, `window`, `fetch`, `localStorage`, timers
 - **Node.js** adds the file system (`fs`), `process`, modules, `Buffer`

This is why `document.querySelector` works in a browser console but throws `ReferenceError: document is not defined` in Node.js — `document` isn't part of JavaScript at all; it's a browser API.



1.5 Compiled vs. interpreted: how V8 actually works

Modern engines like V8 are not purely "interpreted" in the old sense. V8 uses a pipeline:

1. **Parsing** — source code becomes an Abstract Syntax Tree (AST).
2. **Ignition (interpreter)** — generates and runs bytecode quickly, with low startup cost.
3. **TurboFan (optimizing JIT compiler)** — profiles hot code paths (code that runs many times) and compiles them down to highly optimized machine code on the fly.
4. **Deoptimization** — if an assumption TurboFan made turns out to be wrong (e.g., a variable that was always a number becomes a string), V8 "bails out" back to the interpreter and re-optimizes later.

You don't need to memorize this pipeline, but understanding that engines *optimize based on observed behavior* explains real performance advice you'll see later in this book — e.g., "don't change an object's shape after creation" and "keep function argument types consistent."

1.6 Single-threaded, event-driven, non-blocking

JavaScript runs on a **single thread** — one line of code executes at a time, top to bottom, and nothing runs in parallel *within your JS code*. Yet JavaScript is famous for handling thousands of concurrent network requests without blocking. The trick is the **event loop**, which we cover in depth in Chapter 21. For now, the mental model is:

- Your code runs synchronously, one call stack frame at a time.
- Slow operations (network requests, file reads, timers) are handed off to the runtime (browser APIs or Node's libuv thread pool), which notifies JavaScript via a callback/queue once the work is done.
- JavaScript never blocks waiting for that work — it moves on to the next line, and the callback runs later when the engine is free.

This "don't block, get notified later" pattern is why asynchronous programming (callbacks → promises → async/await) is such a central topic in JavaScript, more so than in many other mainstream languages.

1.7 Dynamic and weakly typed

JavaScript variables don't have a fixed type — the *value* has a type, not the variable box that holds it:

```
let x = 42;           // x holds a number
x = "hello";         // now x holds a string – perfectly legal
x = [1, 2, 3];       // now x holds an array
```

JavaScript is also **weakly typed** (or "loosely typed"): it will automatically convert types for you in many operations, which is a frequent source of bugs for newcomers:

```
"5" + 3           // "53" (number coerced to string, then concatenated)
"5" - 3           // 2    (string coerced to number, then subtracted)
"5" * "2"         // 10   (both coerced to number)
1 + true          // 2    (true coerced to 1)
0 == "0"          // true  (loose equality coerces types)
0 === "0"         // false (strict equality does not coerce)
```

We'll spend real time in Chapter 4 on type coercion rules so these stop being surprises and start being tools you use deliberately.

1.8 Why learn JavaScript deeply?

You can write React apps, Vue apps, or Express servers while treating JavaScript as a black box you copy-paste patterns into. Many people do. But every "weird" bug you'll hit — `this` pointing to the wrong object, a `for` loop closure capturing the wrong variable, an array method silently returning `undefined`, a promise that never resolves — has a precise, learnable explanation rooted in how the language actually works. This book is structured to build that mental model layer by layer: syntax first, then data structures, then the engine-level concepts (scope, `this`, prototypes), then asynchronous programming, then the browser and Node.js platforms, and finally the practices and patterns professional engineers use to write JavaScript that scales.

1.9 Chapter summary

- JavaScript is a dynamic, single-threaded, event-driven language that runs natively in every web browser and, via Node.js, on servers.
- ECMAScript is the specification; JavaScript engines (V8, SpiderMonkey, JavaScriptCore) are implementations of it.
- "Modern JavaScript" generally means ES2015 (ES6) and later.
- The language itself is separate from its host environment — the DOM belongs to browsers, `fs` / `process` belong to Node.js.

- JavaScript is dynamically and weakly typed, which enables flexibility but requires understanding coercion rules to avoid bugs.

1.10 Exercises

1. Open your browser's DevTools console (F12) and type `typeof window`, then open a Node.js REPL (`node`) and type the same thing. What's the difference, and why?
2. Predict the output of `"10" - "4" + "2"` and `"10" + "4" - "2"` before running them. Then run `examples/01-introduction-to-javascript.js` to check your answers.
3. List three things you can build with JavaScript that you didn't know before reading this chapter.

Chapter 2 — Setting Up Your Environment

2.1 Two places JavaScript runs, two ways to start

Before writing real programs you need a place to run code and a way to see its output. This book uses two environments throughout:

1. **The browser** — for anything touching the DOM, events, or the visual page (Part V).
2. **Node.js** — for everything else: learning the language itself, scripting, servers, tests, and tooling. Every example file in this book's `examples/` folder is a plain Node.js script unless the chapter says otherwise.

You do not need to choose one over the other permanently — professional JavaScript developers move between both constantly.

2.2 Installing Node.js

Node.js is a JavaScript runtime built on Chrome's V8 engine that lets you run JavaScript outside a browser. Installing it also installs **npm** (Node Package Manager), which you'll use starting in Chapter 31.

Recommended installation methods:

- **Official installer** — download the "LTS" (Long-Term Support) build from nodejs.org. LTS is more stable than "Current" and is what production servers use.
- **Version manager (recommended for developers)** — tools like `nvm` (Node Version Manager) let you install and switch between multiple Node versions per project:

```
# install nvm (one-time), then:
nvm install --lts
nvm use --lts
node --version    # e.g. v20.11.0
npm --version     # e.g. v10.2.4
```

Version managers matter because different projects often pin different Node versions, and reinstalling Node globally every time you switch projects is painful. This is enough setup detail for this book — a full DevOps/tooling guide is out of scope, but know that `nvm` (macOS/Linux) and `nvm-windows` / `fnm` (Windows) are the standard tools.

Verify your install:

```
node --version
npm --version
```

2.3 Running a script with Node.js

Create a file `hello.js`:

```
console.log("Hello, JavaScript!");
```

Run it from your terminal:

```
node hello.js
```

That's it — no compilation step, no build tool required for plain scripts. This is exactly how every example in this book's `examples/` folder is meant to be run: `node examples/NN-topic.js`.

2.4 The Node REPL

Typing `node` with no filename starts the **REPL** (Read-Eval-Print Loop), an interactive JavaScript shell:

```
$ node
> 1 + 1
2
> const greeting = "hi";
undefined
> greeting
'hi'
> .exit
```

The REPL is invaluable for quickly testing a one-liner without creating a file. Note that it prints `undefined` after statements that don't produce a value (like `const greeting = "hi"`), which surprises newcomers — that's normal, not an error.

Browsers have an equivalent: open DevTools (F12, or right-click → Inspect) and click the **Console** tab. You can type JavaScript directly there and it has access to the current page's `document`, `window`, and any scripts already loaded.

2.5 Choosing an editor: VS Code

Any text editor can write JavaScript, but **Visual Studio Code (VS Code)** is the de facto standard for JS/TS development because of its built-in IntelliSense (autocomplete), integrated terminal, and huge extension ecosystem. Recommended setup:

- Install VS Code from code.visualstudio.com.
- Extensions worth installing early:
 - **ESLint** — surfaces linting errors inline (see 2.7)
 - **Prettier** — auto-formats code on save
 - **Error Lens** — shows errors/warnings inline instead of only in a side panel
- Enable "Format on Save" in settings so you never think about formatting manually.

You don't need any of this to follow along with this book — a plain text editor and terminal are enough — but it will make the exercises faster.

2.6 `console` methods beyond `console.log`

`console.log` is the workhorse, but the `console` object has several other methods that make debugging much more pleasant:

```
console.log("info message");
console.warn("something looks off");    // yellow in most consoles
console.error("something broke");       // red, includes a stack trace

console.table([
  { name: "Alice", age: 30 },
  { name: "Bob", age: 25 },
]); // renders an actual table – extremely useful for arrays of objects

console.group("User validation");
console.log("checking name...");
console.log("checking email...");
console.groupEnd(); // indents the grouped logs, then closes the group

console.time("loop");
for (let i = 0; i < 1e6; i++) {} // do some work
console.timeEnd("loop"); // prints "loop: 1.234ms"

console.count("call"); // "call: 1"
console.count("call"); // "call: 2"

console.assert(1 === 2, "1 is not 2"); // only logs when the assertion is false
```

In browser DevTools, `console.error` and `console.trace` also capture a clickable stack trace pointing to exactly where they were called — one of the fastest ways to locate a bug's origin.

2.7 Linting and formatting (conceptual overview)

As programs grow, two categories of tooling become essential:

- **Linters** (e.g., **ESLint**) analyze your code for *likely bugs* and *style violations* without running it — catching things like unused variables, unreachable code, or accidentally using `==` instead of `===`, before you ever run the program.
- **Formatters** (e.g., **Prettier**) rewrite your code's whitespace, quote style, and line breaks to a consistent style automatically, so teams stop arguing about tabs vs. spaces in code review.

You don't need to configure either to follow this book, but know that virtually every professional JavaScript codebase runs both, usually wired into the editor (format/lint on save) and into CI (fail the build on lint errors). We revisit ESLint/Prettier configuration briefly in Chapter 31 alongside `package.json`.

2.8 Strict mode

`"use strict"` is a directive that opts your code into a stricter variant of JavaScript that catches common mistakes by throwing errors instead of failing silently or creating implicit globals:

```
"use strict";

x = 10; // ReferenceError: x is not defined
// without strict mode, this silently creates a global variable `x`
```

Key behavior differences under strict mode:

- Assigning to an undeclared variable throws instead of creating a global.
- Assigning to a read-only or non-writable property throws instead of failing silently.
- Duplicate parameter names (`function f(a, a) {}`) are a syntax error.
- `this` inside a plain function call is `undefined` instead of the global object (important for Chapter 15).

You rarely need to write `"use strict"` yourself in modern code because:

- **ES modules** (`import / export`, or `<script type="module">`) are **always** strict mode automatically.
- **Class bodies** are always strict mode automatically.

But you will see it at the top of older files and should recognize what it does when you do.

2.9 Including JavaScript in HTML

For browser-based work, JavaScript is attached to an HTML page with a `<script>` tag:

```

<!DOCTYPE html>
<html>
  <head>
    <title>My Page</title>
  </head>
  <body>
    <h1>Hello</h1>

    <!-- Option A: inline script -->
    <script>
      console.log("inline script ran");
    </script>

    <!-- Option B: external file -->
    <script src="app.js"></script>
  </body>
</html>

```

Where you place `<script>` and which attributes you use changes *when* your code runs relative to the HTML being parsed — this trips up beginners constantly, so it's worth being precise:

- **No attribute, placed in `<head>`** — the browser stops parsing HTML, downloads and *executes* the script immediately, then resumes parsing. If your script tries to access DOM elements that appear later in the HTML, they won't exist yet. This is why the old convention was "put your `<script>` tag right before `</body>`."
- **`defer`** — the script downloads in parallel with HTML parsing but only *executes* after the HTML is fully parsed, and multiple `defer` scripts run in the order they appear. This is the modern default for a page's main script.
- **`async`** — the script downloads in parallel and executes as soon as it's ready, potentially *before* HTML parsing finishes, and with no guaranteed order relative to other `async` scripts. Good for independent scripts like analytics that don't touch the DOM or depend on other scripts.
- **`type="module"`** — marks the script as an ES module (Chapter 24), enabling `import` / `export`. Module scripts are deferred by default and always run in strict mode.

```

<script src="app.js" defer></script>
<script src="analytics.js" async></script>
<script type="module" src="main.js"></script>

```

```

Head-blocking script:  [download][execute] ----parsing resumes---->
defer script:         [download in background] ----- [execute after parse]
async script:         [download in background] -- [execute immediately when ready]

```

2.10 A minimal project layout

Nothing fancy is required to start. A reasonable folder layout for following along with this book, or starting any small JS project:

```
my-project/
├─ index.html      (only needed for browser work)
├─ app.js          (or src/index.js for larger projects)
└─ package.json    (introduced in Chapter 31)
```

For every chapter from here on, assume you can create a file, write JavaScript in it, and run it with `node filename.js` unless the chapter explicitly says it needs a browser.

2.11 Chapter summary

- Node.js runs JavaScript outside the browser; install it via nodejs.org or a version manager like `nvm`, and verify with `node --version`.
- `node file.js` runs a script; plain `node` opens an interactive REPL.
- Browser DevTools' **Console** tab is the browser equivalent of the REPL.
- VS Code plus the ESLint and Prettier extensions is the most common professional setup, though any editor works for learning.
- `console` has many methods beyond `log`: `warn`, `error`, `table`, `group / groupEnd`, `time / timeEnd`, `count`, `assert`.
- Linters catch likely bugs statically; formatters enforce consistent style automatically.
- `"use strict"` catches silent mistakes by throwing errors instead; ES modules and classes are strict by default.
- `<script>` placement and the `defer / async / type="module"` attributes control *when* a script executes relative to HTML parsing.

2.12 Exercises

1. Install Node.js (or verify it's installed) and run `node examples/02-setting-up-environment.js`.
2. Open the Node REPL, and without creating a file, compute the sum of the numbers 1 through 10 using a `for` loop typed directly into the REPL.
3. Create a two-line HTML file with a `<script defer>` tag that tries to read `document.title` and log it — confirm it works even when the script tag is placed in `<head>`, then try it again without `defer` and see if the behavior changes for this simple example.

Chapter 3 — Variables and Data Types

3.1 Declaring variables: `var`, `let`, and `const`

JavaScript has three keywords for declaring a variable, and the differences between them are one of the most common interview questions and real-world bug sources in the language.

```
var a = 1;    // old (pre-ES6), function-scoped, hoisted and initialized to undefined
let b = 2;    // ES6+, block-scoped, hoisted but NOT initialized (temporal dead zone)
const c = 3;  // ES6+, block-scoped, must be initialized, cannot be reassigned
```

Rule of thumb for modern code: default to `const`. Use `let` only when you know the variable needs to be reassigned. Avoid `var` entirely.

3.2 `const` does not mean immutable

A very common misconception: `const` prevents **reassignment** of the variable binding, not **mutation** of the value it points to.

```
const arr = [1, 2, 3];
arr.push(4);    // fine – mutating the array's contents
console.log(arr); // [1, 2, 3, 4]

arr = [5, 6];    // TypeError: Assignment to constant variable.
```

```
const user = { name: "Alice" };
user.name = "Bob";    // fine – mutating a property
user.age = 30;        // fine – adding a property
console.log(user);    // { name: 'Bob', age: 30 }

user = { name: "Carol" }; // TypeError: Assignment to constant variable.
```

If you need a truly immutable object, use `Object.freeze()` (shallow freeze only — nested objects are still mutable):

```
const frozen = Object.freeze({ name: "Alice" });
frozen.name = "Bob"; // silently fails in sloppy mode, throws in strict mode
console.log(frozen.name); // "Alice" – unchanged
```

3.3 Scope: function-scoped vs. block-scoped

`var` is **function-scoped** — it's visible throughout the entire function it was declared in, ignoring block boundaries like `if` or `for`:

```
function varScopeDemo() {
  if (true) {
    var x = 10;
  }
  console.log(x); // 10 – leaked out of the if-block
}
```

`let` and `const` are **block-scoped** — confined to the nearest enclosing `{ }`:

```
function letScopeDemo() {
  if (true) {
    let y = 10;
  }
  console.log(y); // ReferenceError: y is not defined
}
```

This matters enormously in loops:

```
// Classic interview bug, using var:
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log("var i:", i), 0);
}
// prints "var i: 3" three times – all callbacks share the SAME `i`,
// and by the time they run, the loop has already finished with i === 3

for (let j = 0; j < 3; j++) {
  setTimeout(() => console.log("let j:", j), 0);
}
// prints "let j: 0", "let j: 1", "let j: 2" – each iteration of a `let`
// loop gets its OWN binding of j
```

We'll formalize why this happens with closures in Chapter 14, but the practical takeaway now is: **this is one of the strongest reasons to prefer `let / const` over `var`**.

3.4 Hoisting and the Temporal Dead Zone

Hoisting means variable and function *declarations* are processed before any code runs, conceptually "moved to the top" of their scope. But `var`, `let`, and `const` are hoisted differently:

```
console.log(varVariable); // undefined (declaration hoisted, not the assignment)
var varVariable = "hello";

console.log(letVariable); // ReferenceError: Cannot access 'letVariable' before initialization
let letVariable = "hello";
```

`let` and `const` ARE hoisted (the engine knows they exist), but they are not initialized until their declaration line executes. The gap between the start of the scope and the declaration line is called the **Temporal Dead Zone (TDZ)** — accessing the variable in that gap throws. This is a deliberate safety feature that catches use-before-declaration bugs that `var` would silently allow.

```
// Function declarations are fully hoisted, including their body:
sayHi(); // works! "Hi!"
function sayHi() { console.log("Hi!"); }

// Function EXPRESSIONS follow the variable's hoisting rules:
sayBye(); // TypeError: sayBye is not a function (var sayBye is hoisted as undefined)
var sayBye = function () { console.log("Bye!"); };
```

3.5 Redeclaration rules

```
var v = 1;
var v = 2; // fine – var allows redeclaration in the same scope

let l = 1;
let l = 2; // SyntaxError: Identifier 'l' has already been declared

const c = 1;
let c = 2; // SyntaxError – can't mix, same rule applies
```

This is another reason `let` / `const` catch bugs `var` doesn't: accidentally redeclaring a variable (e.g., a typo reusing a loop variable name) is an error instead of silent overwriting.

3.6 Primitive types

JavaScript has **seven primitive types**. Primitives are immutable and are compared and copied **by value**.

Type	Example	<code>typeof</code> result
Number	42, 3.14, -7	"number"
String	"hello"	"string"
Boolean	true, false	"boolean"

Type	Example	typeof result
undefined	undefined	"undefined"
null	null	"object" (a famous, permanent language bug)
Symbol	Symbol("id")	"symbol"
BigInt	9007199254740993n	"bigint"

undefined VS null

Both represent "no value," but with different intent:

- **undefined** — a variable has been declared but not assigned; a function with no explicit **return** returns **undefined**; accessing a non-existent object property returns **undefined**. This is JavaScript's own way of saying "nothing here."
- **null** — an intentional, explicit "no value," assigned by *you* (or by some APIs, like `document.getElementById` when nothing matches) to signal "this is deliberately empty."

```
let a;
console.log(a); // undefined – never assigned

let b = null;
console.log(b); // null – deliberately empty

console.log(typeof undefined); // "undefined"
console.log(typeof null);      // "object" <- historical bug in JS, kept for compatibility
console.log(null == undefined); // true (loose equality treats them as equal)
console.log(null === undefined); // false (different types)
```

Symbol

A **Symbol** is a unique, immutable primitive value, often used as a collision-free object property key:

```
const id1 = Symbol("id");
const id2 = Symbol("id");
console.log(id1 === id2); // false – every Symbol is unique, even with the same description

const user = {
  name: "Alice",
  [id1]: "hidden-internal-id-12345",
};
console.log(Object.keys(user)); // ["name"] – symbol keys don't show up in normal enumeration
```

BigInt

Regular numbers cannot safely represent integers beyond `Number.MAX_SAFE_INTEGER` ($2^{53} - 1$).

`BigInt` (suffix `n`) exists for arbitrary-precision integers:

```
console.log(Number.MAX_SAFE_INTEGER);    // 9007199254740991
console.log(Number.MAX_SAFE_INTEGER + 1); // 9007199254740992 (correct, barely)
console.log(Number.MAX_SAFE_INTEGER + 2); // 9007199254740992 (WRONG – should be 993, pr

const big = 9007199254740993n;
console.log(big + 1n); // 9007199254740994n – exact

// You cannot mix BigInt and Number in arithmetic directly:
// console.log(1n + 1); // TypeError: Cannot mix BigInt and other types
```

3.7 The object type (reference type)

Everything that isn't a primitive is an **object** — including plain objects `{}`, arrays `[]`, functions, `Date`, `Map`, `Set`, regular expressions, and more. Objects are compared and copied **by reference**, which is the single biggest behavioral difference from primitives.

```
console.log(typeof {});           // "object"
console.log(typeof []);           // "object" (arrays are objects!)
console.log(typeof function(){}); // "function" (a special callable object)
console.log(typeof new Date());   // "object"
console.log(Array.isArray([]));   // true – the reliable way to check for an array
```

3.8 Value semantics vs. reference semantics

This is one of the most important mental models in the entire language.

Primitives are copied by value — assigning or passing a primitive creates an independent copy:

```
let x = 10;
let y = x; // y gets a COPY of the value 10
y = 20;
console.log(x, y); // 10 20 – x is untouched
```

Objects are copied by reference — the variable doesn't hold the object itself, it holds a reference (like a pointer/address) to where the object lives in memory. Assigning an object copies the *reference*, not the object:

```
let obj1 = { value: 10 };
let obj2 = obj1; // obj2 points to the SAME object as obj1
obj2.value = 20;
console.log(obj1.value); // 20 – obj1 sees the change too, because
                        // obj1 and obj2 reference the same underlying object
```

```
obj1 ┌──┐
      │ └─> { value: 20 } (one object in memory, two references to it)
obj2 └──┘
```

This explains why comparing two objects with identical contents is `false`:

```
console.log({ a: 1 } === { a: 1 }); // false – different objects in memory
const a = { a: 1 };
const b = a;
console.log(a === b); // true – same reference
```

And it explains why passing an object into a function lets that function mutate the caller's data:

```
function addAdmin(user) {
  user.role = "admin"; // mutates the object the caller passed in
}
const me = { name: "Alice" };
addAdmin(me);
console.log(me); // { name: 'Alice', role: 'admin' } – changed!
```

We revisit this in depth when discussing shallow vs. deep copies in Chapter 9 (Objects) and immutability patterns in Chapter 19.

3.9 `typeof` and its quirks

```
typeof 42;           // "number"
typeof "hi";         // "string"
typeof true;         // "boolean"
typeof undefined;    // "undefined"
typeof null;         // "object" <- the famous bug, never fixed for backward comp
typeof Symbol();     // "symbol"
typeof 10n;          // "bigint"
typeof {};           // "object"
typeof [];           // "object" <- use Array.isArray() instead
typeof function(){}; // "function"
typeof class Foo {}; // "function" <- classes are functions under the hood (C
typeof NaN;          // "number" <- NaN is technically a number!
```

3.10 NaN : Not a Number (but typeof number)

NaN results from invalid numeric operations, and has one bizarre property: **it is the only value in JavaScript that is not equal to itself.**

```
console.log(0 / 0);           // NaN
console.log("abc" * 2);       // NaN
console.log(NaN === NaN);     // false !!
console.log(NaN == NaN);      // false !!

console.log(isNaN("abc"));    // true – but isNaN coerces its argument first
console.log(isNaN(undefined)); // true – undefined coerces to NaN too, misleading!
console.log(Number.isNaN("abc")); // false – does NOT coerce, and "abc" is not the NaN
console.log(Number.isNaN(NaN)); // true – correctly identifies actual NaN
console.log(Number.isNaN(undefined)); // false – correct, undefined isn't NaN, just coer
```

Rule: always prefer `Number.isNaN()` over the global `isNaN()`. The global version coerces its argument to a number first, producing false positives for values like `undefined`, `{}`, or `"hello"`.

To check for NaN reliably without either function:

```
const value = NaN;
console.log(value !== value); // true only for NaN – a classic trick, but Number.isNaN is
```

3.11 Checking a variable's type reliably

Because `typeof null === "object"` and `typeof [] === "object"`, a robust type-check function often needs `Object.prototype.toString`:

```
function getType(value) {
  if (value === null) return "null";
  if (Array.isArray(value)) return "array";
  return typeof value;
}

console.log(getType(null));           // "null"
console.log(getType([1, 2]));        // "array"
console.log(getType({}));            // "object"
console.log(getType(42));            // "number"

// The most precise tool, useful for Map, Set, Date, RegExp, etc:
console.log(Object.prototype.toString.call([])); // "[object Array]"
console.log(Object.prototype.toString.call(null)); // "[object Null]"
console.log(Object.prototype.toString.call(new Date())); // "[object Date]"
```

3.12 Chapter summary

- Prefer `const` by default, `let` when reassignment is needed, and avoid `var` in modern code.
- `const` prevents reassigning the *variable*, not mutating the *value*.
- `var` is function-scoped and hoisted with an `undefined` initial value; `let` / `const` are block-scoped and hoisted into a Temporal Dead Zone that throws if accessed before declaration.
- JavaScript has seven primitive types, compared/copied **by value**, and one object type, compared/copied **by reference**.
- `undefined` means "never assigned"; `null` means "intentionally empty."
- `typeof null` is `"object"` — a decades-old bug kept for compatibility; use `=== null` or `Array.isArray()` for reliable checks.
- `NaN` is never equal to itself; use `Number.isNaN()`, not the coercing global `isNaN()`.

3.13 Exercises

1. Predict what `console.log(a); let a = 5;` does, and why, before running it.
2. Write a function `deepEqualPrimitive(a, b)` that correctly returns `true` for `deepEqualPrimitive(NaN, NaN)` without using `Number.isNaN` twice (hint: `Object.is`).
3. Given `const list = [1, 2, 3]; const copy = list;`, explain in your own words why `copy.push(4)` also changes what `list` contains, and then run `examples/03-variables-and-data-types.js` to confirm.

Chapter 4 — Operators

4.1 Arithmetic operators

```
5 + 3;    // 8    addition (or string concatenation if either side is a string)
5 - 3;    // 2    subtraction
5 * 3;    // 15   multiplication
5 / 3;    // 1.6666666666666667 division
5 % 3;    // 2    remainder (modulo)
5 ** 3;   // 125  exponentiation (ES2016)

let n = 5;
n++;      // post-increment: returns old value, then increments -> n is now 6
n--;      // post-decrement: returns old value, then decrements -> n is now 5
++n;      // pre-increment: increments first, then returns new value -> n is now 6
--n;      // pre-decrement -> n is now 5
```

The difference between `n++` and `++n` only matters when you use the *result* of the expression:

```
let a = 5;
let b = a++; // b = 5 (old value), a is now 6
let c = 6;
let d = ++c; // d = 7 (new value), c is now 7
```

4.2 Assignment operators

```
let x = 10;
x += 5; // x = x + 5 -> 15
x -= 3; // x = x - 3 -> 12
x *= 2; // x = x * 2 -> 24
x /= 4; // x = x / 4 -> 6
x %= 4; // x = x % 4 -> 2
x **= 3; // x = x ** 3 -> 8
```

Logical assignment operators (ES2021)

These combine a logical operator with assignment, and — critically — they **only evaluate/assign the right-hand side when needed**, unlike a plain `|| / ??` assignment which always evaluates:

```

let a = null;
a ??= "default";      // assigns only if a is null or undefined -> a = "default"

let count = 0;
count ||= 10;         // assigns only if count is falsy -> count = 10 (0 is falsy!)

let config = { theme: "dark" };
config.theme &&= "light"; // assigns only if config.theme is truthy -> "light"

```

4.3 Comparison operators: `==` vs `===`

This is the single most important operator distinction in JavaScript.

- `===` (**strict equality**) compares value *and* type. No conversion happens. **Use this by default, always.**
- `==` (**loose equality**) converts (coerces) operands to the same type before comparing, following a specific, sometimes-surprising algorithm.

```

5 === 5;      // true
5 === "5";    // false -- different types
5 == "5";     // true -- "5" coerced to 5

null === undefined; // false
null == undefined;  // true -- special case, they're loosely equal to each other and nothing else

0 == false;    // true
0 == "";       // true
0 == "0";      // true
false == "0";  // true
"" == "0";     // false <- this one breaks the "transitive" intuition!

NaN == NaN;    // false -- NaN is never equal to anything, including itself

```

The `"" == "0"` result above (`false`) despite `0 == ""` and `0 == "0"` both being `true` is a classic demonstration of why loose equality is considered dangerous: it isn't even transitive in the way you'd expect from "equality." **Modern style guides and linters (ESLint's `eqeqeq` rule) ban `==` / `!=` entirely except for the one common idiom `x == null` (which conveniently checks for both `null` and `undefined` in one comparison).**

Relational comparisons (`<`, `>`, `<=`, `>=`) also coerce non-numeric operands:

```

"10" > "9";    // false! -- both are strings, compared lexicographically ('1' < '9')
10 > 9;         // true -- both are numbers
"10" > 9;       // true -- "10" coerced to number 10

```

4.4 Logical operators and short-circuiting

```
true && false; // false
true || false; // true
!true;         // false
```

`&&` and `||` don't just produce booleans — they return one of their **actual operands**, which is extremely commonly used in real code:

```
// && returns the first falsy operand, or the last operand if all are truthy
console.log(0 && "hello");           // 0 (first falsy value, stops here)
console.log(1 && "hello");           // "hello" (1 is truthy, so evaluate and return the next)
console.log("a" && "b" && "c");      // "c" (all truthy, returns the last)

// || returns the first truthy operand, or the last operand if all are falsy
console.log(0 || "default");         // "default" (0 is falsy, moves on)
console.log("value" || "default");   // "value" (first operand is truthy)
console.log(0 || "" || null || "last"); // "last" (all falsy until the end)
```

This is **short-circuiting**: the right-hand side is only evaluated if needed, which makes both operators useful for conditional execution, not just conditional values:

```
const user = { loggedIn: true, greet: () => console.log("Welcome!") };
user.loggedIn && user.greet(); // only calls greet() if loggedIn is truthy

function getUser() { return null; }
const name = getUser() || "Guest"; // classic (but flawed) default-value pattern
```

The `|| "Guest"` pattern above has a bug: it treats *any* falsy value (`0`, `""`, `false`) as "missing," not just `null` / `undefined`. That's exactly the gap the nullish coalescing operator closes.

4.5 Nullish coalescing (`??`) — ES2020

`??` returns the right-hand operand **only when the left side is `null` or `undefined`** — not for any other falsy value. This fixes the `||` default-value bug:

```

const count1 = 0;
console.log(count1 || 10); // 10 -- WRONG if 0 is a valid, meaningful value!
console.log(count1 ?? 10); // 0 -- correct, 0 is not null/undefined

const count2 = null;
console.log(count2 ?? 10); // 10 -- correctly falls back

function getConfig(options) {
  const retries = options.retries ?? 3; // 0 retries is respected; only null/undefined falls back
  return retries;
}
console.log(getConfig({ retries: 0 })); // 0
console.log(getConfig({})); // 3

```

Rule: use `??` for "give me a default if this is missing," and `||` only when you genuinely want to treat every falsy value as "missing." Note: `??` cannot be directly mixed with `&&/||` without parentheses — `a || b ?? c` is a `SyntaxError` by design, to prevent ambiguous precedence bugs.

4.6 Optional chaining (`?.`) — ES2020

`?.` short-circuits to `undefined` instead of throwing when accessing a property, calling a method, or indexing into something that is `null` or `undefined`:

```

const user = { profile: { name: "Alice" } };

console.log(user.profile?.name); // "Alice"
console.log(user.address?.city); // undefined -- no error, even though user.address is null
// console.log(user.address.city); // TypeError: Cannot read properties of undefined (reading 'city')

console.log(user.sayHi?.()); // undefined -- calls only if sayHi exists, else returns undefined
console.log(user.profile?.tags?.[0]); // undefined -- safe chained array access too

const arr = null;
console.log(arr?.[0]); // undefined, no crash

```

`?.` and `??` are often combined:

```

const city = user.address?.city ?? "Unknown";
console.log(city); // "Unknown"

```

4.7 The ternary (conditional) operator

```
const age = 20;
const category = age >= 18 ? "adult" : "minor";
console.log(category); // "adult"

// Ternaries can be chained, but this hurts readability past 1-2 levels:
const grade = 85;
const letter = grade >= 90 ? "A" : grade >= 80 ? "B" : grade >= 70 ? "C" : "F";
console.log(letter); // "B"
```

4.8 typeof, instanceof, in, and delete

```
typeof 42;           // "number"
typeof "hi";         // "string"

const now = new Date();
now instanceof Date; // true -- checks the prototype chain (Chapter 16)
[] instanceof Array; // true
[] instanceof Object; // true -- arrays are also objects

const obj = { name: "Alice" };
"name" in obj;           // true -- checks if the key exists (own OR inherited)
"toString" in obj;       // true -- inherited from Object.prototype
obj.hasOwnProperty("name"); // true -- checks OWN properties only, not inherited

delete obj.name;         // removes the property, returns true on success
console.log(obj);        // {}
```

4.9 Bitwise operators

Less commonly used in everyday application code, but important for flags, low-level work, and certain performance tricks:

```
5 & 1;    // 1  AND  (0101 & 0001 = 0001)
5 | 1;    // 5  OR   (0101 | 0001 = 0101)
5 ^ 1;    // 4  XOR  (0101 ^ 0001 = 0100)
~5;       // -6 NOT  (inverts all bits)
5 << 1;   // 10 left shift (multiply by 2)
5 >> 1;   // 2  right shift (divide by 2, floor)
-5 >>> 1; // large positive number -- unsigned right shift, ignores sign bit

// A common trick: double bitwise NOT truncates to a 32-bit integer, faster
// than Math.floor for positive numbers (rarely necessary, but seen in the wild):
~~4.7;    // 4
```

4.10 Operator precedence

When operators combine, precedence determines evaluation order (higher precedence binds tighter, evaluated first). You don't need to memorize the full table, but know the common surprises:

```
2 + 3 * 4;           // 14, not 20 -- * binds tighter than +
(2 + 3) * 4;         // 20 -- parentheses override precedence

true || false && false; // true -- && binds tighter than ||, evaluates as: true || (false && false)

typeof 1 + 1;         // "number1" !! -- typeof (higher precedence) runs first: (typeof 1) + 1

let a, b;
a = b = 5;            // right-associative: b = 5, then a = (b's value) -> both are 5
```

When in doubt, use parentheses. They cost nothing and remove all ambiguity for the next reader (including future you).

4.11 Chapter summary

- Prefer `===` / `!==` over `==` / `!=` always, except the `x == null` idiom.
- `&&` and `||` return operand values, not just booleans, and short-circuit — this powers common conditional-execution and default-value idioms.
- `??` is the correct default-value operator when `0`, `""`, or `false` should be treated as valid values (unlike `||`).
- `?.` prevents "Cannot read properties of undefined/null" errors by short-circuiting property/method/index access.
- `instanceof` checks the prototype chain; `in` checks for a key (including inherited ones); `hasOwnProperty` checks own keys only.
- When precedence is unclear, add parentheses — it's free and unambiguous.

4.12 Exercises

1. Without running it, predict the output of `"5" + 3 - 1` and `5 + 3 + "1"`, then verify with the example file.
2. Rewrite `const x = options.timeout || 5000;` to correctly allow a timeout of `0`, using the right operator.
3. Given `const user = {};`, write one line using `?.` and `??` together that safely reads `user.settings.theme` and falls back to `"light"`.

Chapter 5 — Control Flow

5.1 `if` / `else if` / `else`

```
const hour = 14;

if (hour < 12) {
  console.log("Good morning");
} else if (hour < 18) {
  console.log("Good afternoon");
} else {
  console.log("Good evening");
}
```

Conditions are evaluated for **truthiness**, not just booleans (see 5.3), so any value can appear in an `if`:

```
const username = "";
if (username) {
  console.log(`Hello, ${username}`);
} else {
  console.log("No username provided"); // runs, because "" is falsy
}
```

5.2 Guard clauses

A **guard clause** exits a function early for invalid or edge-case input, avoiding deep nesting. This is widely considered better style than wrapping the "happy path" in a big `if`:

```
// Nested style – harder to read as conditions grow
function processOrderNested(order) {
  if (order) {
    if (order.items.length > 0) {
      if (order.paid) {
        return "Processing order";
      } else {
        return "Order not paid";
      }
    } else {
      return "Empty order";
    }
  } else {
    return "No order provided";
  }
}

// Guard clause style – flat, reads top-to-bottom
function processOrder(order) {
  if (!order) return "No order provided";
  if (order.items.length === 0) return "Empty order";
  if (!order.paid) return "Order not paid";

  return "Processing order";
}
```

5.3 Truthy and falsy values

JavaScript coerces any value used in a boolean context (`if` , `&&` , `||` , `!` , ternary conditions) to `true` or `false` . There are exactly **eight** falsy values — everything else is truthy:

```
false
0
-0
0n          (BigInt zero)
""          (empty string)
null
undefined
NaN
```

Everything else — including `"0"` (a non-empty string!), `[]` (an empty array!), and `{}` (an empty object!) — is **truthy**:

```
if ("0") console.log("truthy: the string \"0\""); // runs
if ([]) console.log("truthy: empty array");      // runs
if ({}) console.log("truthy: empty object");     // runs
if (" ") console.log("truthy: a string with just a space"); // runs
```

This trips up developers coming from languages where empty arrays/objects are falsy. In JavaScript, **any object reference is always truthy**, no matter what it contains.

5.4 `switch`

```
const day = "TUE";

switch (day) {
  case "MON":
  case "TUE":
  case "WED":
  case "THU":
  case "FRI":
    console.log("Weekday");
    break;
  case "SAT":
  case "SUN":
    console.log("Weekend");
    break;
  default:
    console.log("Invalid day");
}
```

`switch` compares with **strict equality** (`===`), no coercion — so `switch ("5") { case 5: ... }` will NOT match.

The fallthrough trap

Without a `break`, execution **falls through** into the next case, regardless of whether its condition matches. This is either a deliberate tool (as with the grouped cases above) or a classic bug:

```
function describe(n) {
  let result = "";
  switch (n) {
    case 1:
      result += "one ";
      // no break! falls through into case 2
    case 2:
      result += "two ";
      break;
    case 3:
      result += "three";
      break;
  }
  return result;
}

console.log(describe(1)); // "one two " -- probably not intended
console.log(describe(2)); // "two "
```

Rule: always add `break` (or `return`) at the end of every case unless the fallthrough is intentional and commented. Many linters flag missing `break` statements by default for exactly this reason.

5.5 `switch` vs. object lookup vs. `if / else if`

For simple "match a value, return a result" logic, an object literal used as a lookup table is often clearer and avoids the fallthrough trap entirely:

```
const dayType = {
  MON: "Weekday", TUE: "Weekday", WED: "Weekday",
  THU: "Weekday", FRI: "Weekday",
  SAT: "Weekend", SUN: "Weekend",
};
console.log(dayType["TUE"] ?? "Invalid day"); // "Weekday"
```

Use `if / else if` when conditions involve ranges or multiple variables (`switch` only compares one value); use `switch` when you have many exact matches against a single value and want the branch grouping to read clearly; use an object/`Map` lookup when the logic is a pure "input -> output" mapping with no side effects per branch.

5.6 Ternary vs. `if / else`: style tradeoffs

```
// Ternary – good for short, single-expression results assigned to a variable
const status = isActive ? "Active" : "Inactive";

// if/else – better when branches have multiple statements or side effects
let status2;
if (isActive) {
  logActivation();
  status2 = "Active";
} else {
  logDeactivation();
  status2 = "Inactive";
}
```

Rule of thumb: use a ternary only when both branches are simple expressions and the whole thing reads on roughly one line. Once you need multiple statements, side effects, or nested ternaries, switch to `if / else` for readability. Nested ternaries in particular (`a ? b : c ? d : e`) are a common code review complaint — more than one level deep, most style guides prefer `if / else if` or a lookup table.

5.7 Common control-flow bugs

A handful of mistakes account for the vast majority of real-world control-flow bugs. Recognizing them on sight is worth more than any amount of theory:

```
// BUG 1: assignment instead of comparison
let isReady = false;
// if (isReady = true) { ... } // always runs! assigns true, then evaluates it as truthy
if (isReady === true) {
  console.log("ready");
}

// BUG 2: comparing against the wrong falsy-adjacent value
function processCount(count) {
  if (!count) {
    // WRONG if count === 0 is a valid, meaningful input – 0 is falsy!
    return "no count given";
  }
  return `count is ${count}`;
}
console.log(processCount(0)); // "no count given" -- probably a bug if 0 is valid data

function processCountFixed(count) {
  if (count === undefined || count === null) {
    return "no count given";
  }
  return `count is ${count}`;
}
console.log(processCountFixed(0)); // "count is 0" -- correct

// BUG 3: off-by-one in range checks
function isValidIndex(arr, index) {
  // return index >= 0 && index <= arr.length; // WRONG: allows arr.length, one past the
  return index >= 0 && index < arr.length; // correct: last valid index is length - 1
}
console.log(isValidIndex([1, 2, 3], 3)); // false -- correctly rejects the out-of-bounds

// BUG 4: forgetting that else-if chains stop at the first match
function categorize(score) {
  if (score >= 0) return "non-negative"; // matches EVERYTHING >= 0, including 95!
  if (score >= 90) return "excellent"; // unreachable – dead code
  return "other";
}
console.log(categorize(95)); // "non-negative" -- the intended "excellent" branch never r
// Fix: order conditions from most specific to least specific
function categorizeFixed(score) {
  if (score >= 90) return "excellent";
  if (score >= 0) return "non-negative";
  return "other";
}
console.log(categorizeFixed(95)); // "excellent"
```

Rule: when writing a chain of `if / else if` range checks, order the conditions from most specific (narrowest range) to least specific (widest range) — otherwise a broad early condition silently swallows the cases meant for later branches.

5.8 Multi-value comparisons without a long `||` chain

Comparing one value against several possibilities is common enough to deserve its own idiom. The naive approach repeats the variable name for every comparison:

```
function isWeekendVerbose(day) {
  return day === "SAT" || day === "SUN";
}
```

For three or more values, `Array.prototype.includes()` is shorter and scales better without repeating the variable:

```
function isWeekend(day) {
  return ["SAT", "SUN"].includes(day);
}

function isVowel(char) {
  return ["a", "e", "i", "o", "u"].includes(char.toLowerCase());
}

console.log(isWeekend("SUN"), isVowel("E")); // true true
```

This reads as "is `day` one of these values?" directly, and adding a new value to check against is a one-word edit to the array instead of adding a whole new `||` clause. The tradeoff: `.includes()` allocates a new array on every call, which matters only in extremely hot loops (Chapter 34) — for everyday code, prefer the version that reads most clearly.

5.9 Chapter summary

- `if` conditions coerce any value to boolean via truthy/falsy rules — memorize the eight falsy values: `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, `NaN`. Everything else, including `[]` and `{}`, is truthy.
- Guard clauses (`if (!valid) return;`) flatten nested conditionals and are generally preferred over deep `if / else` nesting.
- `switch` uses strict equality and **falls through** without `break` — always terminate cases explicitly unless the fallthrough is intentional.
- For pure value-to-value mappings, an object or `Map` lookup is often clearer than `switch`.
- Use ternaries for short, single-expression conditionals; use `if / else` once branches need multiple statements or side effects.

5.10 Exercises

1. List which of these are truthy: `"false"`, `0`, `"0"`, `null`, `[]`, `NaN`, `-1`. Check your answers by running `examples/05-control-flow.js`.
2. Rewrite a `switch` statement that maps HTTP status codes 200, 201, 204 to `"success"`, 400, 404 to `"client error"`, and 500 to `"server error"`, defaulting to `"unknown"`, as an object lookup instead.
3. Take the nested `processOrderNested` function from this chapter and rewrite it with guard clauses without looking at the example first.

Chapter 6 — Loops and Iteration

6.1 The classic `for` loop

```
for (let i = 0; i < 5; i++) {  
  console.log(i); // 0, 1, 2, 3, 4  
}
```

The three clauses — initialization, condition, increment — are all optional, giving you full control:

```
// counting down  
for (let i = 5; i > 0; i--) {  
  console.log(i);  
}  
  
// multiple variables  
for (let i = 0, j = 10; i < j; i++, j--) {  
  console.log(i, j);  
}  
  
// infinite loop (needs a break inside)  
for (;;) {  
  console.log("runs once");  
  break;  
}
```

6.2 `while` and `do...while`

```
let count = 0;  
while (count < 3) {  
  console.log("while:", count);  
  count++;  
}  
  
let n = 0;  
do {  
  console.log("do-while:", n); // runs at least once, even if the condition is false  
  n++;  
} while (n < 3);  
  
let m = 10;  
do {  
  console.log("this still runs once:", m);  
} while (m < 5); // condition is false immediately, but the body already ran
```

do...while guarantees the body runs at least once, because the condition is checked *after* the first iteration — useful for "do the work, then decide whether to repeat" logic like menu prompts or retry loops.

6.3 **for...of** — iterating values

for...of iterates over the **values** of any *iterable* — arrays, strings, **Map**, **Set**, and anything implementing the iterator protocol (Chapter 20). This is almost always what you want for arrays:

```
const fruits = ["apple", "banana", "cherry"];
for (const fruit of fruits) {
  console.log(fruit); // "apple", "banana", "cherry"
}

for (const char of "hi") {
  console.log(char); // "h", "i"
}

const uniqueNums = new Set([1, 2, 2, 3]);
for (const num of uniqueNums) {
  console.log(num); // 1, 2, 3 (duplicates already removed by Set)
}

const map = new Map([["a", 1], ["b", 2]]);
for (const [key, value] of map) {
  console.log(key, value); // "a" 1, then "b" 2
}

// Getting the index too, via entries():
for (const [index, fruit] of fruits.entries()) {
  console.log(index, fruit); // 0 "apple", 1 "banana", 2 "cherry"
}
```

6.4 **for...in** — iterating keys

for...in iterates over the **enumerable property keys** of an object (including inherited ones from the prototype chain). It exists for objects, not arrays:

```
const user = { name: "Alice", age: 30 };
for (const key in user) {
  console.log(key, user[key]); // "name" "Alice", then "age" 30
}
```

Why not to use **for...in on arrays**

```
const arr = ["a", "b", "c"];
for (const index in arr) {
  console.log(typeof index, index); // "string" "0", "string" "1", "string" "2" -- indices
}

// Worse: it also picks up inherited/enumerable properties added to Array.prototype or th
Array.prototype.extra = "oops"; // pollutes every array (never do this in real code!)
for (const key in arr) {
  console.log(key); // "0", "1", "2", "extra" <- unexpected!
}
delete Array.prototype.extra; // clean up
```

Rule: use `for...of` (or array methods like `.forEach` / `.map`) for arrays. Use `for...in` only for plain objects, and even then, prefer `Object.keys()` / `Object.entries()` with `for...of`, which sidesteps the inherited-properties issue entirely:

```
for (const key of Object.keys(user)) {
  console.log(key);
}
for (const [key, value] of Object.entries(user)) {
  console.log(key, value);
}
```

6.5 `break` and `continue`

```
for (let i = 0; i < 10; i++) {
  if (i === 5) break; // exits the loop entirely
  console.log("break demo:", i); // 0, 1, 2, 3, 4
}

for (let i = 0; i < 5; i++) {
  if (i % 2 === 0) continue; // skips the rest of THIS iteration, moves to next
  console.log("continue demo:", i); // 1, 3
}
```

Labeled loops

A **label** lets `break` / `continue` target an *outer* loop from inside a nested one — useful for exiting nested loops in one step instead of juggling flag variables:

```

outer: for (let i = 0; i < 3; i++) {
  for (let j = 0; j < 3; j++) {
    if (j === 1) continue outer; // skips to the NEXT iteration of the outer loop
    console.log(i, j);
  }
}

search: for (let i = 0; i < 3; i++) {
  for (let j = 0; j < 3; j++) {
    if (i === 1 && j === 1) break search; // breaks out of BOTH loops at once
    console.log("searching", i, j);
  }
}

```

Labels are uncommon in everyday code (they have a reputation for reading like `goto`), but are the cleanest tool for this specific "break out of nested loops" problem — better than a boolean flag checked in every iteration.

6.6 Iterating `Map` and `Set`

```

const scores = new Map([["Alice", 90], ["Bob", 85]]);
for (const [name, score] of scores) {
  console.log(name, score);
}
scores.forEach((score, name) => console.log(name, score)); // note: (value, key) order!

const tags = new Set(["js", "web", "js"]);
for (const tag of tags) {
  console.log(tag); // "js", "web" -- duplicate silently ignored by Set
}

```

6.7 Performance notes

For the vast majority of code, loop performance differences are negligible compared to what the loop *body* does (network calls, DOM updates, allocations). Some practical notes, not premature-optimization advice:

- A classic `for` loop with a numeric index is typically the fastest option in any engine, but the difference vs. `for...of` or `.forEach()` is rarely meaningful outside hot paths processing millions of elements.
- Caching `array.length` in a variable before a loop (`const len = arr.length;`) mattered on very old engines; modern V8 optimizes `.length` access on arrays effectively, so this is no longer necessary for correctness or performance in typical code.

- `.forEach()`, `.map()`, `.filter()` etc. (Chapter 8) cannot be exited early with `break` — if you need early exit, use a `for...of` loop with `break`, or `.some()` / `.every()` / `.find()` which stop as soon as their condition is satisfied.
- Prefer readability first. Only reach for manual loop-optimization once you've profiled and confirmed a loop is actually a bottleneck.

6.8 Common infinite-loop bugs

```
// BUG: forgot to increment i
// for (let i = 0; i < 5;) {
//   console.log(i); // loops forever, i never changes
// }

// BUG: condition never becomes false because of a typo
// let i = 0;
// while (i = 5) { // assignment (=) instead of comparison (===) -- always truthy!
//   console.log(i);
// }

// BUG: mutating the wrong variable
// for (let i = 0; i < 5; j++) { // increments j, not i -- i stays 0 forever
//   console.log(i);
// }
```

All three are extremely common typos. Most linters flag assignment inside a condition (`while (i = 5)`) by default — another good reason to run a linter (Chapter 2.7) from day one.

6.9 Chapter summary

- `for`, `while`, and `do...while` give manual control over iteration; `do...while` always runs its body at least once.
- `for...of` iterates **values** of any iterable (arrays, strings, `Map`, `Set`) and is the default choice for arrays.
- `for...in` iterates **enumerable keys**, including inherited ones — use it for plain objects only, never arrays; prefer `Object.keys()` / `Object.entries()` with `for...of` for safety.
- `break` exits a loop; `continue` skips to the next iteration; labels let both target an outer loop from a nested one.
- Loop performance rarely matters compared to what's inside the loop body — optimize for readability first.

6.10 Exercises

1. Given `const nums = [1, 2, 3, 4, 5, 6];`, write a `for...of` loop that logs only even numbers using `continue`.
2. Write a nested loop over a 3×3 grid that breaks out of both loops entirely as soon as it finds coordinates where `x === y === 1`, using a label.
3. Rewrite a `for...in` loop over an array into a safer `for...of` loop using `.entries()`, and explain in a comment why the original was risky.

Chapter 7 — Functions

7.1 Three ways to write a function

```
// Function declaration
function add(a, b) {
  return a + b;
}

// Function expression (anonymous, assigned to a variable)
const subtract = function (a, b) {
  return a - b;
};

// Named function expression (name only visible inside its own body, useful for recursion)
const multiply = function multiplyImpl(a, b) {
  return a - 1 === 0 ? a : a * multiplyImpl(a - 1, b); // silly example just to show self
};

// Arrow function (ES6)
const divide = (a, b) => a / b;
```

These are not just stylistic variants — they differ in **hoisting**, **this binding**, and the **arguments object**, all covered below.

7.2 Parameters, arguments, return values

```
function greet(name) {
  return `Hello, ${name}`;
}
console.log(greet("Alice")); // "Hello, Alice"

// Calling with too few/many arguments never throws in JavaScript –
// missing parameters are simply undefined, extra arguments are ignored
// (unless you use the arguments object or rest parameters to access them):
console.log(greet()); // "Hello, undefined"
console.log(greet("Alice", "Bob")); // "Hello, Alice" -- "Bob" is silently dropped

function noReturn() {
  console.log("did something");
}
console.log(noReturn()); // undefined -- functions without a `return` return undefined
```

7.3 Default parameters (ES6)

```
function greetWithDefault(name = "Guest") {
  return `Hello, ${name}`;
}
console.log(greetWithDefault());           // "Hello, Guest"
console.log(greetWithDefault(undefined)); // "Hello, Guest" -- undefined triggers the def
console.log(greetWithDefault(null));       // "Hello, null" -- null does NOT trigger the
console.log(greetWithDefault("Bob"));      // "Hello, Bob"

// Defaults can reference earlier parameters:
function makeRange(start, end = start + 10) {
  return [start, end];
}
console.log(makeRange(5)); // [5, 15]
```

Only `undefined` triggers a default parameter — `null` is treated as an explicit, deliberate value. This is the same rule nullish coalescing (`??`) follows, and it's intentional consistency in the language.

7.4 Rest parameters

The rest parameter (`...args`) collects any number of trailing arguments into a real array:

```
function sum(...numbers) {
  return numbers.reduce((total, n) => total + n, 0);
}
console.log(sum(1, 2, 3));           // 6
console.log(sum(1, 2, 3, 4, 5));    // 15
console.log(sum());                  // 0

function logFirstAndRest(first, ...rest) {
  console.log("first:", first, "rest:", rest);
}
logFirstAndRest(1, 2, 3, 4); // first: 1 rest: [ 2, 3, 4 ]
```

Rest must be the **last** parameter — `function f(...a, b) {}` is a syntax error.

7.5 The `arguments` object

Every regular function (not arrow functions) has access to an array-like `arguments` object containing all passed arguments, regardless of the declared parameters:

```
function showArgs() {
  console.log(arguments);           // Arguments(3) [1, 2, 3]
  console.log(arguments.length);    // 3
  console.log(arguments[0]);        // 1
}
showArgs(1, 2, 3);
```

`arguments` is **array-like, not a real array** — it has `indices` and `.length`, but not `.map()`, `.filter()`, etc. directly:

```
function showArgsArray() {
  // arguments.map(x => x * 2); // TypeError: arguments.map is not a function
  const realArray = Array.from(arguments); // or [...arguments]
  return realArray.map((x) => x * 2);
}
console.log(showArgsArray(1, 2, 3)); // [2, 4, 6]
```

Rest parameters are the modern replacement for `arguments` — they give you a real array directly and work in arrow functions (where `arguments` doesn't exist at all — see 7.6). Prefer `...args` in new code.

7.6 Arrow functions: the real differences

Arrow functions aren't just shorter syntax. They differ from regular functions in three fundamental ways:

1. No own `this` — arrows inherit `this` from their enclosing scope

```
const obj = {
  name: "Timer",
  startRegular: function () {
    setTimeout(function () {
      console.log("regular function this.name:", this?.name); // undefined -- `this` is 1
    }, 0);
  },
  startArrow: function () {
    setTimeout(() => {
      console.log("arrow function this.name:", this.name); // "Timer" -- inherits `this`
    }, 0);
  },
};
obj.startRegular();
obj.startArrow();
```

This is *the* main reason arrow functions exist and the main reason to choose one over a regular function — full explanation in Chapter 15.

2. No `arguments` object

```
const arrowNoArgs = () => {
  // console.log(arguments); // ReferenceError (in a module) or refers to an OUTER
  //                          // function's `arguments` if nested inside one
};
```

Use rest parameters (`...args`) in arrow functions instead.

3. Cannot be used as a constructor

```
const Person = (name) => { this.name = name; };
// new Person("Alice"); // TypeError: Person is not a constructor
```

Arrow functions have no internal `[[Construct]]` behavior — `new` simply doesn't work on them. Use a regular function or a `class` (Chapter 17) when you need something instantiable with `new`.

Arrow function syntax shortcuts

```
const square = (x) => x * x;           // implicit return, no braces needed
const add2 = (a, b) => a + b;          // multiple params still need parens
const noop = () => {};                 // empty body, explicit return undefined
const makeObj = () => ({ key: "value" }); // parens needed to return an object literal
                                        // (otherwise {} is parsed as a function B
const identity = (x) => x;              // single param, parens optional: x => x a
```

7.7 Function hoisting

```
console.log(hoisted()); // "I work!" -- function declarations are FULLY hoisted (body inc
function hoisted() {
  return "I work!";
}

// console.log(notHoisted()); // TypeError: notHoisted is not a function
var notHoisted = function () {
  return "I don't work yet";
};
// (var is hoisted as undefined; the assignment only happens at this line)
```

7.8 Immediately Invoked Function Expressions (IIFEs)

An IIFE runs a function the instant it's defined, creating a private scope. Before ES6 modules and `let / const` block scoping existed, this was the standard way to avoid polluting the global scope:

```

(function () {
  const privateVar = "not visible outside";
  console.log("IIFE ran:", privateVar);
})();

// Arrow function IIFE
(() => {
  console.log("arrow IIFE ran");
})();

// IIFEs that return a value (the "module pattern")
const counter = (function () {
  let count = 0;
  return {
    increment: () => ++count,
    reset: () => (count = 0),
  };
})();
console.log(counter.increment()); // 1
console.log(counter.increment()); // 2
console.log(counter.reset());      // 0

```

IIFEs are less common today (ES modules give every file its own scope automatically — Chapter 24), but you'll still see them in bundled library code and older codebases, and the module pattern above is a direct ancestor of the closure-based patterns in Chapter 14.

7.9 First-class functions

In JavaScript, functions are **values** — they can be stored in variables, passed as arguments, returned from other functions, and stored in data structures, exactly like numbers or strings. This is called "functions as first-class citizens" and it's the foundation of callbacks, array methods (Chapter 19), and much of JavaScript's expressive power.

```
// Stored in a variable – already shown throughout this chapter

// Passed as an argument ("callback")
function processArray(arr, callback) {
  const result = [];
  for (const item of arr) {
    result.push(callback(item));
  }
  return result;
}
console.log(processArray([1, 2, 3], (x) => x * 10)); // [10, 20, 30]

// Returned from a function ("higher-order function," full chapter: 19)
function makeMultiplier(factor) {
  return function (x) {
    return x * factor;
  };
}
const triple = makeMultiplier(3);
console.log(triple(7)); // 21

// Stored in a data structure
const operations = {
  add: (a, b) => a + b,
  subtract: (a, b) => a - b,
};
console.log(operations["add"](2, 3)); // 5
```

7.10 Pure functions vs. side effects

A **pure function** always returns the same output for the same input and causes no observable side effects (no mutating external state, no I/O, no relying on anything outside its parameters):

```
// Pure – same input always gives same output, touches nothing outside itself
function addPure(a, b) {
  return a + b;
}

// Impure – depends on external mutable state
let taxRate = 0.1;
function addTaxImpure(price) {
  return price + price * taxRate; // result changes if taxRate changes elsewhere
}

// Impure – mutates its argument (a side effect visible to the caller)
function addItemImpure(cart, item) {
  cart.push(item); // mutates the array the caller passed in
  return cart;
}

// Pure equivalent – returns a new array instead of mutating
function addItemPure(cart, item) {
  return [...cart, item];
}
```

Pure functions are easier to test (no setup/mockings needed), easier to reason about (no hidden dependencies), safe to run in any order or in parallel, and are the foundation of predictable state management in UI frameworks like React. You won't write 100% pure code in practice — I/O, logging, and DOM updates are inherently side effects — but **isolating your side effects at the edges of your program, and keeping the core logic pure, is one of the most valuable habits in software engineering.**

7.11 Recursion

A function that calls itself, with a **base case** that stops the recursion:

```
function factorial(n) {
  if (n <= 1) return 1; // base case
  return n * factorial(n - 1); // recursive case
}
console.log(factorial(5)); // 120

function fibonacci(n) {
  if (n <= 1) return n; // base case
  return fibonacci(n - 1) + fibonacci(n - 2);
}
console.log(fibonacci(10)); // 55
// Warning: this naive version is exponential time – recomputes the same
// values repeatedly. Chapter 19 covers memoization to fix this.

// Recursion is natural for tree-shaped data:
function sumNestedArray(arr) {
  let total = 0;
  for (const item of arr) {
    total += Array.isArray(item) ? sumNestedArray(item) : item;
  }
  return total;
}
console.log(sumNestedArray([1, [2, 3], [4, [5, 6]]])); // 21
```

Every recursive call adds a frame to the **call stack**. Too many levels of recursion without a base case (or with a base case that's never reached) throws `RangeError: Maximum call stack size exceeded`:

```
function infiniteRecursion() {
  return infiniteRecursion(); // no base case!
}
// infiniteRecursion(); // RangeError: Maximum call stack size exceeded
```

Unlike some languages, standard JavaScript engines do **not** guarantee tail-call optimization in practice (it's in the ES6 spec but essentially unimplemented in V8), so deep recursion (tens of thousands of levels) on large inputs is a real risk — an iterative loop is often safer for unbounded-depth problems.

7.12 Chapter summary

- Function declarations are fully hoisted; function expressions follow the hoisting rules of the variable they're assigned to.
- Default parameters trigger only on `undefined`, not `null`; rest parameters (`...args`) collect trailing arguments into a real array and are the modern replacement for the array-like `arguments` object.
- Arrow functions differ from regular functions in three real ways: they inherit `this` from their enclosing scope, have no `arguments` object, and cannot be used as constructors.

- Functions are first-class values in JavaScript — they can be stored, passed, and returned like any other value, enabling callbacks and higher-order functions.
- Pure functions (same input → same output, no side effects) are easier to test and reason about; isolate side effects where you can.
- Recursive functions need a reachable base case; JavaScript engines generally don't optimize tail calls, so very deep recursion can overflow the call stack.

7.13 Exercises

1. Write `makeAdder(x)` that returns a function adding `x` to whatever it's called with, so `makeAdder(5)(3)` returns `8`.
2. Convert this function to an arrow function and explain, in a comment, whether the conversion changes its behavior when used as an object method: `function sayName() { return this.name; }`.
3. Write a recursive function `flattenDeep(arr)` that flattens an arbitrarily nested array (like the `sumNestedArray` example, but returning a flat array instead of a sum).

Chapter 8 — Arrays

8.1 What is an array?

An array is an ordered, index-based collection of values. In JavaScript, arrays are a special kind of object — `typeof []` returns `"object"` — but they have integer-keyed elements, an automatically-maintained `length` property, and a large built-in set of methods (`Array.prototype`) tuned for working with sequences of data.

```
const fruits = ["apple", "banana", "cherry"];
console.log(fruits[0]);      // "apple"
console.log(fruits.length);  // 3
console.log(typeof fruits);  // "object"
console.log(Array.isArray(fruits)); // true – the reliable type check
```

Arrays can hold values of any type, including mixed types and other arrays:

```
const mixed = [1, "two", true, null, { id: 4 }, [5, 6]];
```

This is legal but rarely a good idea outside of quick scripts — mixed-type arrays make code harder to reason about and defeat engine optimizations that assume a consistent element type (see §8.9).

8.2 Creating arrays

```
const a = [1, 2, 3];           // array literal – always prefer this
const b = new Array(1, 2, 3);   // equivalent, but avoid the constructor
const c = new Array(5);         // DANGER: creates a sparse array of
                                // length 5, NOT [5]. Classic footgun.
const d = Array.from({ length: 5 }, (_, i) => i * 2); // [0,2,4,6,8]
const e = Array.of(5);         // [5] – Array.of fixes the Array() footgun
const f = [...a, ...b];         // spread-based construction
```

`Array.from` is one of the most useful array constructors: it converts *array-like* or *iterable* values (strings, `Set` s, `Map` s, `NodeList` s, arguments objects) into real arrays, optionally applying a mapping function as the second argument.

```
Array.from("hello");           // ["h","e","l","l","o"]
Array.from(new Set([1, 2, 2, 3])); // [1, 2, 3]
Array.from({ length: 3 }, (_, i) => i ** 2); // [0, 1, 4]
```

8.3 Indexing, length, and out-of-bounds access

Arrays are zero-indexed. Reading an index that doesn't exist returns `undefined` rather than throwing:

```
const arr = [10, 20, 30];
console.log(arr[10]);      // undefined — no error
console.log(arr[-1]);     // undefined — negative indices are NOT
                          // supported by bracket access (unlike Python)
console.log(arr.at(-1));  // 30 — Array.prototype.at() DOES support
                          // negative indices (ES2022)
```

`length` is not read-only — setting it truncates or extends the array:

```
const nums = [1, 2, 3, 4, 5];
nums.length = 3;
console.log(nums); // [1, 2, 3]

nums.length = 5;
console.log(nums); // [1, 2, 3, <2 empty items>] — sparse "holes", not undefined
```

8.4 Mutating methods (change the original array)

These methods modify the array in place and (mostly) return something other than the new array itself — a common trap for developers who expect a new array back.

Method	Effect	Returns
<code>push(...items)</code>	adds to the end	new length
<code>pop()</code>	removes from the end	removed element
<code>unshift(...items)</code>	adds to the start	new length
<code>shift()</code>	removes from the start	removed element
<code>splice(start, deleteCount, ...items)</code>	removes/inserts at any index	array of removed elements
<code>sort(compareFn)</code>	sorts in place	the same array (sorted)
<code>reverse()</code>	reverses in place	the same array (reversed)
<code>fill(value, start, end)</code>	fills range with a value	the same array
<code>copyWithin(target, start, end)</code>	copies part of array to another index	the same array

```
const stack = [1, 2, 3];
stack.push(4);           // [1, 2, 3, 4]
stack.pop();             // returns 4; stack is now [1, 2, 3]

const queue = [1, 2, 3];
queue.unshift(0);        // [0, 1, 2, 3]
queue.shift();           // returns 0; queue is now [1, 2, 3]

const letters = ["a", "b", "c", "d", "e"];
const removed = letters.splice(1, 2, "x", "y", "z");
console.log(letters); // ["a", "x", "y", "z", "d", "e"]
console.log(removed); // ["b", "c"]
```

push / pop are **O(1)** (fast). **shift / unshift** are **O(n)** because every remaining element has to be re-indexed. If you're building a queue that gets large, prefer pushing and reading from the end, or use a proper deque-like structure, rather than repeatedly calling `shift()`.

8.5 Non-mutating methods (return a new array or value)

These are the methods you should reach for by default, because they don't have hidden side effects on the original array — critical for predictable code, and required by frameworks like React that rely on referential equality to detect changes.

```
const nums = [5, 3, 8, 1, 9, 2];

nums.slice(1, 4);        // [3, 8, 1] – original untouched
nums.concat([10, 11]);   // [5,3,8,1,9,2,10,11] – original untouched
nums.map(n => n * 2);     // [10,6,16,2,18,4]
nums.filter(n => n > 4);  // [5,8,9]
nums.reduce((sum, n) => sum + n, 0); // 28
nums.includes(8);        // true
nums.indexOf(8);         // 2
nums.find(n => n > 4);     // 5 (first match)
nums.findIndex(n => n > 4); // 0
nums.some(n => n > 8);     // true
nums.every(n => n > 0);    // true
nums.join(", ");         // "5, 3, 8, 1, 9, 2"
```

.map() VS **.forEach()**

forEach runs a callback for each element and always returns `undefined` — use it purely for side effects (logging, pushing into an external array). **map** builds and returns a brand-new array from the callback's return values — use it when you want a transformed array back.

```
// Wrong: map used for side effects, return value thrown away, wasteful
nums.map(n => console.log(n));

// Right:
nums.forEach(n => console.log(n));
const doubled = nums.map(n => n * 2);
```

`.reduce()` deep dive

`reduce` is the most general-purpose array method — `map`, `filter`, and even `forEach` can all be implemented in terms of `reduce`. Its signature:

```
array.reduce((accumulator, currentValue, index, array) => { ... }, initialValue)
```

```
// Sum
[1, 2, 3, 4].reduce((sum, n) => sum + n, 0); // 10

// Group by property – a very common real-world use
const people = [
  { name: "Ana", dept: "eng" },
  { name: "Bo", dept: "sales" },
  { name: "Cy", dept: "eng" },
];
const byDept = people.reduce((groups, person) => {
  (groups[person.dept] ??= []).push(person);
  return groups;
}, {});
// { eng: [Ana, Cy], sales: [Bo] }

// Flatten one level (equivalent to Array.prototype.flat())
[[1, 2], [3, 4]].reduce((flat, arr) => flat.concat(arr), []); // [1,2,3,4]
```

Always pass an initial value to `reduce` unless you deliberately want the first element used as the seed — omitting it on an empty array throws `TypeError: Reduce of empty array with no initial value`.

`.flat()` and `.flatMap()`

```
[1, [2, 3], [4, [5, 6]]].flat();           // [1, 2, 3, 4, [5, 6]] (depth 1)
[1, [2, 3], [4, [5, 6]]].flat(2);         // [1, 2, 3, 4, 5, 6]
[1, [2, [3, [4]]]].flat(Infinity);        // fully flatten, any depth

// flatMap = map then flatten one level – useful for "expand" operations
["hello world", "foo bar"].flatMap(s => s.split(" "));
// ["hello", "world", "foo", "bar"]
```

8.6 Searching and testing arrays

```
const users = [
  { id: 1, name: "Ana", active: true },
  { id: 2, name: "Bo", active: false },
];

users.find(u => u.id === 2);      // { id: 2, name: "Bo", active: false }
users.findIndex(u => u.id === 2); // 1
users.findLast(u => u.active);    // last match, ES2023
users.findLastIndex(u => u.active); // ES2023
users.some(u => u.active);        // true – at least one
users.every(u => u.active);       // false – not all
```

`indexOf` / `includes` use strict equality (`===`) and so work well for primitives but cannot find objects by *value* — only by *reference*:

```
{ id: 1 }.includes({ id: 1 }); // false! different object references
```

8.7 Sorting — the classic footgun

`Array.prototype.sort()` converts elements to strings and sorts **lexicographically (alphabetically)** by **default** — not numerically. This surprises nearly every JavaScript beginner:

```
[10, 1, 21, 2].sort();
// [1, 10, 2, 21] <-- WRONG if you wanted numeric order!
```

Always supply a compare function for numbers:

```
[10, 1, 21, 2].sort((a, b) => a - b); // ascending: [1, 2, 10, 21]
[10, 1, 21, 2].sort((a, b) => b - a); // descending: [21, 10, 2, 1]

// Sorting objects by a field:
users.sort((a, b) => a.name.localeCompare(b.name)); // string sort, locale-aware
```

`sort()` mutates the original array. If you need to keep the original order intact, sort a copy: `[...arr].sort(...)` or `arr.toSorted(...)` (ES2023's non-mutating sibling, alongside `toReversed`, `toSpliced`, and `with`).

8.8 Multi-dimensional arrays

JavaScript has no native 2D array type — you nest arrays inside arrays:

```
const grid = [
  [0, 0, 0],
  [0, 1, 0],
  [0, 0, 0],
];
console.log(grid[1][1]); // 1 – row 1, column 1

// Building an n x m grid safely (avoid Array(n).fill(Array(m)) – that
// shares ONE inner array reference across every row!)
const rows = 3, cols = 3;
const safeGrid = Array.from({ length: rows }, () => Array(cols).fill(0));
safeGrid[0][0] = 9;
console.log(safeGrid[1][0]); // 0, not 9 – each row is a distinct array
```

8.9 Performance notes: sparse vs. dense arrays

A **dense** array has a value at every index from `0` to `length - 1`. A **sparse** array has gaps ("holes"):

```
const sparse = [1, , 3]; // hole at index 1
const alsoSparse = new Array(3); // 3 holes, no values at all
```

Engines like V8 optimize dense arrays with a consistent element type (all numbers, or all "packed" objects) far more aggressively than sparse or mixed-type arrays. Practical takeaways:

- Prefer `Array.from({length: n}, fn)` or `push` in a loop over `new Array(n)` followed by manual index assignment, to avoid holes.
- Avoid `delete arr[i]` (creates a hole) — use `splice` instead.
- Iteration methods (`forEach`, `map`, `filter`, ...) **skip holes entirely**, which is a subtle but important difference from a `for` loop that would see `undefined` at a hole.

```
const s = [1, , 3];
s.forEach(x => console.log(x)); // logs 1, then 3 – hole is skipped!
for (let i = 0; i < s.length; i++) console.log(s[i]); // logs 1, undefined, 3
```

8.10 A preview of destructuring with arrays

Array destructuring (fully covered in Chapter 13) lets you unpack values by position:

```
const [first, second, ...rest] = [1, 2, 3, 4, 5];
console.log(first, second, rest); // 1 2 [3, 4, 5]

const [, , third] = [1, 2, 3]; // skip elements with empty commas
console.log(third); // 3
```

8.11 Chapter summary

- Arrays are ordered, index-based objects with a dynamic `length` and a rich `Array.prototype` method set.
- Mutating methods (`push` , `pop` , `splice` , `sort` , `reverse` , ...) change the original array; non-mutating methods (`map` , `filter` , `slice` , `concat` , ...) return a new array and leave the original untouched — prefer the latter for predictable code.
- `sort()` is lexicographic by default; always pass a compare function for numbers.
- `reduce` is the most general array method and underlies grouping, flattening, and summing patterns.
- Avoid sparse arrays (`new Array(n)` , `delete arr[i]`) — they defeat engine optimizations and behave inconsistently across iteration methods.

8.12 Exercises

1. Write a function `unique(arr)` that returns an array with duplicate values removed, using `filter` + `indexOf` and, separately, using `Set` .
2. Given `const orders = [{amount: 20},{amount: 5},{amount: 40}]` , use `reduce` to compute the total amount.
3. Explain, without running it, what `[3, 25, 8, 1].sort()` returns and why. Then fix it to sort numerically ascending.
4. Run `examples/08-arrays.js` and read through every logged line — predict each output before it prints.

Chapter 9 — Objects

9.1 What is an object?

An object is an unordered collection of key-value pairs (called **properties**). Keys are always strings or `Symbol`s (numbers get converted to strings), and values can be anything — primitives, functions, arrays, or other objects. Objects are JavaScript's primary way of modeling structured, named data.

```
const user = {  
  name: "Ada",  
  age: 36,  
  isAdmin: true,  
};
```

Unlike arrays, objects don't guarantee numeric-index ordering by default — though in practice modern engines iterate integer-like keys first in ascending order, then string keys in insertion order, then symbols. Don't rely on object property order for anything load-bearing; use an array (or a `Map`, §9.10) when order matters.

9.2 Creating objects

```
const literal = { a: 1, b: 2 };           // object literal – most common  
const empty = {};  
const fromCtor = new Object();           // rarely used, avoid  
const fromCreate = Object.create(null);  // an object with NO prototype  
                                         // (no inherited methods at all –  
                                         // useful for pure dictionaries)
```

`Object.create(null)` is worth remembering: it produces an object that doesn't even have `toString` or `hasOwnProperty` inherited from `Object.prototype`, which makes it a safer choice when using an object as a plain key-value dictionary where keys come from untrusted input (no risk of a key named `"toString"` accidentally shadowing a real method — see §9.8).

9.3 Accessing and modifying properties

```
const user = { name: "Ada", age: 36 };

// Dot notation – use when the key is a known, valid identifier
console.log(user.name); // "Ada"

// Bracket notation – required when the key is dynamic or not a valid
// identifier (contains spaces, starts with a number, etc.)
const key = "age";
console.log(user[key]); // 36
console.log(user["full name"]); // undefined (property doesn't exist)

user.age = 37; // update existing property
user.email = "ada@example.com"; // add a new property
delete user.email; // remove a property

console.log(user); // { name: 'Ada', age: 37 }
```

Accessing a property that doesn't exist returns `undefined`, never an error — this is different from many other languages and is a frequent source of `undefined is not a function`-style bugs when a typo goes unnoticed (`user.naem`).

9.4 Computed property names and shorthand syntax

```
const field = "score";
const dynamicObj = {
  [field]: 100, // computed property name (ES2015)
  [`${field}_bonus`]: 10,
};
console.log(dynamicObj); // { score: 100, score_bonus: 10 }

// Shorthand property names – when a variable name matches the key you want
const name = "Grace";
const age = 45;
const person = { name, age }; // same as { name: name, age: age }

// Shorthand method syntax
const calculator = {
  value: 0,
  add(n) { // same as add: function(n) { ... }
    this.value += n;
    return this;
  },
};
```

9.5 Enumerating an object: keys, values, entries

```
const scores = { alice: 90, bob: 85, cy: 95 };

Object.keys(scores);    // ["alice", "bob", "cy"]
Object.values(scores);  // [90, 85, 95]
Object.entries(scores); // [["alice",90],["bob",85],["cy",95]]

// entries() pairs beautifully with for...of and destructuring
for (const [name, score] of Object.entries(scores)) {
  console.log(`${name}: ${score}`);
}

// and with Object.fromEntries() to go back the other direction –
// e.g. transforming every value
const doubled = Object.fromEntries(
  Object.entries(scores).map(([name, score]) => [name, score * 2])
);
console.log(doubled); // { alice: 180, bob: 170, cy: 190 }
```

9.6 Copying and merging objects

```
const base = { a: 1, b: 2 };
const extra = { b: 20, c: 3 };

// Object.assign mutates and returns the FIRST argument
const merged1 = Object.assign({}, base, extra); // {} as target -> no mutation of base
console.log(merged1); // { a: 1, b: 20, c: 3 } – later sources win on conflicts

// Object spread – the modern, more readable equivalent (ES2018)
const merged2 = { ...base, ...extra };
console.log(merged2); // { a: 1, b: 20, c: 3 }

// Overriding a spread field explicitly
const updated = { ...base, b: 99 };
console.log(updated); // { a: 1, b: 99 }
```

Both `Object.assign` and spread perform a *shallow copy* — nested objects/arrays are copied by reference, not deeply cloned:

```
const original = { info: { city: "Lagos" } };
const shallowCopy = { ...original };
shallowCopy.info.city = "Nairobi";
console.log(original.info.city); // "Nairobi" – the nested object was shared!
```

Deep cloning

```
// structuredClone – built into modern Node.js and browsers, handles most
// real-world data (but not functions, and it has its own rules for
// special types like Map/Set/Date, which it DOES support correctly)
const deepCopy = structuredClone(original);
deepCopy.info.city = "Accra";
console.log(original.info.city); // "Nairobi" – untouched this time

// The old JSON trick – works for plain data but silently DROPS functions,
// undefined values, and Symbol keys, and turns Dates into strings. Only
// use it when you know the data is simple.
const jsonCopy = JSON.parse(JSON.stringify(original));
```

9.7 Freezing, sealing, and immutability

```
const config = { debug: true };

Object.freeze(config); // prevents adding, removing, or changing
config.debug = false; // silently fails in non-strict mode
console.log(config.debug); // still true
console.log(Object.isFrozen(config)); // true

const record = { id: 1 };
Object.seal(record); // allows changing existing values,
record.id = 2; // but not adding/removing properties
record.extra = "no"; // silently ignored
console.log(record); // { id: 2 }
```

Object.freeze is shallow, just like spread — a frozen object's nested objects are still mutable unless you recursively freeze them.

9.8 The `in` operator vs `hasOwnProperty`

```
const obj = { a: 1 };

"a" in obj; // true – checks OWN and INHERITED properties
"toString" in obj; // true! inherited from Object.prototype
obj.hasOwnProperty("toString"); // false – only checks the object's OWN properties
Object.hasOwn(obj, "toString"); // false – modern (ES2022), safer alternative
// to .hasOwnProperty because it works even
// if `obj` has no prototype (Object.create(null))
```

Prefer `Object.hasOwn(obj, key)` over `obj.hasOwnProperty(key)` in new code — it can't be broken by an object that happens to have its own property named `hasOwnProperty`, and it works on `Object.create(null)` objects that have no `hasOwnProperty` method to call at all.

9.9 Getters, setters, and property descriptors

Every object property actually has a hidden descriptor controlling its behavior.

`Object.defineProperty` gives you direct access to it:

```
const person = { firstName: "Ada", lastName: "Lovelace" };

Object.defineProperty(person, "fullName", {
  get() {
    return `${this.firstName} ${this.lastName}`;
  },
  set(value) {
    [this.firstName, this.lastName] = value.split(" ");
  },
  enumerable: true,
});

console.log(person.fullName); // "Ada Lovelace"
person.fullName = "Grace Hopper";
console.log(person.firstName); // "Grace"
```

The same getter/setter syntax is available directly inside an object literal, which is far more common in day-to-day code:

```
const circle = {
  radius: 5,
  get area() {
    return Math.PI * this.radius ** 2;
  },
  set diameter(d) {
    this.radius = d / 2;
  },
};

console.log(circle.area.toFixed(2)); // "78.54"
circle.diameter = 20;
console.log(circle.radius); // 10
```

Descriptors also control `writable`, `enumerable` (does it show up in `for...in` / `Object.keys`?), and `configurable` (can it be deleted or redefined?) — this is how `Object.freeze` works under the hood, and how built-in methods like `Array.prototype.map` stay hidden from `Object.keys([])`.

9.10 Object vs Map: which to use

`Map` (covered fully as part of the built-in collections landscape) is often a better choice than a plain object for dictionary-style data:

	Object	Map
Key types	strings/symbols only	any value, including objects
Guaranteed insertion order	mostly, with integer-key caveat	always
Size	manual (<code>Object.keys(o).length</code>)	<code>map.size</code>
Iteration	needs <code>Object.entries</code> + loop	directly iterable
Performance for frequent add/remove	can degrade	optimized for this
JSON support	native	needs manual conversion

Rule of thumb: use a plain object for fixed-shape records (a "struct") where you know the field names ahead of time (`{ name, age, email }`); use a `Map` for dynamic collections where keys are added/removed frequently or aren't known in advance.

```
const cache = new Map();
cache.set("user:1", { name: "Ada" });
cache.set(someObjectAsKey, "works fine as a key, unlike with objects");
console.log(cache.get("user:1"));
console.log(cache.size);
for (const [key, value] of cache) {
  console.log(key, value);
}
```

9.11 Optional chaining with objects

Optional chaining (`?.`, ES2020) short-circuits to `undefined` instead of throwing when accessing a property on `null` / `undefined`:

```
const response = { data: { user: null } };

console.log(response.data.user.name); // TypeError: Cannot read properties of null
console.log(response.data.user?.name); // undefined – no crash
console.log(response.data?.missing?.deeper?.field); // undefined, safely
console.log(response.data?.getUser?.()); // optional chaining on a method call too
```

Combine it with nullish coalescing (Chapter 4) for safe defaults:

```
const displayName = response.data.user?.name ?? "Guest";
```

9.12 Chapter summary

- Objects are unordered key-value collections; keys are strings or symbols, values can be anything.

- Prefer object/computed/shorthand literal syntax over `new Object()`.
- `Object.assign` and spread both perform **shallow** copies; use `structuredClone` for real deep clones of plain data.
- `Object.freeze` / `seal` are also shallow — freeze nested objects individually if you need deep immutability.
- Prefer `Object.hasOwn(obj, key)` over `in` (which checks the whole prototype chain) or `.hasOwnProperty` (which can be shadowed).
- Getters/setters let a property look like data but run code on access.
- Choose `Map` over a plain object for dynamic, frequently-mutated key-value collections, especially with non-string keys.

9.13 Exercises

1. Write a function `pick(obj, keys)` that returns a new object containing only the given keys from `obj`, using `Object.fromEntries` and `Object.entries`.
2. Demonstrate, with code, why `{ ...original }` is not a safe way to copy an object containing a nested array, and fix it with `structuredClone`.
3. Explain the difference between `Object.freeze(obj)` and `Object.seal(obj)` in your own words, then verify your explanation by running both against the same object and testing add/remove/modify.
4. Run `examples/09-objects.js` and predict each line's output before it prints.

Chapter 10 — Strings

10.1 String basics and immutability

Strings represent text and are one of JavaScript's primitive types. A critical fact: **strings are immutable** — no method ever changes a string in place. Every string operation returns a brand-new string.

```
let greeting = "hello";
greeting[0] = "H";           // silently does nothing (non-strict mode)
console.log(greeting);       // "hello" — unchanged

greeting = greeting.toUpperCase(); // reassigning with a NEW string
console.log(greeting);         // "HELLO"
```

Because strings can't be mutated, methods like `.trim()`, `.replace()`, and `.slice()` always return a new string that you must capture (assign or use directly) — calling `str.trim()` and throwing away the result does nothing.

10.2 Creating strings: quotes and template literals

```
const single = 'hello';
const double = "hello";
const template = `hello`; // template literal (ES2015, backticks)
```

Single and double quotes are functionally identical — pick one convention and stick with it (this book uses double quotes for simple strings). Template literals unlock two features single/double quotes can't do:

Interpolation — embed expressions directly with `${...}`:

```
const name = "Wole";
const age = 30;
console.log(`${name} is ${age} years old, born in ${2026 - age}.`);
// "Wole is 30 years old, born in 1996."
```

Multi-line strings — no more `"line1\n" + "line2"` concatenation:

```
const message = `Dear customer,  
  
Thank you for your order.  
Regards,  
The Team`;
```

Tagged templates

A function placed immediately before a template literal receives the string parts and interpolated values separately, letting you post-process the whole thing — this is how libraries like `styled-components` and `graphql-tag` work:

```
function highlight(strings, ...values) {  
  return strings.reduce((result, str, i) => {  
    const value = values[i] !== undefined ? `**${values[i]}**` : "";  
    return result + str + value;  
  }, "");  
}  
  
const item = "laptop", price = 999;  
console.log(highlight`The ${item} costs ${price} dollars.`);  
// "The **laptop** costs **999** dollars."
```

10.3 Common string methods

```

const s = "  Hello, World!  ";

s.trim();           // "Hello, World!" (removes leading/trailing whitespace)
s.trimStart(); s.trimEnd(); // one-sided trims

s.toUpperCase();    // "  HELLO, WORLD!  "
s.toLowerCase();    // "  hello, world!  "

const text = "Hello, World!";
text.length;        // 13
text.slice(7, 12);   // "World" (start inclusive, end exclusive)
text.slice(-6);      // "World!" (negative = from the end)
text.substring(7, 12); // "World" (like slice, but no negative support,
                        // and swaps args if start > end instead of
                        // returning "" – slice is generally preferred)

text.split(", ");    // ["Hello", "World!"]
text.replace("World", "JS"); // "Hello, JS!" (replaces FIRST match only)
text.replaceAll("l", "L");  // "HeLLo, WorLd!" (ES2021, replaces ALL matches)
text.includes("World");    // true
text.startsWith("Hello");  // true
text.endsWith("!");        // true
text.indexOf("o");         // 4 (first occurrence)
text.lastIndexOf("o");     // 8
text.charAt(0);            // "H"
text[0];                   // "H" (bracket access also works, read-only)
text.padStart(15, "*");    // "**Hello, World!"
text.padEnd(15, "*");      // "Hello, World!*"
"ab".repeat(3);           // "ababab"

```

`replace` / `replaceAll` also accept a regular expression as the first argument and a function as the second (covered fully in Chapter 26):

```
"2026-09-16".replace(/(\d+)-(\d+)-(\d+)/, "$3/$2/$1"); // "16/09/2026"
```

10.4 Converting between strings and numbers

There are several ways to convert a string to a number, and they don't all behave the same on edge cases — a common source of subtle bugs.

```

Number("42");           // 42
Number("42px");          // NaN – Number() requires the ENTIRE string to be numeric
Number("");              // 0 – surprising! empty string converts to 0
Number(" 42 ");          // 42 – whitespace is trimmed
Number(null);            // 0 – another surprise
Number(undefined);       // NaN

parseInt("42px");         // 42 – parseInt reads as many leading digits as it can
parseInt("px42");         // NaN – but stops immediately if it can't start
parseInt("3.99");         // 3 – parseInt ignores everything after the decimal
parseFloat("3.99abc");    // 3.99 – parseFloat keeps the decimal portion

+"42";                   // 42 – unary plus, same rules as Number()
+"42px";                 // NaN

"42" * 1;                // 42 – multiplying by 1 also coerces

```

Practical guidance: use `Number(str)` when you want strict "is this entirely a valid number" behavior, and `parseInt(str, 10)` / `parseFloat(str)` when you're deliberately extracting a leading number from a larger string (like `"42px"` from CSS). **Always pass a radix (10) to `parseInt`** — `parseInt("08")` without a radix has historically caused bugs on some engines interpreting leading-zero numbers as octal.

Going the other direction, number-to-string:

```

String(42);              // "42"
(42).toString();         // "42"
(42).toString(2);        // "101010" – base-2 (binary) representation
(255).toString(16);      // "ff" – hexadecimal
`${42}`;                 // "42" – template literal implicit conversion

```

10.5 Unicode, UTF-16, and string length surprises

JavaScript strings are sequences of UTF-16 code units, not "characters" in the way humans think of them. Most common characters (Latin letters, digits, basic punctuation) fit in a single 16-bit code unit, but many emoji, some CJK (Chinese/Japanese/Korean) characters, and other symbols outside the "Basic Multilingual Plane" require **two** code units — a "surrogate pair." This means `.length` can lie about the number of visible characters:

```

console.log("hello".length); // 5 – as expected
console.log("😊".length);    // 2 – ONE emoji, but TWO UTF-16 code units!
console.log("café".length);  // 4 – fine, é fits in one code unit

const smiley = "😄";
console.log([...smiley].length); // 1 – spreading into an array of
                                // code POINTS gets this right
console.log(Array.from(smiley).length); // 1 – same idea, Array.from is
                                // iterable-aware
console.log(smiley[0]);           // an unpaired surrogate (broken glyph!)
console.log([...smiley][0]);      // the correct, whole emoji

```

Practical rule: when counting or slicing user-facing text that might contain emoji or non-BMP characters, iterate with `for...of` or `spread ( [...str] )` rather than indexing/ `.length` directly, because those iterate by *code point*, not raw UTF-16 unit.

10.6 Comparing strings

```

"apple" < "banana"; // true – lexicographic (dictionary) comparison
"Apple" < "apple";   // true – uppercase letters sort BEFORE lowercase
                    // in ASCII/UTF-16 ('A' is 65, 'a' is 97)

"apple".localeCompare("banana"); // -1 (locale-aware, handles accents/case
                                // more sensibly than < >)
["Banana", "apple", "Cherry"].sort((a, b) => a.localeCompare(b));
// ["apple", "Banana", "Cherry"] – sensible alphabetical order
["Banana", "apple", "Cherry"].sort();
// ["Banana", "Cherry", "apple"] – naive sort, capital letters sort first!

```

10.7 Building HTML/SQL from strings: a security note

It's tempting to build HTML or SQL by concatenating strings or using template literals with user input directly:

```

// DANGEROUS – if `username` comes from user input, this is an XSS hole
const html = `<div>Welcome, ${username}!</div>`;
element.innerHTML = html; // an attacker could set username to
                        // "<img src=x onerror=alert(1)>"

// DANGEROUS – never build SQL with string interpolation; this is a
// textbook SQL injection vulnerability
const query = `SELECT * FROM users WHERE name = '${username}'`;

```

The fix is never "escape it by hand" — use the platform's safe APIs instead: `element.textContent` (not `innerHTML`) for inserting plain text into the DOM (Chapter 27), and parameterized queries / prepared statements for SQL (`db.query("SELECT * FROM users WHERE name = ?", [username])`),

which every real database driver supports.

10.8 Chapter summary

- Strings are immutable; every method returns a new string rather than modifying the original.
- Template literals (backticks) support interpolation (`${expr}`), multi-line text, and tagged template functions for custom processing.
- `Number()` , `parseInt()` , and `parseFloat()` have different edge-case behaviors when converting strings to numbers — choose deliberately.
- `.length` counts UTF-16 code units, not visible characters — `spread ( [...str] )` or `for...of` to iterate correctly over emoji/non-BMP text.
- Never build HTML or SQL via raw string interpolation of user input — use `textContent` /safe DOM APIs and parameterized database queries.

10.9 Exercises

1. Write a function `truncate(str, maxLength)` that shortens a string to `maxLength` characters and appends `"..."` if it was truncated, being careful not to split a surrogate pair in half.
2. Predict the output of `Number("")` , `Number(" ")` , and `Number(" 1 ")` before running `examples/10-strings.js` to check.
3. Explain why `["Banana", "apple"].sort()` doesn't give "sensible" alphabetical order, and fix it two different ways.

Chapter 11 — Numbers and Math

11.1 There's only one number type

Unlike languages with separate `int`, `float`, `double`, and `long` types, JavaScript has a single `number` type: a 64-bit floating-point number following the **IEEE 754 double-precision** standard. `42` and `42.5` are both just `number` under the hood.

```
typeof 42;           // "number"
typeof 42.5;         // "number"
typeof NaN;          // "number" — yes, "Not a Number" is itself typed "number"
typeof Infinity;     // "number"
```

This design is simple but has real, learnable consequences: integer-only languages don't have to worry about floating-point rounding error on every arithmetic operation, but JavaScript does.

11.2 Floating-point precision: the `0.1 + 0.2` problem

```
console.log(0.1 + 0.2);           // 0.30000000000000004 — NOT 0.3!
console.log(0.1 + 0.2 === 0.3);   // false
```

This isn't a JavaScript bug — it's a consequence of IEEE 754 binary floating point, shared by nearly every mainstream language (Python, Java, C, Go all show the same behavior). Some fractions (like 0.1) cannot be represented exactly in binary, the same way 1/3 cannot be represented exactly in decimal. The fix is to never compare floats for exact equality; instead, check if they're close enough:

```
function approximatelyEqual(a, b, epsilon = Number.EPSILON * 100) {
  return Math.abs(a - b) < epsilon;
}
console.log(approximatelyEqual(0.1 + 0.2, 0.3)); // true
```

For money, prefer working in integer cents (`1050` instead of `10.50`) or use `BigInt` / a decimal library, rather than floating-point dollars.

11.3 Integer safety limits

```

Number.MAX_SAFE_INTEGER; // 9007199254740991 (2^53 - 1)
Number.MIN_SAFE_INTEGER; // -9007199254740991

console.log(Number.MAX_SAFE_INTEGER + 1); // 9007199254740992 – correct
console.log(Number.MAX_SAFE_INTEGER + 2); // 9007199254740992 – WRONG!
// (should be ...993, but
// precision is lost beyond
// the safe integer range)

Number.isSafeInteger(9007199254740991); // true
Number.isSafeInteger(9007199254740992); // false

```

Beyond `MAX_SAFE_INTEGER`, integers can no longer be represented uniquely — multiple mathematically distinct integers collapse to the same floating point value. This matters for things like database IDs from systems that use 64-bit integers (e.g., Twitter/X snowflake IDs, some database auto-increment columns) — those should be handled as strings or `BigInt` in JavaScript, never as regular numbers.

11.4 NaN: "Not a Number"

`NaN` represents a failed or undefined numeric operation, and it has one famously bizarre property: **it is the only value in JavaScript that is not equal to itself.**

```

console.log(0 / 0); // NaN
console.log("abc" * 2); // NaN
console.log(NaN === NaN); // false!
console.log(NaN == NaN); // false too – no coercion rescues this

// Correct ways to test for NaN:
console.log(Number.isNaN(NaN)); // true – the ONLY reliable check
console.log(isNaN(NaN)); // true

console.log(isNaN("hello")); // true – DANGER: global isNaN()
// coerces its argument to a number
// first ("hello" -> NaN -> true)
console.log(Number.isNaN("hello")); // false – Number.isNaN does NOT
// coerce; "hello" is not the
// NaN value, so it's false

```

Always prefer `Number.isNaN()` over the global `isNaN()` — the global version's implicit coercion produces `true` for any non-numeric, non-convertible value, which is almost never what you actually want to ask.

11.5 Useful `Number` methods and properties

```

Number.isInteger(42);      // true
Number.isInteger(42.5);    // false
Number.isFinite(42);      // true – does NOT coerce (unlike global isFinite)
Number.isFinite(Infinity); // false
Number.parseInt("42px");   // same as global parseInt, namespace
Number.parseFloat("3.14"); // same as global parseFloat, namespace

(1234.5678).toFixed(2);    // "1234.57" – returns a STRING, rounds
(0.1).toFixed(20);         // "0.10000000000000000555" – reveals
                           // the underlying floating-point
                           // imprecision
(1234.5678).toPrecision(6); // "1234.57" – total significant digits

```

11.6 The `Math` object

`Math` is a built-in object (not a constructor — you never write `new Math()`) bundling constants and functions for common numeric operations.

```

Math.round(4.5);    // 5 – rounds half up
Math.round(-4.5);   // -4 – careful, rounds toward +Infinity on ties, not
                     // "away from zero" as many expect
Math.floor(4.9);    // 4 – always rounds down
Math.ceil(4.1);     // 5 – always rounds up
Math.trunc(4.9);    // 4 – chops off the decimal, no rounding logic
Math.trunc(-4.9);   // -4 – unlike floor(-4.9) which is -5

Math.abs(-7);       // 7
Math.pow(2, 10);    // 1024 (same as 2 ** 10)
Math.sqrt(64);      // 8
Math.cbrt(27);      // 3
Math.min(4, 1, 9, -2); // -2
Math.max(4, 1, 9, -2); // 9
Math.min(...[4, 1, 9, -2]); // -2 – spread an array into min/max
Math.PI;            // 3.141592653589793
Math.E;             // 2.718281828459045
Math.random();      // a float in [0, 1)

```

Generating random integers in a range

```

// Random integer between min (inclusive) and max (inclusive)
function randomInt(min, max) {
  return Math.floor(Math.random() * (max - min + 1)) + min;
}
randomInt(1, 6); // simulate a dice roll: 1, 2, 3, 4, 5, or 6

```

11.7 BigInt: when numbers aren't enough

`BigInt` (ES2020) is a second numeric type for arbitrary-precision integers, created by appending `n` to an integer literal or calling `BigInt()` :

```
const huge = 9007199254740993n; // BigInt literal
const alsoHuge = BigInt("9007199254740993");
console.log(huge + 10n);          // 9007199254741003n – exact, no rounding

console.log(typeof huge);          // "bigint" – a genuinely different type

// BigInt and Number CANNOT be mixed directly – this throws:
// console.log(huge + 1);           // TypeError: Cannot mix BigInt and
//                                // other types
console.log(huge + BigInt(1));      // must convert explicitly
console.log(Number(huge) + 1);      // or convert the BigInt down,
//                                // losing precision if it's huge
```

Use `BigInt` for things like cryptography, very large ID numbers, or exact integer arithmetic beyond `Number.MAX_SAFE_INTEGER` — not for everyday math, since it's slower and can't represent fractions at all.

11.8 Formatting numbers for humans

```
(1234567.891).toLocaleString();
// "1,234,567.891" (in en-US locale – varies by locale!)

new Intl.NumberFormat("en-US", {
  style: "currency",
  currency: "USD",
}).format(1234.5);
// "$1,234.50"

new Intl.NumberFormat("de-DE").format(1234567.891);
// "1.234.567,891" – German locale uses '.' and ',' swapped from US

new Intl.NumberFormat("en-US", { style: "percent" }).format(0.756);
// "76%"
```

`Intl.NumberFormat` is the correct, locale-aware way to display numbers, currency, and percentages to users — manual string manipulation (inserting commas by hand) breaks for non-US locales and is easy to get wrong for edge cases like negative numbers.

11.9 Chapter summary

- JavaScript has one `number` type: a 64-bit IEEE 754 double. There is no separate integer type (unless you use `BigInt`).

- Floating-point arithmetic is imprecise by nature (`0.1 + 0.2 !== 0.3`); never compare floats for exact equality, and use integer cents or `BigInt` /decimal libraries for money.
- `Number.MAX_SAFE_INTEGER` ($2^{53} - 1$) is the largest integer JS can represent exactly; beyond it, precision silently degrades.
- Always use `Number.isNaN()` , never the coercing global `isNaN()` .
- `Math` provides rounding, power/root, min/max, and random functions; combine `Math.random()` and `Math.floor()` for random integers.
- `BigInt` handles arbitrary-precision integers but cannot mix with regular numbers without explicit conversion.
- `Intl.NumberFormat` is the correct tool for locale-aware number, currency, and percentage display.

11.10 Exercises

1. Write `randomInt(min, max)` (inclusive on both ends) and use it to simulate rolling two six-sided dice 10,000 times, then log how many times the total was 7.
2. Explain why `(0.1 + 0.2).toFixed(1) === "0.3"` returns `true` even though `0.1 + 0.2 === 0.3` returns `false` .
3. Format `1234567.891` as USD currency and as a German-locale number using `Intl.NumberFormat` , and compare the results.

Chapter 12 — Dates and Time

12.1 The `Date` object

JavaScript's built-in `Date` object represents a single moment in time, stored internally as the number of milliseconds since the **Unix epoch** (midnight, January 1, 1970, UTC). Every `Date` you create, no matter how you construct it, boils down to this one timestamp number.

```
const now = new Date();
console.log(now);           // e.g. 2026-09-16T10:30:00.000Z
console.log(now.getTime()); // e.g. 1789554600000 — ms since epoch
console.log(Date.now());    // the same kind of number, without
                             // needing to construct a Date object —
                             // prefer this for simple timestamps
```

12.2 Creating dates: the many constructor forms

```
new Date();           // now
new Date(1789554600000); // from an epoch millisecond timestamp
new Date("2026-09-16"); // from an ISO 8601 date string
                        // (parsed as UTC midnight)
new Date("2026-09-16T10:30:00"); // ISO string with time
                                // (parsed as LOCAL time,
                                // because there's no 'Z'!)
new Date("2026-09-16T10:30:00Z"); // ISO string with explicit UTC
new Date(2026, 8, 16); // year, month (0-indexed!),
                        // day — MONTH IS ZERO-BASED,
                        // a classic gotcha (8 = Sept)
new Date(2026, 8, 16, 10, 30, 0); // + hours, minutes, seconds
```

The two biggest `Date` gotchas to memorize immediately:

1. **Months are zero-indexed** (0 = January, 11 = December) — `new Date(2026, 0, 1)` is January 1st, not February.
2. `new Date("2026-09-16")` (date-only ISO string) parses as **UTC midnight**, while `new Date("2026-09-16T00:00:00")` (with an explicit time but no `Z`/offset) parses as **local time midnight**. Depending on your timezone, these can represent different actual moments and even display as different calendar days when formatted.

12.3 Getting and setting date components

```
const d = new Date(2026, 8, 16, 14, 30, 45);

d.getFullYear();    // 2026
d.getMonth();       // 8 (September, zero-indexed – NEVER use getYear(),
                    // a deprecated, non-Y2K-safe method)
d.getDate();        // 16 (day of the month, 1-31)
d.getDay();         // day of the WEEK, 0 (Sunday) - 6 (Saturday)
d.getHours();       // 14
d.getMinutes();     // 30
d.getSeconds();     // 45
d.getMilliseconds(); // 0
d.getTimezoneOffset(); // minutes behind/ahead of UTC (sign flipped –
                    // e.g. UTC-5 reports as +300)

// Setters mutate the Date object in place
d.setFullYear(2027);
d.setMonth(0);       // change to January
d.setDate(1);        // change to the 1st

// getUTC* variants read the SAME instant in UTC, ignoring local timezone
d.getUTCHours();
```

12.4 Date arithmetic

`Date` has no built-in "add days" method — you work with the underlying millisecond timestamp, or manipulate components directly (JavaScript auto-normalizes overflowing values, e.g. day 32 rolls into the next month):

```
function addDays(date, days) {
  const result = new Date(date);
  result.setDate(result.getDate() + days);
  return result; // handles month/year rollover automatically
}

const today = new Date(2026, 8, 30); // Sept 30, 2026
console.log(addDays(today, 5)); // Oct 5, 2026 – rolled into next month correctly

function daysBetween(a, b) {
  const msPerDay = 1000 * 60 * 60 * 24;
  return Math.round((b.getTime() - a.getTime()) / msPerDay);
}
console.log(daysBetween(new Date(2026, 0, 1), new Date(2026, 11, 31))); // 364
```

Adding months/years by simply calling `setMonth(getMonth() + 1)` can produce surprising results near month boundaries — e.g. adding one month to January 31st "overflows" into March 3rd (because February doesn't have 31 days), which is exactly why serious date-arithmetic needs are usually handled by a library rather than hand-rolled code (§12.6).

12.5 Formatting dates

```
const d = new Date(2026, 8, 16, 14, 30, 0);

d.toISOString();      // "2026-09-16T18:30:00.000Z" (always UTC, machine-
                        // readable – use this for APIs and storage)
d.toDateString();     // "Wed Sep 16 2026"
d.toLocaleDateString(); // "9/16/2026" (format varies by locale!)
d.toLocaleTimeString(); // "2:30:00 PM"
d.toLocaleString();    // "9/16/2026, 2:30:00 PM"

// Intl.DateTimeFormat gives full control over locale-aware formatting
new Intl.DateTimeFormat("en-US", {
  weekday: "long",
  year: "numeric",
  month: "long",
  day: "numeric",
}).format(d);
// "Wednesday, September 16, 2026"

new Intl.DateTimeFormat("fr-FR", { dateStyle: "full" }).format(d);
// "mercredi 16 septembre 2026"
```

For storage and APIs, always use `toISOString()` (or store the raw epoch number) — never a locale-formatted string, which is ambiguous (is `9/16/2026` September 16th or the 9th day of month 16?) and impossible to reliably parse back.

12.6 Timezones, and Date's honest design flaws

`Date` has two "modes" baked in that constantly trip people up: some methods operate in the browser/Node process's **local timezone** (`getHours`, `toLocaleDateString`, the `new Date(y, m, d)` constructor), while others operate in **UTC** (`getUTCHours`, `toISOString`). Mixing them without realizing it is one of the most common sources of "off-by-one-day" bugs in real applications, especially for teams distributed across timezones or users traveling across them.

`Date` is also, by wide consensus among JS developers, a genuinely awkward API: it's mutable (setters modify in place, unlike almost every other built-in), month indexing is zero-based while day-of-month is one-based (inconsistent), and there's no built-in duration type. For anything beyond simple formatting and arithmetic, most production codebases reach for a library:

- **date-fns** — a collection of small, tree-shakeable pure functions (`addDays`, `differenceInDays`, `format`) — a popular modern default.
- **Day.js** — a tiny (~2KB) Moment.js-compatible API with a plugin system.
- **Luxon** — built by a Moment.js maintainer, with first-class timezone and `Intl` support.

- **Moment.js** — historically dominant, but now in official maintenance mode; the docs themselves recommend newer alternatives for new projects.

For learning and for genuinely simple needs (log timestamps, basic date math), the native `Date` and `Intl` APIs covered in this chapter are enough — reach for a library once you're doing serious timezone-aware scheduling, recurring events, or duration math.

12.7 Chapter summary

- `Date` stores a single instant as milliseconds since the Unix epoch; `Date.now()` gets that number directly without an object.
- Two critical gotchas: **months are zero-indexed**, and a date-only ISO string parses as UTC while a datetime string without an offset parses as local time.
- There's no built-in "add days" — mutate a copy's components with `setDate` / `setMonth` /etc., which auto-normalizes overflow.
- Use `toISOString()` for storage/APIs; use `Intl.DateTimeFormat` for locale-aware display to users.
- `Date`'s API has real, acknowledged design flaws (mutability, inconsistent indexing) — reach for date-fns, Day.js, or Luxon once your date-handling needs get non-trivial (recurring events, real timezone math).

12.8 Exercises

1. Write `isLeapYear(year)` using `Date`'s auto-normalizing behavior: construct `new Date(year, 1, 29)` and check whether `getMonth()` still reports February (1) or has rolled into March (2).
2. Explain, in your own words, why `new Date("2026-01-01")` and `new Date("2026-01-01T00:00:00")` can represent different moments in time depending on the machine's timezone.
3. Write `formatFriendly(date)` that returns something like "September 16, 2026" using `Intl.DateTimeFormat`.

Chapter 13 — Destructuring, Spread & Rest

These three related ES2015+ features let you unpack values out of arrays and objects (**destructuring**), expand a collection into individual elements (**spread**), and gather multiple elements back into a single array (**rest**). Together they eliminate a huge amount of manual indexing and temporary-variable boilerplate from everyday JavaScript.

13.1 Array destructuring

```
const point = [10, 20, 30];
const [x, y, z] = point;
console.log(x, y, z); // 10 20 30

// Skipping elements with empty commas
const [first, , third] = point;
console.log(first, third); // 10 30

// Default values – used only when the unpacked value is undefined
const [a = 1, b = 2, c = 3, d = 4] = [10, 20];
console.log(a, b, c, d); // 10 20 3 4

// Swapping two variables – no temp variable needed
let p = 1, q = 2;
[p, q] = [q, p];
console.log(p, q); // 2 1

// Rest in array destructuring – gathers the remainder into a new array
const [head, ...tail] = [1, 2, 3, 4, 5];
console.log(head, tail); // 1 [2, 3, 4, 5]
```

Array destructuring works on **any iterable**, not just arrays — strings, `Set` s, `Map` s, the results of generator functions:

```
const [c1, c2, c3] = "abc";
console.log(c1, c2, c3); // "a" "b" "c"

const [firstEntry] = new Map([["k1", "v1"], ["k2", "v2"]]);
console.log(firstEntry); // ["k1", "v1"]
```

13.2 Object destructuring

```

const user = { name: "Ada", age: 36, city: "London" };

const { name, age } = user;
console.log(name, age); // "Ada" 36

// Renaming while destructuring
const { name: fullName, age: years } = user;
console.log(fullName, years); // "Ada" 36

// Default values (used when the property is missing OR undefined)
const { name: n = "Anonymous", country = "Unknown" } = user;
console.log(n, country); // "Ada" "Unknown"

// Nested destructuring
const response = { data: { user: { id: 1, profile: { bio: "Engineer" } } } };
const { data: { user: { profile: { bio } } } } = response;
console.log(bio); // "Engineer"

// Rest in object destructuring – gathers remaining properties into a new object
const { name: userName, ...rest } = user;
console.log(userName, rest); // "Ada" { age: 36, city: "London" }

```

Unlike array destructuring, object destructuring pulls by **property name**, not position, so key order in the object doesn't matter.

13.3 Destructuring in function parameters

This is one of the most common real-world uses — it turns an options object into self-documenting, defaultable named parameters:

```

// Without destructuring – caller must remember argument ORDER
function createUserBad(name, age, isAdmin, country) { /* ... */ }
createUserBad("Ada", 36, false, "UK"); // what does `false` mean here?!

// With destructuring – order doesn't matter, defaults are declarative,
// and the call site is self-documenting
function createUser({ name, age, isAdmin = false, country = "Unknown" }) {
  return { name, age, isAdmin, country };
}
createUser({ age: 36, name: "Ada", country: "UK" });
// { name: "Ada", age: 36, isAdmin: false, country: "UK" }

// Array destructuring in parameters works too – common in .map callbacks
// over [key, value] pairs from Object.entries()
Object.entries({ a: 1, b: 2 }).map(([key, value]) => `${key}=${value}`);
// ["a=1", "b=2"]

```

Caution: if you destructure a parameter and the caller passes no argument at all, it throws (`Cannot destructure property 'name' of 'undefined'`) unless you give the *whole parameter* a default empty object: `function f({ name } = {}) { ... }`.

13.4 The spread operator (`...`)

Spread *expands* an iterable or object into individual elements. It looks identical to rest (`...`) but does the opposite job — the distinction is purely about **where** it appears (rest collects in a binding position; spread expands in a value position).

Spread in arrays

```
const a = [1, 2, 3];
const b = [4, 5, 6];

const combined = [...a, ...b];           // [1, 2, 3, 4, 5, 6]
const withExtra = [0, ...a, 3.5, ...b];  // [0, 1, 2, 3, 3.5, 4, 5, 6]
const copy = [...a];                     // shallow copy of an array
const fromString = [..."hello"];          // ["h","e","l","l","o"]
const max = Math.max(...a);               // spreading into a function call
```

Spread in objects

```
const defaults = { theme: "light", fontSize: 14 };
const overrides = { fontSize: 18 };
const settings = { ...defaults, ...overrides }; // { theme: "light", fontSize: 18 }
// later spreads win on conflicts
```

Spread in function calls

```
function sum3(a, b, c) { return a + b + c; }
const nums = [1, 2, 3];
sum3(...nums); // 6 – equivalent to sum3(1, 2, 3), but without hard-coding
```

13.5 The rest parameter

Rest *collects* multiple remaining arguments into a real array — it's the modern replacement for the old, array-like-but-not-really `arguments` object (Chapter 7):

```
function sum(...numbers) {
  return numbers.reduce((total, n) => total + n, 0);
}
sum(1, 2, 3, 4); // 10 – numbers is a real Array: [1, 2, 3, 4]

// Rest must be the LAST parameter – this is a syntax error otherwise
function logFirstThenRest(first, ...others) {
  console.log("first:", first, "others:", others);
}
logFirstThenRest(1, 2, 3, 4); // first: 1 others: [2, 3, 4]
```

13.6 Combining everything: realistic patterns

```
// Clone-with-override, a extremely common pattern in state management
// (Redux reducers, React setState, config merging)
function updateUser(user, changes) {
  return { ...user, ...changes };
}
const original = { id: 1, name: "Ada", age: 36 };
const updated = updateUser(original, { age: 37 });
console.log(original, updated);
// { id: 1, name: 'Ada', age: 36 } { id: 1, name: 'Ada', age: 37 }

// Extracting a subset of fields while dropping the rest (redacting a
// password before sending a user object to the client, for example)
function toPublicProfile({ password, ssn, ...publicFields }) {
  return publicFields;
}
toPublicProfile({ id: 1, name: "Ada", password: "secret", ssn: "123-45-6789" });
// { id: 1, name: "Ada" }

// Merging default config with nested overrides safely
function withDefaults(userConfig = {}) {
  const defaults = { retries: 3, timeout: 5000, headers: {} };
  return { ...defaults, ...userConfig, headers: { ...defaults.headers, ...userConfig.headers } };
}
```

Note the last example: naive top-level spread merges only the *first level* — if `headers` is an object on both sides, one fully overwrites the other unless you explicitly spread-merge that nested level too. This is the same shallow-merge caveat from Chapter 9 resurfacing here.

13.7 Chapter summary

- Array destructuring unpacks by position (supports skipping, defaults, rest); object destructuring unpacks by property name (supports renaming, nested paths, defaults, rest).

- Destructuring function parameters turns positional arguments into self-documenting, order-independent named options.
- `...` means **spread** (expand a collection into elements) in a value position, and **rest** (collect elements into a collection) in a binding/parameter position — same syntax, opposite jobs, disambiguated by context.
- Spread merges (`{...a, ...b}` , `[...a, ...b]`) are shallow, just like `Object.assign` — nested structures still need their own explicit merge.

13.8 Exercises

1. Given `const [a, b, ...rest] = [1, 2, 3, 4, 5]` , predict all three values, then verify in `examples/13-destructuring-spread-rest.js` .
2. Write a function `omit(obj, keysToRemove)` using object destructuring rest that returns a new object without the specified keys.
3. Rewrite a function that currently takes 5 positional parameters (make one up) to instead take a single destructured options object with sensible defaults.

Chapter 14 — Scope and Closures

14.1 What is scope?

Scope answers one question: *from this line of code, which variables can I see?* Every variable you declare lives inside a scope, and JavaScript decides which scope a variable belongs to at the moment you write the code — not at the moment it runs. That property is called **lexical scoping** (or static scoping), and it's the foundation for everything in this chapter.

JavaScript has three kinds of scope:

- **Global scope** — variables declared outside any function or block. Visible everywhere in the program.
- **Function scope** — variables declared with `var` inside a function are visible anywhere in that function, regardless of nested blocks.
- **Block scope** — variables declared with `let` or `const` inside `{ }` (an `if`, `for`, `while`, or bare block) are visible only inside that block.

```
var globalVar = "I'm global";

function outer() {
  var functionScoped = "I'm function-scoped";

  if (true) {
    let blockScoped = "I'm block-scoped";
    var stillFunctionScoped = "var ignores the if-block";
    console.log(blockScoped); // fine, we're inside the block
  }

  console.log(functionScoped);      // fine
  console.log(stillFunctionScoped); // fine – var leaked out of the if
  console.log(typeof blockScoped);  // "undefined" – out of scope, not visible
}
```

The single most important practical rule in this chapter: **`var` is function-scoped, `let` and `const` are block-scoped**. This difference explains most of the "weird" bugs you'll hit with `var`, and it's why modern JavaScript style guides tell you to never use `var`.

14.2 The scope chain

When JavaScript looks up a variable, it doesn't just check the current scope — it walks *outward* through each enclosing scope until it finds the variable or runs out of scopes (at which point you get a `ReferenceError`). This chain of nested scopes is the **scope chain**.

```

const a = "global a";

function outer() {
  const b = "outer b";

  function inner() {
    const c = "inner c";
    console.log(a, b, c); // "global a" "outer b" "inner c"
    // inner() can see a, b, and c because it looks outward through
    // its own scope -> outer's scope -> global scope.
  }

  inner();
  console.log(typeof c); // "undefined" - outer() cannot see inner's c.
                          // Scope only looks OUTWARD, never inward.
}

outer();

```

The rule to memorize: **inner scopes can see outer variables, but outer scopes can never see inner variables.** Scope is a one-way mirror, and it's determined entirely by *where you physically wrote the code* — not by which function called which. This is what "lexical" means: fixed by the text (the "lexicon") of the program, not by the runtime call path.

14.3 Hoisting, precisely

"Hoisting" is the term for JavaScript's behavior of processing declarations before running any code in a scope. It is often explained as "declarations are moved to the top," which is a useful lie for beginners but not accurate enough for real debugging. Here's what actually happens: before executing a scope, the engine scans it and registers every declaration (`var`, `let`, `const`, `function`, `class`) in that scope's environment — but *how* each kind of declaration is initialized differs.

`var` — hoisted and initialized to `undefined`

```

console.log(myVar); // undefined (not a ReferenceError!)
var myVar = 5;
console.log(myVar); // 5

```

The declaration `var myVar` is hoisted to the top of the function/global scope and immediately initialized to `undefined`. The assignment `myVar = 5` stays exactly where you wrote it. This is why reading a `var` before its assignment gives `undefined` instead of crashing.

Function declarations — hoisted with their full body

```
sayHi(); // "Hi!" – works, because the whole function is hoisted

function sayHi() {
  console.log("Hi!");
}
```

Function declarations (`function name() {}`) are hoisted completely, including their body, so you can call them before their textual position.

`let` and `const` — hoisted but NOT initialized (the Temporal Dead Zone)

```
console.log(myLet); // ReferenceError: Cannot access 'myLet' before initialization
let myLet = 5;
```

`let` and `const` are hoisted — the engine knows about them before this line runs — but they are not given a value. The span between the start of the scope and the line where the `let / const` is declared is called the **Temporal Dead Zone (TDZ)**. Accessing the variable anywhere in the TDZ throws a `ReferenceError`, which is JavaScript intentionally failing loudly instead of silently giving you `undefined`. This is one of the practical reasons `let / const` are safer than `var`: they turn "used before declared" from a silent bug into a crash you notice immediately.

Function expressions and arrow functions — follow their variable's rule

```
sayBye(); // TypeError: sayBye is not a function (var hoisted as undefined)
var sayBye = function () {
  console.log("Bye!");
};

sayYo(); // ReferenceError: Cannot access 'sayYo' before initialization
const sayYo = () => console.log("Yo!");
```

A function *expression* assigned to a `var / let / const` is hoisted only as far as its variable declaration rule allows — the function body itself is not hoisted, only the variable binding. This trips up a lot of developers who assume "it's a function, so it must hoist like a function declaration."

Declaration	Hoisted?	Initial value	Accessing before declaration
<code>var x</code>	Yes	<code>undefined</code>	Returns <code>undefined</code>
<code>let x / const x</code>	Yes (binding only)	Uninitialized (TDZ)	<code>ReferenceError</code>
<code>function f(){} </code>	Yes, fully	The function itself	Works normally
<code>const f = () => {}</code>	Yes (binding only)	Uninitialized (TDZ)	<code>ReferenceError</code>

14.4 Closures: the definition that actually matters

A **closure** is what happens when a function "remembers" the variables from the scope it was defined in, even after that outer scope has finished executing. Every function in JavaScript forms a closure over its lexical scope — this isn't a special feature you opt into, it's just how scope chains work combined with functions being first-class values that can be returned or passed around.

```
function makeCounter() {
  let count = 0; // this variable lives inside makeCounter's scope

  return function increment() {
    count++; // increment "closes over" count
    return count;
  };
}

const counter1 = makeCounter();
const counter2 = makeCounter();

console.log(counter1()); // 1
console.log(counter1()); // 2
console.log(counter1()); // 3
console.log(counter2()); // 1 – counter2 has its OWN closure over its OWN count
```

Even though `makeCounter()` has already returned and its call-stack frame is long gone, the returned `increment` function still has access to `count`. The JavaScript engine keeps `count` alive in memory precisely *because* a live function still references it — this is a form of garbage collection awareness that's useful to understand: **closures keep their captured variables alive for as long as the closure itself is reachable**. This is powerful, but it also means closures can accidentally cause memory to be retained longer than you expect if you're not careful (e.g., a closure capturing a huge array that's never needed again).

Closures for private state

Before JavaScript had private class fields (`#field`, Chapter 17), closures were *the* way to create private variables:

```
function createBankAccount(initialBalance) {
  let balance = initialBalance; // not accessible from outside directly

  return {
    deposit(amount) {
      balance += amount;
      return balance;
    },
    withdraw(amount) {
      if (amount > balance) throw new Error("Insufficient funds");
      balance -= amount;
      return balance;
    },
    getBalance() {
      return balance;
    },
  };
}

const account = createBankAccount(100);
console.log(account.getBalance()); // 100
account.deposit(50);
console.log(account.getBalance()); // 150
console.log(account.balance);      // undefined – truly private, no back door
```

There is no way to reach into `account` and directly read or set `balance` except through the methods that closure exposes. This pattern is called the **module pattern**, and it's the ancestor of ES modules (Chapter 24).

Closures for memoization

```
function memoize(fn) {
  const cache = new Map(); // captured by the returned function

  return function (...args) {
    const key = JSON.stringify(args);
    if (cache.has(key)) {
      console.log("cache hit for", key);
      return cache.get(key);
    }
    const result = fn(...args);
    cache.set(key, result);
    return result;
  };
}

function slowSquare(n) {
  for (let i = 0; i < 1e8; i++) {} // pretend this is expensive
  return n * n;
}

const fastSquare = memoize(slowSquare);
fastSquare(5); // computes, slow
fastSquare(5); // cache hit, instant
```

14.5 The classic `var` + loop + `setTimeout` bug

This single example explains why `let` was added to the language and why "just use `let`" is such common advice.

```
// The bug
for (var i = 0; i < 3; i++) {
  setTimeout(function () {
    console.log("var i:", i);
  }, 100);
}
// Output (after ~100ms):
// var i: 3
// var i: 3
// var i: 3
```

Why? `var i` is function/global-scoped — there is only **one** `i` for the entire loop. All three `setTimeout` callbacks close over the *same* `i` variable. By the time any of them actually runs (100ms later, after the loop has already finished all three iterations), `i` is `3`.

```
// The fix: let
for (let i = 0; i < 3; i++) {
  setTimeout(function () {
    console.log("let i:", i);
  }, 100);
}
// Output:
// let i: 0
// let i: 1
// let i: 2
```

With `let`, the specification says each iteration of a `for` loop gets a **fresh binding** of `i`, copied from the previous iteration's value. Each `setTimeout` callback closes over its own separate `i`, so the values are preserved correctly.

Before `let` existed, developers fixed this with an **IIFE (Immediately Invoked Function Expression)** to manually create a new scope per iteration:

```
// The old-school fix: IIFE
for (var i = 0; i < 3; i++) {
  (function (capturedI) {
    setTimeout(function () {
      console.log("IIFE i:", capturedI);
    }, 100);
  })(i); // pass the CURRENT value of i in immediately
}
// Output: IIFE i: 0 / IIFE i: 1 / IIFE i: 2
```

Understanding both the bug and both fixes is a strong signal of real JavaScript fluency — it's one of the most common interview questions for a reason.

14.6 Nested closures and shared state

Multiple functions defined in the same scope share closures over the same variables — they don't each get their own copy:

```
function createToggle() {
  let isOn = false;

  return {
    toggle() {
      isOn = !isOn;
      return isOn;
    },
    status() {
      return isOn ? "ON" : "OFF";
    },
  };
}

const light = createToggle();
console.log(light.status()); // OFF
light.toggle();
console.log(light.status()); // ON – toggle() and status() share the same isOn
```

`toggle` and `status` are two different functions, but they both close over the *same* `isOn` variable, so changes made by one are visible to the other. This is exactly how a JavaScript module keeps internal state that multiple exported functions can read and mutate.

14.7 Chapter summary

- Scope determines variable visibility and is resolved lexically — by where code is *written*, not by how it's called.
- `var` is function-scoped; `let / const` are block-scoped. Prefer `let / const` always.
- All declarations are hoisted, but only `var` and function declarations get a usable value immediately; `let / const` sit in the Temporal Dead Zone until their line executes.
- A closure is a function bundled with a reference to its lexical scope — every function forms one automatically. Closures let variables outlive the function call that created them.
- Closures enable private state (module pattern), memoization, and callback factories, but can also retain memory if not managed carefully.
- The classic `var`-in-a-loop bug is caused by all iterations sharing one variable; `let` fixes it by giving each iteration its own binding.

14.8 Exercises

1. Predict the output before running: what does each `console.log` print in a loop using `var j` versus `let j` combined with `setTimeout`?
2. Write a `createIdGenerator()` function using a closure that returns a function which produces a new incrementing ID (1, 2, 3, ...) every time it's called, starting fresh for each generator instance.

3. Explain in your own words why `let x; console.log(x);` prints `undefined` at the top of a script, while `console.log(x); let x;` throws a `ReferenceError`.

Chapter 15 — The `this` Keyword

15.1 The one rule that fixes 90% of confusion

`this` is probably the single most misunderstood concept for developers coming from other languages. In many object-oriented languages, `this` (or `self`) always refers to "the instance the method was defined on." In JavaScript, that's not true at all. The rule that actually governs `this` is:

`this` is determined by *how* a function is called, not by *where* it is defined.

The same function can have a completely different `this` value every time it's called, depending on the call-site syntax. This chapter walks through every rule that determines `this`, in the priority order JavaScript actually applies them.

```
function whoAmI() {
  console.log(this);
}

const obj = { name: "obj", whoAmI };

whoAmI();      // this === undefined (strict mode) or globalThis (sloppy mode)
obj.whoAmI();  // this === obj
```

It's the *exact same function* both times — only the call syntax changed, and that alone changed what `this` refers to.

15.2 Rule 1 — Default binding (plain function call)

When a function is called with no context object in front of it (`fn()` , not `obj.fn()`), `this` uses "default binding":

```
"use strict";
function showThis() {
  console.log(this);
}
showThis(); // undefined, because this file/function runs in strict mode
```

```
// Without "use strict" (sloppy mode, the historical default):
function showThisSloppy() {
  console.log(this);
}
showThisSloppy(); // the global object (globalThis / window)
```

ES modules and class bodies are *always* strict mode automatically, which is one more reason plain function calls with `this` inside them are dangerous in modern code — you'll get `undefined` rather than accidentally mutating the global object.

15.3 Rule 2 — Implicit binding (method call)

When a function is called *as a property of an object* — `obj.method()` — `this` inside that function is bound to the object immediately to the left of the dot at the call site:

```
const user = {
  name: "Ada",
  greet() {
    console.log(`Hi, I'm ${this.name}`);
  },
};

user.greet(); // "Hi, I'm Ada" — this === user
```

The critical trap: **implicit binding is only determined by the call site, not by where the method was defined or which object "owns" it conceptually.** If you pull the method off the object and call it bare, you lose the binding:

```
const greetFn = user.greet;
greetFn(); // "Hi, I'm undefined" — this is no longer `user`!
```

This is exactly what happens when you pass a method as a callback:

```
function invokeLater(fn) {
  fn(); // called bare, with no object in front of it
}
invokeLater(user.greet); // "Hi, I'm undefined" — same problem
```

This single mechanism explains an enormous fraction of real-world "why is `this` undefined" bugs: `setTimeout(user.greet, 1000)`, `button.addEventListener("click", user.greet)`, and `array.map(obj.transform)` are all "bare calls" from JavaScript's perspective, even though it *looks* like the method still belongs to its object.

15.4 Rule 3 — Explicit binding: `call`, `apply`, and `bind`

JavaScript gives you a way to *force* what `this` will be, overriding the default and implicit rules.

- `fn.call(thisArg, arg1, arg2, ...)` — calls `fn` immediately, with `this` set to `thisArg`, passing arguments individually.
- `fn.apply(thisArg, [arg1, arg2, ...])` — identical to `call`, but arguments are passed as a single array.
- `fn.bind(thisArg, arg1, ...)` — does NOT call `fn`. Instead it returns a **new function** with `this` permanently locked to `thisArg` (this new binding cannot be overridden later, even by another `.call`).

```
function introduce(greeting) {
  console.log(`${greeting}, I'm ${this.name}`);
}

const ada = { name: "Ada Lovelace" };
const grace = { name: "Grace Hopper" };

introduce.call(ada, "Hello");    // "Hello, I'm Ada Lovelace"
introduce.apply(grace, ["Hi"]);  // "Hi, I'm Grace Hopper"

const introduceAda = introduce.bind(ada);
introduceAda("Hey");            // "Hey, I'm Ada Lovelace"
introduceAda.call(grace, "Yo");  // still "Yo, I'm Ada Lovelace" — bind wins!
```

The classic fix using `bind`

```
const timer = {
  seconds: 0,
  start() {
    setInterval(function () {
      this.seconds++; // BUG: `this` here is not `timer` (bare call by setInterval)
      console.log(this.seconds);
    }, 1000);
  },
};
```

```
const timerFixed = {
  seconds: 0,
  start() {
    setInterval(
      function () {
        this.seconds++;
        console.log(this.seconds);
      }.bind(this), // lock `this` to whatever `this` is right here (timerFixed)
      1000
    );
  },
};
```

15.5 Rule 4 — `new` binding

When a function is called with `new`, JavaScript creates a brand new empty object, sets `this` inside the function to that new object, and (unless the function explicitly returns another object) returns it automatically. This is covered in full mechanical detail in Chapter 16; for `this` purposes, the key point is that `new` creates a *fresh* `this` every single call:

```
function Person(name) {
  this.name = name; // this === the newly created object
}

const p1 = new Person("Ada");
const p2 = new Person("Grace");
console.log(p1.name, p2.name); // "Ada" "Grace" — separate `this` per call
```

15.6 Rule priority: which rule wins?

When more than one rule could apply, JavaScript resolves them in this order, from highest to lowest precedence:

1. **new binding** — `new fn()`
2. **Explicit binding** — `fn.call(obj)`, `fn.apply(obj)`, or a function already `.bind()`-locked
3. **Implicit binding** — `obj.fn()`
4. **Default binding** — `fn()` alone

```
function show() {
  console.log(this.label);
}

const bound = show.bind({ label: "bound object" });
const obj = { label: "obj", show: bound };

obj.show(); // "bound object" — explicit (bind) beats implicit (obj.show())
```

15.7 Arrow functions: no `this` of their own

Arrow functions are the one major exception to everything above. **Arrow functions do not have their own `this` binding at all.** Instead, they capture `this` lexically — exactly like a closure captures a variable — from the nearest enclosing non-arrow function (or the module/global scope if there is none). None of the four rules above apply to an arrow function; you cannot change an arrow function's `this` with `call`, `apply`, or `bind`.

```
const user2 = {
  name: "Ada",
  greetArrow: () => {
    // this is captured from the surrounding scope at definition time -
    // here that's the module/global scope, NOT user2.
    console.log(this?.name); // undefined
  },
  greetRegular() {
    console.log(this.name); // "Ada" - implicit binding, correct
  },
};
user2.greetArrow();
user2.greetRegular();
```

This looks like a limitation, but it's exactly the tool you need to fix the `setTimeout` /callback problem cleanly, *without* `bind` or a `self = this` workaround:

```
const timerArrow = {
  seconds: 0,
  start() {
    setInterval(() => {
      // arrow function: `this` is captured from `start()`'s scope,
      // where `this` correctly refers to `timerArrow`.
      this.seconds++;
      console.log(this.seconds);
    }, 1000);
  },
};
```

This is why "use an arrow function for callbacks" became standard modern JavaScript advice: arrow functions simply don't participate in the rebinding problem at all.

The old-school fix: `const self = this`

Before arrow functions existed (pre-ES6), developers solved this by capturing `this` in a regular variable that a closure *could* see (since closures capture variables, just not `this` specifically):

```
const timerOldSchool = {
  seconds: 0,
  start() {
    const self = this; // capture into an ordinary variable
    setInterval(function () {
      self.seconds++; // `self` is just a normal closed-over variable
      console.log(self.seconds);
    }, 1000);
  },
};
```

You'll still see `self` / `that` / `_this` patterns in older codebases — now you know exactly what problem they're solving.

15.8 `this` inside classes

Class methods behave like implicit-binding object methods: `this` refers to the instance when called as `instance.method()`. The same "bare call loses `this`" trap applies to class methods just as it does to plain object methods — this is extremely common with event handlers:

```
class Button {
  constructor(label) {
    this.label = label;
  }
  handleClick() {
    console.log(`${this.label} clicked`);
  }
}

const btn = new Button("Submit");
// btn.handleClick(); // works: "Submit clicked"
// domElement.addEventListener("click", btn.handleClick); // BUG: bare call, this is unde
```

Two idiomatic fixes are common in real class-based UI code:

```

class ButtonFixedBind {
  constructor(label) {
    this.label = label;
    this.handleClick = this.handleClick.bind(this); // bind once, in the constructor
  }
  handleClick() {
    console.log(`${this.label} clicked`);
  }
}

class ButtonFixedArrow {
  label;
  constructor(label) {
    this.label = label;
  }
  // class field holding an arrow function: `this` is lexically captured
  // at the point the field is defined (i.e. bound to the instance).
  handleClick = () => {
    console.log(`${this.label} clicked`);
  };
}

```

15.9 Chapter summary

- `this` is set by the call site — how a function is invoked — not by where the function is defined.
- Priority order: `new` binding > explicit (`call` / `apply` / `bind`) > implicit (`obj.method()`) > default (bare `fn()`).
- Passing a method as a callback (`setTimeout(obj.method)` , `addEventListener("click", obj.method)`) strips its implicit binding — this is the single most common source of `this` -related bugs.
- `.bind()` returns a new function permanently locked to a given `this` and cannot be re-bound later.
- Arrow functions have no `this` of their own; they inherit it lexically from their enclosing scope, which makes them ideal for callbacks that need to preserve an outer `this` .
- In classes, bind event-handler methods in the constructor or define them as arrow-function class fields to avoid losing `this` .

15.10 Exercises

1. Given `const obj = { val: 42, getVal() { return this.val; } }`, write three different ways to call `getVal` such that it returns `undefined` or throws, and one way (other than `obj.getVal()`) that correctly returns `42` .
2. Explain why `array.forEach(obj.method)` can silently break `this` even though the code "looks fine," and show the fix using an arrow function.

3. Without running it, predict what `new (function () { console.log(this); })()` logs, and why `new` binding takes priority over everything else.

Chapter 16 — Prototypes and Inheritance

16.1 JavaScript is prototype-based, not class-based

Before ES6 introduced `class` syntax, JavaScript had no `class` keyword at all — and even today, `class` is **syntactic sugar** over the same mechanism that has always powered JavaScript: **prototypal inheritance**. Understanding prototypes is understanding what `class` actually compiles down to, and it explains a large category of behavior that otherwise looks like magic.

The core idea: every JavaScript object has an internal, hidden link to another object called its **prototype**. When you access a property on an object and that property isn't found directly on the object itself, JavaScript automatically looks it up on the object's prototype. If it's not there either, JavaScript looks at the prototype's prototype, and so on, until it either finds the property or reaches the end of the chain (`null`). This chain of linked objects is the **prototype chain**.

```
const animal = {
  eats: true,
  describe() {
    return "I am an animal";
  },
};

const dog = Object.create(animal); // dog's prototype is `animal`
dog.barks = true;

console.log(dog.eats);           // true — not on `dog`, found on its prototype `animal`
console.log(dog.barks);         // true — found directly on `dog`
console.log(dog.describe());    // "I am an animal" — method found via the chain
```

`dog` doesn't have an `eats` property of its own — you can verify this with `dog.hasOwnProperty("eats")` (`false`) vs `dog.hasOwnProperty("barks")` (`true`). The property lookup *walked up* the prototype chain to find `eats` on `animal`.

16.2 Inspecting and setting the prototype

```
console.log(Object.getPrototypeOf(dog) === animal); // true
console.log(dog.__proto__ === animal);              // true (legacy accessor, same thing)

const cat = {};
Object.setPrototypeOf(cat, animal);
console.log(cat.eats); // true
```

`Object.getPrototypeOf()` / `Object.setPrototypeOf()` are the modern, recommended API.

`__proto__` is a legacy accessor property that does the same thing but is considered deprecated for direct use (it's kept only for backward compatibility with old browser code) — `Object.create` and `Object.getPrototypeOf` / `setPrototypeOf` are preferred in new code.

Every object literal you create with `{}` automatically gets `Object.prototype` as its prototype (unless you override it), which is where all the familiar methods like `.hasOwnProperty()`, `.toString()`, and `.valueOf()` actually live:

```
const plain = {};  
console.log(Object.getPrototypeOf(plain) === Object.prototype); // true  
console.log(Object.getPrototypeOf(Object.prototype));           // null – end of the chain
```

`Object.prototype` is where the chain terminates for ordinary objects — its own prototype is `null`.

16.3 Constructor functions and `.prototype`

Before `class`, JavaScript developers created "instances" using regular functions called with `new`, combined with a special property every function automatically has: `.prototype`.

Important terminology trap: every *function* has a `.prototype` *property* (an object that will become the prototype of instances created with `new`), which is completely different from the internal prototype *link* every object has (accessible via `Object.getPrototypeOf`). Beginners conflate these constantly.

```
function Animal(name) {  
  this.name = name;  
}  
  
// Methods go on the constructor's .prototype, NOT redefined per instance.  
Animal.prototype.describe = function () {  
  return `I am ${this.name}`;  
};  
  
const rex = new Animal("Rex");  
console.log(rex.describe());           // "I am Rex"  
console.log(Object.getPrototypeOf(rex) === Animal.prototype); // true
```

Putting `describe` on `Animal.prototype` instead of inside the constructor means every instance *shares one copy* of the function in memory, rather than each instance carrying its own separate copy — a meaningful memory and performance win when you create many instances.

```
// BAD: wastes memory – every instance gets its own copy of the function
function AnimalWasteful(name) {
  this.name = name;
  this.describe = function () {
    return `I am ${this.name}`;
  };
}
```

16.4 What `new` actually does, step by step

`new Animal("Rex")` performs four steps, in order:

1. A brand-new, empty object is created: `{}`.
2. That new object's internal prototype is linked to `Animal.prototype` (i.e., `Object.setPrototypeOf(newObj, Animal.prototype)`).
3. The constructor function runs with `this` bound to the new object (implementing Rule 4: `new` binding from Chapter 15).
4. If the constructor function doesn't explicitly return an object of its own, `new` automatically returns the new object that was built in steps 1–3. (If the function *does* return some other object, that object is returned instead and the newly created one is discarded — a rarely used escape hatch.)

Here is that exact sequence written out manually, to fully demystify it:

```
function myNew(Constructor, ...args) {
  const newObj = {}; // step 1
  Object.setPrototypeOf(newObj, Constructor.prototype); // step 2
  const result = Constructor.apply(newObj, args); // step 3
  return (typeof result === "object" && result !== null) ? result : newObj; // step 4
}

function Animal2(name) {
  this.name = name;
}
Animal2.prototype.describe = function () {
  return `I am ${this.name}`;
};

const manualRex = myNew(Animal2, "Rex");
console.log(manualRex.describe()); // "I am Rex" – behaves identically to `new Animal2("R
```

Forgetting `new`

If you call a constructor function *without* `new`, none of the four steps happen — it's just a regular function call, and `this` follows the normal rules from Chapter 15 (in strict mode, `this` is `undefined`):

```
function Car(make) {
  "use strict";
  this.make = make; // TypeError: Cannot set properties of undefined
}
// const oops = Car("Toyota"); // throws, because Car() has no `new`, this is undefined
```

This is one reason class constructors (Chapter 17) throw an explicit error if called without `new` — the language decided to make this footgun impossible for `class`, even though it's still possible for old-style constructor functions.

16.5 `instanceof` and property shadowing

`instanceof` checks whether an object's prototype chain contains a given constructor's `.prototype` object anywhere along it:

```
console.log(rex instanceof Animal); // true – Animal.prototype is in rex's chain
console.log(rex instanceof Object); // true – Object.prototype is further up the chain
console.log({} instanceof Animal); // false – plain object's chain never reaches Animal.
```

Property shadowing: if an instance defines its own property with the same name as something on its prototype, the instance's own property "wins" — the lookup finds it first and never continues up the chain:

```
const rex2 = new Animal("Rex");
console.log(rex2.describe()); // "I am Rex" (from Animal.prototype)

rex2.describe = function () {
  return "I'm overriding the prototype method!";
};
console.log(rex2.describe()); // "I'm overriding the prototype method!" (own property wins)

delete rex2.describe;
console.log(rex2.describe()); // back to "I am Rex" – shadowing removed, chain lookup resumes
```

This is exactly how method overriding works in prototype-based inheritance — and it's exactly what `super.method()` inside a subclass lets you bypass deliberately (Chapter 17).

16.6 Building an inheritance chain manually

Before `class extends` existed, this multi-step dance was how you built inheritance between two constructor functions. Seeing it written out in full is the best way to understand what `extends` (Chapter 17) does for you automatically.

```
function Animal3(name) {
  this.name = name;
}
Animal3.prototype.describe = function () {
  return `${this.name} is an animal`;
};

function Dog(name, breed) {
  Animal3.call(this, name); // "super constructor" call: reuse Animal3's setup logic
  this.breed = breed;
}

// Link Dog.prototype's prototype to Animal3.prototype (this is the inheritance step)
Dog.prototype = Object.create(Animal3.prototype);
Dog.prototype.constructor = Dog; // repair the constructor reference Object.create broke

// Dog can now add or override methods
Dog.prototype.bark = function () {
  return `${this.name} says woof!`;
};
Dog.prototype.describe = function () {
  // call the "parent" version explicitly, then extend it
  const base = Animal3.prototype.describe.call(this);
  return `${base} (specifically, a ${this.breed})`;
};

const fido = new Dog("Fido", "Labrador");
console.log(fido.describe()); // "Fido is an animal (specifically, a Labrador)"
console.log(fido.bark()); // "Fido says woof!"
console.log(fido instanceof Dog); // true
console.log(fido instanceof Animal3); // true – inheritance chain confirmed
```

`Object.create(Animal3.prototype)` is the crucial line: it creates a new, empty object whose prototype is `Animal3.prototype`, and assigns that as `Dog.prototype`. Now any `Dog` instance's chain is: `fido -> Dog.prototype -> Animal3.prototype -> Object.prototype -> null`.

Common mistake: writing `Dog.prototype = Animal3.prototype` directly (instead of `Object.create(Animal3.prototype)`). This makes `Dog` and `Animal3` share the *exact same* prototype object, so adding a method to `Dog.prototype` would also silently add it to `Animal3.prototype` and every plain `Animal3` instance. `Object.create` avoids this by creating a fresh, separate object that merely *links to* `Animal3.prototype` rather than *being* it.

16.7 Prototypal vs. classical inheritance

Classical inheritance (Java, C++, C#) works by copying behavior at *compile time* into a rigid class hierarchy defined up front. Prototypal inheritance works by *linking* objects together at runtime, and lookups happen dynamically at the moment a property is accessed. This has a few consequences that surprise developers coming from classical languages:

- You can change an object's prototype at runtime (`Object.setPrototypeOf`) — extremely unusual in classical OOP, and generally discouraged for performance reasons (it deoptimizes V8's internal object shape assumptions), but it demonstrates how dynamic the model is.
- You can create an object that inherits from another *specific object* directly, with `Object.create(someObject)` , without ever defining a "class" at all.
- Adding a method to a prototype *after* instances already exist still makes it available to those existing instances immediately, because lookup happens at access time, not at creation time:

```
function Widget() {}
const w = new Widget();
Widget.prototype.render = function () {
  return "rendering!";
};
console.log(w.render()); // "rendering!" – works even though w was created first
```

16.8 Chapter summary

- Every object has an internal prototype link; property lookups walk up the prototype chain until found or the chain ends at `null` .
- `Object.getPrototypeOf` / `setPrototypeOf` inspect/change an object's prototype link; `__proto__` does the same but is legacy.
- Functions have a `.prototype` *property* (distinct from the object prototype *link*) that becomes new instances' prototype when used with `new` .
- `new` performs four steps: create an object, link its prototype, run the constructor with `this` bound to it, and return it (unless the constructor returns its own object).
- Property shadowing means an instance's own property hides a same-named property found further up the prototype chain.
- `class` / `extends` (Chapter 17) is sugar over exactly the manual pattern shown in 16.6: `Object.create` to link prototypes, and `Parent.call(this, ...)` to reuse parent setup logic.

16.9 Exercises

1. Given `function Foo() {}` and `const f = new Foo();` , list every object in `f` 's full prototype chain up to `null` .
2. Explain, in your own words, why `Dog.prototype = Object.create(Animal.prototype)` is correct but `Dog.prototype = Animal.prototype` is a bug.

3. Write a `Shape` constructor with a `.area()` method returning `0`, then a `Circle` constructor (taking a radius) that inherits from `Shape` and overrides `.area()` to return the correct circle area.

Chapter 17 — Classes

17.1 Classes are sugar over prototypes

ES6 introduced the `class` keyword, giving JavaScript syntax that *looks* like classical object-oriented languages. Under the hood, a `class` still builds the exact same prototype-chain structure covered in Chapter 16 — `class` did not change JavaScript's object model, it just gave it a cleaner, less error-prone syntax and closed off some of the sharpest footguns (like forgetting `new`, discussed below).

```
class Animal {
  constructor(name) {
    this.name = name;
  }
  describe() {
    return `I am ${this.name}`;
  }
}

const rex = new Animal("Rex");
console.log(rex.describe());           // "I am Rex"
console.log(typeof Animal);            // "function" – a class IS a function
console.log(Object.getPrototypeOf(rex) === Animal.prototype); // true – same mechanism as
console.log(Animal.prototype.describe === rex.describe);       // true – method lives on t
```

`describe` is defined once on `Animal.prototype`, exactly like the manual `Animal.prototype.describe = function() {...}` pattern from Chapter 16 — `class` just writes that boilerplate for you.

17.2 Constructors, instance methods, and enforced `new`

The `constructor` method runs once per `new` call and sets up instance state. Unlike old-style constructor functions, **class constructors throw a `TypeError` if called without `new`** — one of several footguns `class` closes off deliberately:

```
class Point {
  constructor(x, y) {
    this.x = x;
    this.y = y;
  }
  toString() {
    return `(${this.x}, ${this.y})`;
  }
}

// Point(1, 2); // TypeError: Class constructor Point cannot be invoked without 'new'
const p = new Point(1, 2);
console.log(p.toString()); // "(1, 2)"
console.log(`${p}`);       // "(1, 2)" – template literals call toString() automatically
```

Every method written inside a `class` body (other than `constructor`) automatically becomes a **non-enumerable** property on the prototype (unlike a plain object literal method, which is enumerable). This means `for...in` and `Object.keys()` won't list class methods, matching the behavior of built-in methods like `Array.prototype.map`.

17.3 Static methods and static fields

`static` members belong to the class itself, not to instances — useful for utility/factory functions logically related to the class:

```

class Circle {
  static PI = 3.14159; // static field (class-level "constant")

  constructor(radius) {
    this.radius = radius;
  }

  area() {
    return Circle.PI * this.radius ** 2;
  }

  static fromDiameter(diameter) {
    // static factory method: an alternative way to construct instances
    return new Circle(diameter / 2);
  }
}

console.log(Circle.PI);           // 3.14159 – accessed on the class, not an instance
const c1 = new Circle(2);
console.log(c1.area().toFixed(2)); // "12.57"
const c2 = Circle.fromDiameter(10);
console.log(c2.radius);           // 5
// c1.PI is undefined – static members are NOT inherited onto instances
console.log(c1.PI);               // undefined

```

17.4 Getters and setters

Getters/setters let you define properties that run code on read/write while still being accessed with plain property syntax (`obj.prop` , not `obj.prop()`):

```

class Temperature {
  #celsius; // private field, see 17.5

  constructor(celsius) {
    this.#celsius = celsius;
  }

  get fahrenheit() {
    return this.#celsius * (9 / 5) + 32;
  }

  set fahrenheit(f) {
    this.#celsius = (f - 32) * (5 / 9);
  }

  get celsius() {
    return this.#celsius;
  }
}

const temp = new Temperature(25);
console.log(temp.fahrenheit); // 77 – reads like a plain property, runs code
temp.fahrenheit = 32;
console.log(temp.celsius.toFixed(1)); // "0.0" – write also ran code, converted back

```

17.5 Private fields (#field)

Before ES2022's `#`-prefixed private fields, JavaScript had no true built-in privacy at the class level — only the closure-based module pattern from Chapter 14 could hide state completely. Private fields close that gap directly in class syntax. A field or method prefixed with `#` is only accessible from inside the class body — not from outside, not even via `Object.keys` or bracket-notation tricks:

```

class BankAccount {
  #balance; // must be declared here (or in the constructor) before use

  constructor(initialBalance) {
    this.#balance = initialBalance;
  }

  deposit(amount) {
    this.#balance += amount;
    return this.#balance;
  }

  #validateWithdrawal(amount) {
    // private METHOD – internal helper, not part of the public API
    if (amount > this.#balance) throw new Error("Insufficient funds");
  }

  withdraw(amount) {
    this.#validateWithdrawal(amount);
    this.#balance -= amount;
    return this.#balance;
  }

  get balance() {
    return this.#balance;
  }
}

const account = new BankAccount(100);
console.log(account.balance); // 100 (via the public getter)
account.deposit(50);
console.log(account.balance); // 150
console.log(account.balance); // still fine – public getter is allowed
// console.log(account.#balance); // SyntaxError: outside the class body, # is illegal
console.log(account["#balance"]); // undefined – bracket notation cannot reach it either

```

Unlike the closure-based module pattern, `#` fields are a real part of the class's shape, checked at parse time — accessing `#field` from outside the class isn't just `undefined`, it's a `SyntaxError` before the code even runs.

17.6 `extends` and `super`

`extends` sets up the prototype chain between two classes automatically — exactly the `Object.create(Parent.prototype)` wiring you built by hand in Chapter 16. `super` gives you two related but distinct things: calling `super(...)` in a constructor invokes the parent's constructor, and calling `super.method()` in a method invokes the parent's version of that method.

```

class Animal {
  constructor(name) {
    this.name = name;
  }
  describe() {
    return `${this.name} is an animal`;
  }
  speak() {
    return `${this.name} makes a sound`;
  }
}

class Dog extends Animal {
  constructor(name, breed) {
    super(name); // MUST call super() before using `this` in a subclass constructor
    this.breed = breed;
  }

  describe() {
    // call the parent version, then extend its result
    return `${super.describe()} (specifically, a ${this.breed})`;
  }

  speak() {
    return `${this.name} barks`; // fully overrides the parent version, no super call
  }
}

const fido = new Dog("Fido", "Labrador");
console.log(fido.describe()); // "Fido is an animal (specifically, a Labrador)"
console.log(fido.speak());    // "Fido barks"
console.log(fido instanceof Dog);    // true
console.log(fido instanceof Animal); // true

```

A hard rule: in a subclass constructor, you cannot access `this` at all until `super()` has been called — the parent constructor is responsible for actually creating the instance's initial state (this mirrors the `new` mechanics from Chapter 16, where a "super constructor call" is exactly `Parent.call(this, ...)`).

```

class Broken extends Animal {
  constructor(name) {
    this.name = name; // ReferenceError: must call super() before accessing 'this'
    super(name);
  }
}

```

17.7 Method overriding and polymorphism

Because JavaScript resolves methods dynamically through the prototype chain, a collection of different subclass instances can be treated uniformly — call the same method name on each, and the correct "overridden" version runs automatically. This is **polymorphism**:

```
class Shape {
  area() {
    return 0;
  }
}
class Square extends Shape {
  constructor(side) {
    super();
    this.side = side;
  }
  area() {
    return this.side ** 2;
  }
}
class Circle2 extends Shape {
  constructor(radius) {
    super();
    this.radius = radius;
  }
  area() {
    return Math.PI * this.radius ** 2;
  }
}

const shapes = [new Square(4), new Circle2(3), new Shape()];
for (const shape of shapes) {
  console.log(shape.constructor.name, "area:", shape.area().toFixed(2));
}
// Square area: 16.00
// Circle2 area: 28.27
// Shape area: 0.00
```

17.8 Abstract-class-like patterns

JavaScript has no built-in `abstract` keyword, but you can approximate an abstract base class (a class meant only to be extended, never instantiated directly) by checking in the constructor:

```

class AbstractShape {
  constructor() {
    if (new.target === AbstractShape) {
      throw new TypeError("AbstractShape cannot be instantiated directly");
    }
  }
  area() {
    throw new Error("Subclasses must implement area()");
  }
}

class Triangle extends AbstractShape {
  constructor(base, height) {
    super();
    this.base = base;
    this.height = height;
  }
  area() {
    return (this.base * this.height) / 2;
  }
}

// new AbstractShape(); // throws TypeError
console.log(new Triangle(6, 4).area()); // 12

```

`new.target` refers to the constructor that was actually invoked with `new` — for a direct `new AbstractShape()` call it equals `AbstractShape`, but when constructing via a subclass it equals that subclass instead, which is exactly the distinction this pattern needs.

17.9 Mixins: composing behavior without deep hierarchies

JavaScript classes only support single inheritance (`extends` takes one parent). **Mixins** are a pattern for sharing behavior across unrelated classes by writing functions that take a base class and return an extended version of it:

```

const Serializable = (Base) =>
  class extends Base {
    toJSON() {
      // Return a plain object, NOT JSON.stringify(this) – JSON.stringify()
      // automatically calls .toJSON() on any object that defines one, so
      // calling it on `this` from inside toJSON() would recurse forever.
      return { ...this };
    }
  };

const Comparable = (Base) =>
  class extends Base {
    equals(other) {
      // toJSON() returns a fresh object each call, so compare the
      // serialized strings rather than the objects themselves with ===.
      return JSON.stringify(this.toJSON()) === JSON.stringify(other.toJSON());
    }
  };

class Point {
  constructor(x, y) {
    this.x = x;
    this.y = y;
  }
}

class EnhancedPoint extends Comparable(Serializable(Point)) {}

const p1 = new EnhancedPoint(1, 2);
const p2 = new EnhancedPoint(1, 2);
console.log(p1.toJSON());    // { x: 1, y: 2 }
console.log(p1.equals(p2));  // true – behavior from BOTH mixins, composed

```

17.10 Common class design mistakes

- **Deep inheritance chains** (`A extends B extends C extends D`) tend to become fragile — a change to a distant ancestor can break far-away subclasses in surprising ways ("the fragile base class problem"). Prefer composition (mixins, or simply passing collaborator objects in) over deep hierarchies where practical.
- **Putting mutable shared state on a static field** and expecting each instance to have its own copy — static fields are shared across *all* instances, exactly like `Animal.prototype` methods are shared, not copied.
- **Overriding a method but forgetting `super.method()`** when the subclass was supposed to *extend* behavior rather than fully replace it — silently dropping the parent's logic.
- **Using classes for things that are really just data** — a plain object or a factory function (Chapter 33) is often simpler and avoids `this` pitfalls entirely when you don't actually need inheritance or private state.

17.11 Chapter summary

- `class` is syntactic sugar over the prototype mechanism from Chapter 16 — methods still live on `.prototype` and are shared across instances.
- Class constructors throw if called without `new`, unlike old-style constructor functions.
- `static` members belong to the class itself, not instances; they are not inherited by instances.
- Getters/setters (`get` / `set`) let property-style syntax run custom logic.
- `#field` gives true, enforced-at-parse-time private state and methods.
- `extends` / `super` wire up the prototype chain and let subclass constructors/methods call into their parent's implementation.
- Mixins compose behavior across unrelated classes when single inheritance isn't enough — prefer composition over deep inheritance chains.

17.12 Exercises

1. Write a `Stack` class with private `#items`, and `push`, `pop`, `peek`, and a `size` getter. Verify from outside the class that `#items` cannot be accessed directly.
2. Create an `Employee` class and a `Manager` subclass that overrides a `describe()` method but still calls `super.describe()` to include the base description.
3. Explain why `new.target === AbstractShape` correctly distinguishes a direct instantiation from a subclass instantiation, referencing how `super()` sets up `this` from Chapter 16's `new` mechanics.

Chapter 18 — Error Handling

18.1 Why errors need a strategy, not just a reaction

Every non-trivial program encounters situations it cannot proceed through normally: a file doesn't exist, a network request times out, a user types "abc" into a field expecting a number. JavaScript gives you a structured way to signal that something went wrong (`throw`) and a structured way to react to it (`try / catch / finally`). Learning to use these deliberately — rather than sprinkling defensive `if` checks everywhere — is what separates code that fails predictably from code that fails mysteriously in production at 3 a.m.

18.2 `try`, `catch`, and `finally`

```
try {
  const result = JSON.parse("{ invalid json");
  console.log(result); // never reached
} catch (error) {
  console.log("Parsing failed:", error.message);
} finally {
  console.log("This always runs, error or not.");
}
```

- Code in `try` runs normally until something throws.
- If something throws, execution jumps immediately to `catch`, skipping the rest of the `try` block entirely — even code several lines after the point of failure never runs.
- `finally` runs **unconditionally** — whether the `try` succeeded, threw and was caught, or even if the `catch` block itself re-throws. It's the right place for cleanup that must always happen (closing a file handle, hiding a loading spinner, releasing a lock).

```
function readConfig() {
  console.log("opening resource...");
  try {
    throw new Error("disk read failed");
  } finally {
    console.log("closing resource..."); // still runs, even though nothing catches the error
  }
}
try {
  readConfig();
} catch (e) {
  console.log("caught at outer level:", e.message);
}
// Output order: "opening resource...", "closing resource...", "caught at outer level: disk read failed"
```

Optional catch binding

If you don't need the error object itself, you can omit the parentheses entirely (ES2019+):

```
try {
  riskyOperation();
} catch {
  console.log("Something went wrong, but we don't need the details here.");
}
```

18.3 `throw`: you can throw anything, but you shouldn't

JavaScript technically lets you `throw` any value — a string, a number, a plain object — but you should always throw an `Error` object (or a subclass of it). `Error` objects automatically capture a **stack trace** (`error.stack`) showing exactly where the error originated and the call path that led there — information you permanently lose if you throw a bare string.

```
function divide(a, b) {
  if (b === 0) {
    throw new Error("Division by zero");
  }
  return a / b;
}

try {
  divide(10, 0);
} catch (error) {
  console.log(error.message);           // "Division by zero"
  console.log(error instanceof Error); // true
  console.log(typeof error.stack);      // "string" — a full stack trace
}
```

18.4 Custom error types

Subclassing `Error` (using the `class / extends` mechanics from Chapter 17) lets you create semantically meaningful error types that calling code can distinguish between with `instanceof`, instead of parsing error message strings (which is brittle — messages change, types shouldn't).

```
class ValidationError extends Error {
  constructor(message, field) {
    super(message); // sets this.message, sets up the stack trace
    this.name = "ValidationError"; // overrides the default "Error" name shown in stack t
    this.field = field; // custom data specific to this error type
  }
}

class NotFoundError extends Error {
  constructor(resource) {
    super(`${resource} not found`);
    this.name = "NotFoundError";
  }
}

function validateAge(age) {
  if (typeof age !== "number" || age < 0) {
    throw new ValidationError("Age must be a non-negative number", "age");
  }
  return age;
}

try {
  validateAge(-5);
} catch (error) {
  if (error instanceof ValidationError) {
    console.log(`Validation failed on field "${error.field}": ${error.message}`);
  } else if (error instanceof NotFoundError) {
    console.log("Not found:", error.message);
  } else {
    throw error; // unknown error type: re-throw, don't silently swallow it
  }
}
```

Re-throwing unknown errors in the last `else` branch is an important habit: a `catch` block that's only prepared to handle specific error types should let anything else propagate up rather than silently hiding a bug it doesn't understand.

18.5 Error propagation

An uncaught error inside a function doesn't just vanish — it propagates up through every enclosing function call until something catches it, or, if nothing does, it crashes the program (in Node.js) or gets logged to the console (in browsers).

```
function level3() {
  throw new Error("failure deep in level3");
}
function level2() {
  level3(); // no try/catch here – the error just passes through
}
function level1() {
  level2(); // no try/catch here either
}

try {
  level1();
} catch (error) {
  console.log("Caught at the top:", error.message); // "Caught at the top: failure deep i
}
```

This is a deliberate and useful design: you don't need to handle every possible error at the exact point it might occur. You can let errors "bubble up" to a place in your code that actually knows what a sensible recovery looks like (e.g., a top-level request handler that returns a 500 response) instead of littering every low-level function with try/catch blocks it can't meaningfully act on.

18.6 try / catch with async / await (preview)

Chapters 21–23 cover asynchronous JavaScript in depth, but it's worth previewing here: try / catch works with await ed promises exactly like it works with synchronous throws, which is one of the biggest ergonomic wins async / await brought over raw promise chains.

```
async function fetchUserSafely(id) {
  try {
    const response = await fetch(`/api/users/${id}`); // hypothetical call
    if (!response.ok) {
      throw new Error(`Request failed with status ${response.status}`);
    }
    return await response.json();
  } catch (error) {
    console.log("Failed to fetch user:", error.message);
    return null;
  }
}
```

A rejected promise behaves, inside an async function, exactly like a synchronous throw — it's caught by the nearest enclosing try/catch.

18.7 Defensive programming vs. fail-fast

There are two broad philosophies for handling invalid input or unexpected state, and good engineers apply both, deliberately, in different places:

- **Fail-fast:** throw immediately when something is wrong, as close to the source of the problem as possible. This makes bugs loud and obvious during development, rather than letting bad data silently propagate and cause a confusing failure somewhere unrelated much later.
- **Defensive/graceful degradation:** catch expected failure modes (a network timeout, a missing optional field) and provide a sensible fallback so the whole program doesn't crash over something recoverable.

The mistake to avoid is applying defensive catch-and-ignore everywhere, which hides real bugs:

```
// BAD: swallows every error, including genuine bugs, silently
function getUsernameBad(user) {
  try {
    return user.profile.name.toUpperCase();
  } catch {
    return "Unknown"; // masks a typo, a null profile, AND a real crash, identically
  }
}

// BETTER: validate explicitly, fail loudly on truly unexpected shapes
function getUsernameBetter(user) {
  if (!user?.profile?.name) {
    return "Unknown"; // an EXPECTED, named condition, not a caught exception
  }
  return user.profile.name.toUpperCase();
}
```

18.8 `console.error` / `console.warn` vs. throwing

Not every problem should halt execution. `console.error` / `console.warn` are for *logging* — surfacing information to a developer without changing control flow — while `throw` actually interrupts execution and forces the caller to deal with the problem (or crash). A rough guideline:

- Use `throw` when the function **cannot** produce a valid result and the caller must be forced to handle that (or explicitly choose not to).
- Use `console.warn` for something suspicious but recoverable (a deprecated option was passed, but the function can still proceed sensibly).
- Use `console.error` for logging an error you've already caught and handled, when you still want a visible trace for debugging.

18.9 Global error handlers (brief)

As a last line of defense, both browsers and Node.js let you catch errors that escaped every `try/catch` in your code — useful for logging/reporting tools, not for normal control flow:

```
// Browser:
window.addEventListener("error", (event) => {
  console.log("Uncaught error:", event.message);
});
window.addEventListener("unhandledrejection", (event) => {
  console.log("Unhandled promise rejection:", event.reason);
});

// Node.js:
process.on("uncaughtException", (error) => {
  console.log("Uncaught exception:", error.message);
  process.exit(1); // best practice: exit after an uncaught exception, state may be corrupt
});
process.on("unhandledRejection", (reason) => {
  console.log("Unhandled rejection:", reason);
});
```

These are safety nets for logging/monitoring (e.g., reporting to an error tracking service), not a substitute for handling errors properly at the point they occur.

18.10 Chapter summary

- `try / catch / finally` structure error handling: `catch` runs on a thrown error, `finally` runs unconditionally for cleanup.
- Always `throw new Error(...)` (or a subclass), never a bare string or plain object — `Error` objects carry a stack trace.
- Subclass `Error` to create semantically distinct, `instanceof`-checkable custom error types instead of parsing message strings.
- Uncaught errors propagate up through the call stack until something catches them or the program crashes — this lets you centralize handling where it's actually meaningful.
- `try/catch` works identically for `await`ed promise rejections as for synchronous throws (previewed here, covered fully in Chapters 21–23).
- Balance fail-fast (throw loudly, close to the bug) against defensive fallback (catch genuinely recoverable, expected conditions) — don't blanket-swallow every error, which hides real bugs.
- `console.error / warn` are for logging; `throw` is for control flow.

18.11 Exercises

1. Write a `parseJsonSafely(str)` function that returns the parsed value on success, or `null` on failure, without letting a malformed JSON string crash the caller.
2. Create a custom `InsufficientFundsError` extends `Error` and use it in a `withdraw(balance, amount)` function; catch it specifically and print a friendly message, while letting any other unexpected error propagate.
3. Predict the console output order of a `try` block that logs `"A"`, throws, is caught and logs `"B"`, with a `finally` that logs `"C"`.

Chapter 19 — Higher-Order Functions & Functional Programming

19.1 Functions as first-class values

In JavaScript, functions are **first-class values**: they can be stored in variables, put inside arrays or objects, passed as arguments to other functions, and returned from other functions — exactly like a number or a string. This single fact underlies closures (Chapter 14), callbacks (Chapter 21), array methods (Chapter 8), and most of the functional programming techniques in this chapter.

```
const square = function (x) {  
  return x * x;  
};  
  
const operations = {  
  double: (x) => x * 2,  
  square,  
};  
  
const functionList = [square, operations.double];  
  
console.log(operations.square(5)); // 25  
console.log(functionList[1](5));  // 10
```

19.2 Higher-order functions defined

A **higher-order function** is any function that does at least one of:

1. **takes another function as an argument**, or
2. **returns a function as its result**.

You've already been using higher-order functions throughout this book — `array.map(fn)`, `array.filter(fn)`, `setTimeout(fn, ms)`, and the `makeCounter()` / `memoize()` closures from Chapter 14 are all higher-order functions.

```
function withLogging(fn) {
  // takes a function, returns a NEW function that wraps it – higher-order twice over
  return function (...args) {
    console.log(`Calling ${fn.name} with`, args);
    const result = fn(...args);
    console.log(`${fn.name} returned`, result);
    return result;
  };
}

function add(a, b) {
  return a + b;
}

const loggedAdd = withLogging(add);
loggedAdd(2, 3);
// Calling add with [ 2, 3 ]
// add returned 5
```

19.3 `map`, `filter`, `reduce` as a functional toolkit

Chapter 8 introduced these as array methods; here we look at them through a functional lens: each one replaces an imperative loop pattern with a declarative statement of *intent*.

```
const numbers = [1, 2, 3, 4, 5, 6, 7, 8];

// Imperative: describes HOW (a loop, a mutable accumulator)
const evensImperative = [];
for (let i = 0; i < numbers.length; i++) {
  if (numbers[i] % 2 === 0) evensImperative.push(numbers[i]);
}

// Declarative: describes WHAT (filter down to evens)
const evensDeclarative = numbers.filter((n) => n % 2 === 0);

console.log(evensImperative, evensDeclarative); // same result, different style

// A real pipeline: filter -> transform -> aggregate, each step named and testable
const sumOfSquaresOfEvens = numbers
  .filter((n) => n % 2 === 0)
  .map((n) => n * n)
  .reduce((total, n) => total + n, 0);

console.log(sumOfSquaresOfEvens); // 4 + 16 + 36 + 64 = 120
```

`reduce` is the most general of the three — `map` and `filter` can both be implemented *in terms of* `reduce`, which is a useful exercise for understanding what "higher-order" really buys you:

```
function myMap(array, fn) {
  return array.reduce((acc, item) => {
    acc.push(fn(item));
    return acc;
  }, []);
}
function myFilter(array, predicate) {
  return array.reduce((acc, item) => {
    if (predicate(item)) acc.push(item);
    return acc;
  }, []);
}
console.log(myMap([1, 2, 3], (n) => n * 10));      // [10, 20, 30]
console.log(myFilter([1, 2, 3, 4], (n) => n > 2)); // [3, 4]
```

19.4 Function composition

Composition means building a complex operation by chaining together small, single-purpose functions, where each function's output becomes the next function's input. This is the functional equivalent of Unix pipes.

```
const compose = (...fns) =>
  (initialValue) =>
    fns.reduceRight((value, fn) => fn(value), initialValue);

const pipe = (...fns) =>
  (initialValue) =>
    fns.reduce((value, fn) => fn(value), initialValue);

const trim = (s) => s.trim();
const toLowerCase = (s) => s.toLowerCase();
const removeSpaces = (s) => s.replace(/\s+/g, "-");

const slugify = pipe(trim, toLowerCase, removeSpaces);
console.log(slugify(" Hello World ")); // "hello-world"

// compose() applies right-to-left, pipe() applies left-to-right – same
// functions, opposite reading order. Most developers find pipe() more
// intuitive because it reads top-to-bottom like the steps happen.
const slugifyComposed = compose(removeSpaces, toLowerCase, trim);
console.log(slugifyComposed(" Hello World ")); // "hello-world" – same result
```

19.5 Currying and partial application

Currying transforms a function that takes multiple arguments into a sequence of functions that each take a single argument. **Partial application** is the related (but distinct) technique of fixing some arguments of a function ahead of time, producing a new function that only needs the rest.

```
// A normal multi-argument function
function add3(a, b, c) {
  return a + b + c;
}

// Manually curried version
function curriedAdd3(a) {
  return function (b) {
    return function (c) {
      return a + b + c;
    };
  };
}
console.log(curriedAdd3(1)(2)(3)); // 6

// The same thing, written with arrow functions (much more common in practice)
const curriedAdd3Arrow = (a) => (b) => (c) => a + b + c;
console.log(curriedAdd3Arrow(1)(2)(3)); // 6

// A generic curry() helper that can curry ANY function
function curry(fn) {
  return function curried(...args) {
    if (args.length >= fn.length) {
      return fn(...args); // fn.length = number of declared (non-rest) parameters
    }
    return (...moreArgs) => curried(...args, ...moreArgs);
  };
}

const curriedAdd = curry(add3);
console.log(curriedAdd(1)(2)(3)); // 6 – fully curried, one arg at a time
console.log(curriedAdd(1, 2)(3)); // 6 – partial application works too
console.log(curriedAdd(1, 2, 3)); // 6 – calling with all args at once still works
```

Currying is especially useful for creating specialized functions from general ones:

```
const multiply = (a, b) => a * b;
const curriedMultiply = curry(multiply);
const double = curriedMultiply(2); // "freeze" the first argument as 2
const triple = curriedMultiply(3);
console.log(double(5), triple(5)); // 10 15
```

19.6 Pure functions and immutability

A **pure function** has two properties:

1. Given the same inputs, it **always returns the same output**.
2. It has **no side effects** — it doesn't mutate its arguments, doesn't touch external/global state, doesn't perform I/O.

```
// Impure: depends on external state, and mutates its argument
let taxRate = 0.08;
function addTaxImpure(cart) {
  cart.total = cart.total * (1 + taxRate); // mutates the input object!
  return cart.total;
}

// Pure: same inputs always give same output, no mutation, no external reads
function addTaxPure(total, rate) {
  return total * (1 + rate);
}

const cart = { total: 100 };
console.log(addTaxPure(cart.total, 0.08)); // 108 – cart itself is untouched
console.log(cart.total);                  // still 100
```

Pure functions are trivially testable (no setup/mock of external state needed), safely cacheable (Chapter 14's `memoize`), and safe to run in any order or in parallel, since they can't interfere with anything outside themselves. Real programs can't be *entirely* pure — something eventually has to touch the DOM, write to a database, or log to a console — but pushing as much logic as possible into pure functions, and concentrating side effects into a small, clearly identified layer, is a widely used professional discipline (sometimes summarized as "functional core, imperative shell").

Immutability — not mutating data, but creating new copies with changes applied — is the natural companion practice:

```
// Mutating (avoid for shared/external state):
function addItemMutating(cart, item) {
  cart.items.push(item); // mutates the original array in place
  return cart;
}

// Immutable (safer, easier to reason about):
function addItemImmutable(cart, item) {
  return {
    ...cart,
    items: [...cart.items, item], // new array, original untouched
  };
}

const cartA = { items: ["apple"] };
const cartB = addItemImmutable(cartA, "banana");
console.log(cartA.items); // ["apple"] – unchanged
console.log(cartB.items); // ["apple", "banana"] – new object
```

19.7 An honest comparison: imperative vs. functional style in JS

JavaScript is a **multi-paradigm** language — it doesn't force you into either imperative or functional style, and idiomatic JS code usually mixes both pragmatically:

```
// Same task, three styles:
const users = [
  { name: "Ada", active: true },
  { name: "Grace", active: false },
  { name: "Alan", active: true },
];

// Imperative
const activeNamesImperative = [];
for (const user of users) {
  if (user.active) activeNamesImperative.push(user.name);
}

// Functional
const activeNamesFunctional = users
  .filter((u) => u.active)
  .map((u) => u.name);

console.log(activeNamesImperative); // ["Ada", "Alan"]
console.log(activeNamesFunctional); // ["Ada", "Alan"]
```

Neither style is universally "better." The functional/chained version reads clearly as a pipeline of transformations and avoids a mutable accumulator variable, which many developers find easier to reason about. The imperative version can be more efficient for very large datasets or complex control flow with early exits (`break`), since chained array methods create intermediate arrays at each step (`.filter()` allocates a whole new array before `.map()` even starts) — a tradeoff revisited in Chapter 34's performance discussion. Good JavaScript developers read both fluently and choose deliberately, rather than treating either style as a rule to follow blindly.

19.8 Chapter summary

- Functions are first-class values in JavaScript — they can be stored, passed, and returned like any other value.
- A higher-order function takes a function as an argument, returns a function, or both. `map` / `filter` / `reduce` , `setTimeout` , and closures from Chapter 14 are all higher-order functions in practice.
- `map` and `filter` can both be expressed in terms of `reduce` , the most general of the three.
- Composition (`pipe` / `compose`) chains small functions into pipelines; `pipe` applies left-to-right, `compose` applies right-to-left.
- Currying transforms a multi-argument function into a chain of single-argument functions; partial application fixes some arguments ahead of time.

- Pure functions (same input → same output, no side effects) are easier to test, cache, and reason about; immutability (copy-with-changes instead of mutate-in-place) is their natural companion practice.
- JavaScript is multi-paradigm — choose imperative or functional style deliberately based on clarity and performance needs, not dogma.

19.9 Exercises

1. Implement `myReduce(array, fn, initialValue)` from scratch using only a `for` loop (i.e., without calling the built-in `.reduce`).
2. Write a curried `between(min)(max)(value)` function that returns `true` if `value` is within `[min, max]` inclusive, then use `curry()` from this chapter on an uncurried three-argument version to prove it's equivalent.
3. Identify why the `addTaxImpure` function from 19.6 would give different test results depending on when a test runs, and rewrite it to be pure.

Chapter 20 — Iterators and Generators

20.1 What makes something "iterable"

In Chapter 6 you used `for...of` on arrays, strings, `Map`, and `Set` without asking *why* it worked on those but not on plain objects. The answer is the **iterable protocol**: any object that implements a method named `Symbol.iterator` (a well-known Symbol — see Chapter 3.6) which returns an **iterator** can be used with `for...of`, `spread (...)`, destructuring, and `Array.from()`.

An **iterator** is any object with a `.next()` method that returns `{ value, done }`:

```
const numbers = [10, 20, 30];
const iterator = numbers[Symbol.iterator](); // arrays have a built-in iterator

console.log(iterator.next()); // { value: 10, done: false }
console.log(iterator.next()); // { value: 20, done: false }
console.log(iterator.next()); // { value: 30, done: false }
console.log(iterator.next()); // { value: undefined, done: true }
```

`for...of` is essentially sugar for calling `.next()` repeatedly until `done` is `true`:

```
function forOfEquivalent(iterable) {
  const it = iterable[Symbol.iterator]();
  let result = it.next();
  while (!result.done) {
    console.log(result.value);
    result = it.next();
  }
}
forOfEquivalent([1, 2, 3]); // 1, 2, 3
```

Plain objects (`{}`) don't implement `Symbol.iterator` by default, which is exactly why `for...of` throws `TypeError: obj is not iterable` on a plain object, while `for...in` (Chapter 6.4, key-based, not the iterator protocol) works fine.

20.2 Building a custom iterable

You can make any object work with `for...of` by implementing `Symbol.iterator` yourself:

```
const range = {
  from: 1,
  to: 5,
  [Symbol.iterator]() {
    let current = this.from;
    const last = this.to;
    return {
      next() {
        if (current <= last) {
          return { value: current++, done: false };
        }
        return { value: undefined, done: true };
      },
    };
  },
};

for (const n of range) {
  console.log(n); // 1, 2, 3, 4, 5
}

console.log([...range]); // [1, 2, 3, 4, 5] -- spread also uses the iterable protocol
console.log(Array.from(range)); // same result, different syntax
```

This manual pattern — tracking state in closure variables, returning an object with `.next()` — works, but it's verbose and easy to get subtly wrong (as the next section shows). **Generators** exist to make exactly this pattern trivial to write correctly.

20.3 Generator functions

A **generator function** (declared with `function*`) is a function that can pause and resume its own execution using the `yield` keyword. Calling a generator function doesn't run its body immediately — it returns a **generator object**, which is both an iterator *and* an iterable:

```
function* simpleGenerator() {
  console.log("start");
  yield 1;
  console.log("resumed after first yield");
  yield 2;
  console.log("resumed after second yield");
  yield 3;
  console.log("generator finished");
}

const gen = simpleGenerator(); // nothing logs yet -- the body hasn't run
console.log(gen.next()); // logs "start", then returns { value: 1, done: false }
console.log(gen.next()); // logs "resumed after first yield", returns { value: 2, done: false }
console.log(gen.next()); // logs "resumed after second yield", returns { value: 3, done: false }
console.log(gen.next()); // logs "generator finished", returns { value: undefined, done: true }
```

Because generators implement `Symbol.iterator` automatically, they work directly with `for...of` and spread:

```
function* countTo(n) {
  for (let i = 1; i <= n; i++) {
    yield i;
  }
}
for (const n of countTo(5)) {
  console.log(n); // 1, 2, 3, 4, 5
}
console.log([...countTo(3)]); // [1, 2, 3]
```

The `range` example from 20.2, rewritten as a generator, is far shorter and impossible to get the `done / value` bookkeeping wrong on:

```
function* range(from, to) {
  for (let i = from; i <= to; i++) {
    yield i;
  }
}
console.log([...range(1, 5)]); // [1, 2, 3, 4, 5]
```

20.4 Passing values into a generator

`.next(value)` can send a value *into* the generator, which becomes the result of the `yield` expression that was paused:

```
function* conversation() {
  const name = yield "What's your name?";
  const age = yield `Hi ${name}, how old are you?`;
  return `${name} is ${age} years old.`;
}

const convo = conversation();
console.log(convo.next());           // { value: "What's your name?", done: false }
console.log(convo.next("Alice"));    // { value: "Hi Alice, how old are you?", done: false }
console.log(convo.next("30"));        // { value: "Alice is 30 years old.", done: true }
```

Note the offset: the **first** `.next()` call just starts the generator and runs it up to the first `yield` — any value passed to it is discarded because there's no paused `yield` expression yet to receive it.

20.5 `yield*` — delegating to another iterable

`yield*` delegates iteration to another generator or iterable, yielding each of its values in turn — useful for composing generators:

```
function* inner() {
  yield 1;
  yield 2;
}

function* outer() {
  yield "start";
  yield* inner(); // delegates: yields 1, then 2, as if written inline
  yield "end";
}

console.log([...outer()]); // ["start", 1, 2, "end"]

function* flatten(iterableOfIterables) {
  for (const item of iterableOfIterables) {
    yield* item; // works for any iterable, not just other generators
  }
}

console.log([...flatten([[1, 2], [3, 4], [5]])]); // [1, 2, 3, 4, 5]
```

20.6 Infinite sequences and lazy evaluation

Because a generator only computes the next value when `.next()` is called, generators can represent **infinite sequences** without ever running out of memory — something a regular function returning an array could never do:

```
function* naturalNumbers() {
  let n = 1;
  while (true) {
    yield n++;
  }
}

function take(iterable, count) {
  const result = [];
  const it = iterable[Symbol.iterator]();
  for (let i = 0; i < count; i++) {
    const { value, done } = it.next();
    if (done) break;
    result.push(value);
  }
  return result;
}

console.log(take(naturalNumbers(), 5)); // [1, 2, 3, 4, 5] -- computed lazily, 5 at a time
```

```
function* fibonacci() {
  let [a, b] = [0, 1];
  while (true) {
    yield a;
    [a, b] = [b, a + b];
  }
}
console.log(take(fibonacci(), 8)); // [0, 1, 1, 2, 3, 5, 8, 13]
```

This "lazy, pull-based" evaluation — compute only what's asked for, only when it's asked for — is a genuinely different way of structuring code compared to eagerly building arrays with `.map()` / `.filter()`, and it's the same underlying idea that powers Node.js streams and, conceptually, `async / await` (which we cover next, in Chapter 21-23, and which is itself built on a generator-like mechanism under the hood).

20.7 `return()` and `throw()` on generators

Generator objects also have `.return(value)` and `.throw(error)` methods that let external code end a generator early or inject an error into it:

```
function* withCleanup() {
  try {
    yield 1;
    yield 2;
    yield 3;
  } finally {
    console.log("cleanup ran"); // runs even if the generator is stopped early
  }
}

const g = withCleanup();
console.log(g.next());           // { value: 1, done: false }
console.log(g.return("done!")); // logs "cleanup ran", then { value: "done!", done: true }
console.log(g.next());           // { value: undefined, done: true } -- already finished
```

`for...of` automatically calls `.return()` on a generator if you `break` out of the loop early, so `finally` blocks for resource cleanup (closing a file handle, a database cursor) run reliably even on early exit.

20.8 A practical use case: paginated API iteration

Generators are a natural fit for hiding pagination behind a simple `for...of` loop — the caller doesn't need to know or care that data is arriving in pages:

```
function* paginate(items, pageSize) {
  for (let i = 0; i < items.length; i += pageSize) {
    yield items.slice(i, i + pageSize); // yields one PAGE (array) at a time
  }
}

const allItems = Array.from({ length: 10 }, (_, i) => i + 1);
for (const page of paginate(allItems, 3)) {
  console.log("page:", page); // [1,2,3], [4,5,6], [7,8,9], [10]
}
```

In real code, an async generator (`async function*`, briefly noted in Chapter 23) would `yield` pages fetched from a network API one request at a time, and consuming code would use `for await...of` — same idea, adapted for asynchronous data sources.

20.9 Chapter summary

- The **iterable protocol** (`Symbol.iterator` returning an object with `.next()`) is what makes `for...of`, spread, destructuring, and `Array.from()` work on arrays, strings, `Map`, `Set`, and any custom object that implements it.
- Generator functions (`function*`, `yield`) let you write an iterator's pause/resume logic as ordinary, linear-looking code instead of manually managing `{ value, done }` state.
- `.next(value)` sends a value into a paused `yield` expression; `yield*` delegates to another iterable, flattening its values into the current generator's output.
- Generators enable lazy, pull-based evaluation, including representing infinite sequences that only compute as many values as are actually consumed.
- `.return()` / `.throw()` (and `for...of`'s automatic `.return()` on `break`) let a generator's `finally` block run cleanup code reliably, even on early exit.

20.10 Exercises

1. Write a generator `evens(max)` that yields only even numbers from 0 up to `max`, then use it with `for...of` to print them.
2. Using the `take()` helper from 20.6, get the first 6 values of a `powersOfTwo()` generator you write yourself (1, 2, 4, 8, 16, 32).
3. Rewrite the custom `range` object from 20.2 as a generator function and confirm both versions produce identical output when spread into an array.

Chapter 21 — Callbacks and the Event Loop

21.1 Synchronous vs. asynchronous execution

Every JavaScript program starts by running **synchronous** code: statements execute one at a time, in order, and each statement must finish before the next one begins.

```
console.log("A");
console.log("B");
console.log("C");
// Output: A, B, C – always, every time, in that exact order.
```

Asynchronous code breaks that one-thing-at-a-time ordering on purpose. Some operation is *started* now but *finishes* later, and the rest of your program keeps running in the meantime instead of freezing to wait for it.

```
console.log("A");
setTimeout(() => console.log("B"), 0);
console.log("C");
// Output: A, C, B – even with a 0ms delay!
```

`B` prints last even though its delay is `0`. This single surprising result is the doorway into everything this chapter explains. `setTimeout` never runs its callback *immediately*, no matter how small the delay — it always defers it to "later," and "later" is scheduled by a mechanism called the **event loop**.

21.2 The call stack

JavaScript executes code using a **call stack** — a simple, single-threaded data structure that tracks "what function is currently running, and what called it." Every time a function is called, a new **stack frame** is pushed on top. When that function returns, its frame is popped off.

```
function multiply(a, b) {
  return a * b;
}

function square(n) {
  return multiply(n, n);
}

function printSquare(n) {
  console.log(square(n));
}

printSquare(5);
```

Trace the stack for this call:

```
1. printSquare(5) pushed      stack: [printSquare]
2. square(5) pushed          stack: [printSquare, square]
3. multiply(5, 5) pushed      stack: [printSquare, square, multiply]
4. multiply returns 25, popped stack: [printSquare, square]
5. square returns 25, popped  stack: [printSquare]
6. console.log(25) pushed/popped
7. printSquare returns, popped stack: []
```

Because there is only **one** call stack, JavaScript can only ever execute one piece of code at a time — this is what "single-threaded" means in practice. If a function on the stack takes a long time (a huge loop, a synchronous computation), *nothing else can happen* — not a click handler, not a timer callback, not a network response — until that frame finishes and is popped. This is called **blocking the main thread**, and it's the reason a slow synchronous function can freeze an entire web page.

```
function blockFor(ms) {
  const end = Date.now() + ms;
  while (Date.now() < end) {} // busy-wait – burns CPU, blocks everything
}

console.log("start");
blockFor(3000); // the whole tab is frozen for 3 seconds – no clicks,
               // no rendering, no timers fire – the stack is not empty
console.log("done");
```

21.3 Callback-based asynchronous APIs

A **callback** is simply a function passed as an argument to another function, to be invoked later. Asynchronous callbacks are how JavaScript historically handled "do this, and when it's done, run this other code":

```
console.log("1: scheduling work");

setTimeout(function onTimeout() {
  console.log("3: timeout fired");
}, 1000);

console.log("2: scheduling done, moving on");
```

`setTimeout` doesn't pause anything. It registers `onTimeout` with the runtime and returns immediately. The runtime is responsible for calling `onTimeout` back once (at least) 1000ms have passed *and* the call stack is empty.

The error-first callback convention (Node.js style)

Before promises existed, Node.js standardized a convention for callback-based APIs: the callback's **first argument is always reserved for an error** (or `null` if there wasn't one), and subsequent arguments carry the actual result.

```
const fs = require("fs");

fs.readFile("config.json", "utf8", function (err, data) {
  if (err) {
    console.error("Failed to read file:", err.message);
    return; // stop here – don't try to use `data`, it doesn't exist
  }
  console.log("File contents:", data);
});
```

Every built-in Node.js callback API follows this pattern: `fs.readFile`, `fs.writeFile`, `crypto.randomBytes`, database drivers, and most npm packages written before ~2017. Checking `err` first, and returning early on error, is a habit you should build immediately — forgetting it is one of the most common sources of silent bugs in callback-style code.

21.4 Callback hell (the pyramid of doom)

The trouble with callbacks appears when asynchronous operations depend on each other — step 2 can't start until step 1 finishes, step 3 needs step 2's result, and so on. Nesting callback inside callback inside callback produces code that grows sideways instead of downward:

```

getUser(userId, function (err, user) {
  if (err) return handleError(err);
  getOrders(user.id, function (err, orders) {
    if (err) return handleError(err);
    getOrderDetails(orders[0].id, function (err, details) {
      if (err) return handleError(err);
      calculateShipping(details, function (err, shipping) {
        if (err) return handleError(err);
        applyDiscount(shipping, user.membership, function (err, final) {
          if (err) return handleError(err);
          console.log("Final price:", final);
          // five levels deep, and still growing...
        });
      });
    });
  });
});

```

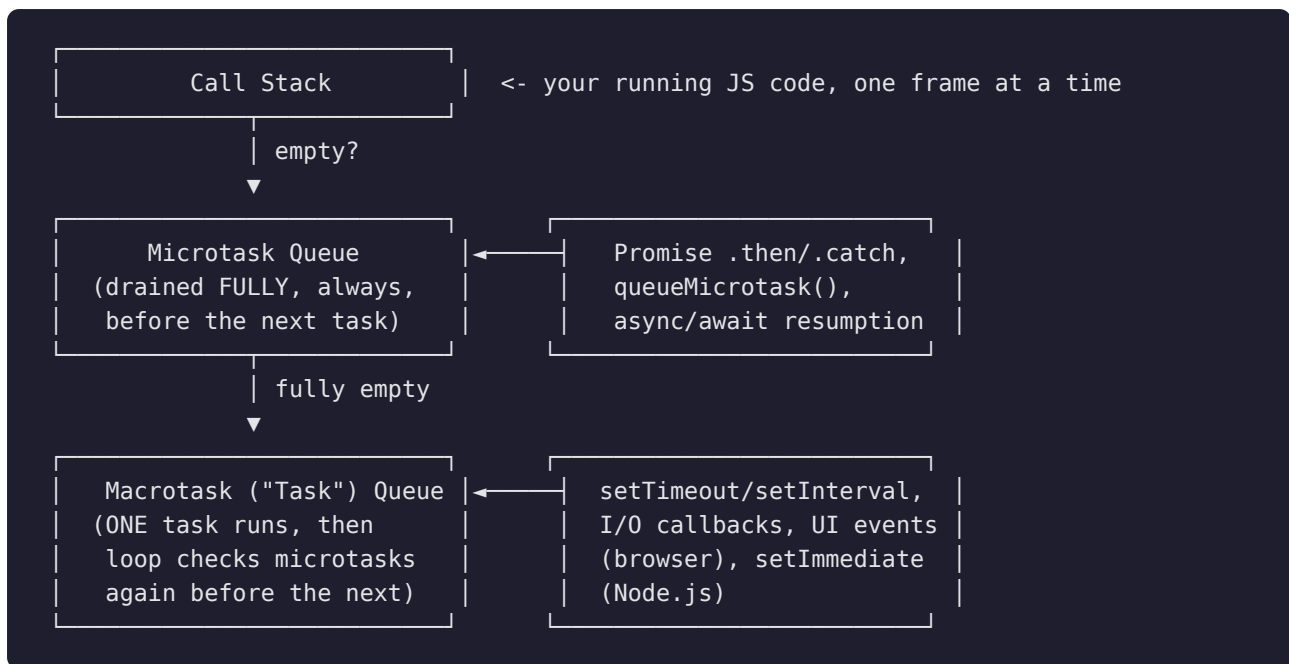
This shape is nicknamed "**callback hell**" or the "**pyramid of doom**". Beyond looking ugly, it has real, practical problems:

- **Error handling is repeated at every level** and easy to forget at any one of them, silently swallowing failures.
- **Control flow is hard to follow** — reading top-to-bottom no longer matches "what happens first."
- **Composing async operations is awkward** — running two independent requests in parallel and waiting for both requires manual counters.
- **Reusability suffers** — the logic is trapped inside deeply nested anonymous functions.

Callback hell is precisely the problem **Promises** (Chapter 22) and later **async/await** (Chapter 23) were designed to solve. It's worth feeling the pain of this pattern once, deliberately, so the motivation for promises isn't just "because everyone says so."

21.5 The event loop: the precise mechanics

This is the single most important diagram in asynchronous JavaScript. The JavaScript runtime is made of a few cooperating pieces:



The **event loop** is the runtime's forever-running coordinator. On every "tick" it does, roughly:

1. Run code on the call stack until it's empty (all synchronous code in the current script/task finishes).
2. Once the stack is empty, drain the **microtask queue completely** — run every microtask, and if a microtask schedules another microtask, run that one too, before moving on. The microtask queue must reach *zero* items before step 3.
3. Take exactly **one** task from the macrotask queue, run it (which involves pushing it onto the call stack and repeating step 1).
4. In browsers, potentially render a frame here if one is due.
5. Go back to step 2.

The critical rule to memorize: all pending microtasks always run before the next macrotask, no exceptions. Microtasks are:

- `Promise.prototype.then/catch/finally` callbacks
- `queueMicrotask(fn)`
- code after an `await` in an async function (it's sugar for `.then`)

Macrotasks ("tasks") are:

- `setTimeout / setInterval` callbacks
- I/O callbacks (`fs.readFile` , network callbacks)
- UI events (click, scroll) in browsers
- `setImmediate` (Node.js only)
- each `<script>` execution (browsers)

Worked example: predict the order

Before reading the explanation, try to predict the console output of this program on your own:

```
console.log("1: script start");

setTimeout(() => console.log("2: setTimeout callback"), 0);

Promise.resolve()
  .then(() => console.log("3: promise .then #1"))
  .then(() => console.log("4: promise .then #2"));

queueMicrotask(() => console.log("5: queueMicrotask"));

console.log("6: script end");
```

Walking through the event loop rules:

1. `"1: script start"` logs immediately — synchronous.
2. `setTimeout(...)` registers its callback as a **macrotask** and returns immediately — nothing logs yet.
3. `Promise.resolve().then(...)` registers `"3: promise .then #1"` as a **microtask** — nothing logs yet, and the second `.then` isn't even queued yet (it only gets queued once the first one resolves).
4. `queueMicrotask(...)` registers `"5: queueMicrotask"` as a microtask.
5. `"6: script end"` logs immediately — synchronous.
6. **Stack is now empty.** The event loop drains the microtask queue in FIFO order: `"3: promise .then #1"` runs, which queues `"4: promise .then #2"` as a *new* microtask — it still runs before any macrotask, because the microtask queue isn't empty yet. Then `"5: queueMicrotask"` runs. Then `"4: promise .then #2"` runs. Now the microtask queue is finally empty.
7. Only now does the event loop take the one macrotask waiting — `"2: setTimeout callback"` — and run it.

Final order: **1, 6, 3, 5, 4, 2.**

If this doesn't match your first guess, you're in good company — this is the #1 JavaScript interview question for a reason. The takeaway rule: **promises always jump the queue ahead of `setTimeout`, no matter the delay**, because promise callbacks are microtasks and `setTimeout` callbacks are macrotasks, and microtasks always fully drain first.

21.6 Why `setTimeout(fn, 0)` isn't "immediately"

`setTimeout(fn, 0)` does not mean "run `fn` right now." It means "run `fn` as a macrotask as soon as possible" — which is *after* the current synchronous code finishes **and** after every pending microtask has run. In practice this makes `setTimeout(fn, 0)` a useful trick to defer work until the browser or Node has had a chance to do other things first (like render a frame or process I/O), but it is never truly instant.

21.7 Callbacks that never let go: repeated events

Not every callback runs once and disappears. Event listeners and `setInterval` register callbacks that can fire many times:

```
let count = 0;
const intervalId = setInterval(() => {
  count++;
  console.log("tick", count);
  if (count === 3) {
    clearInterval(intervalId); // always store the id so you can cancel it
  }
}, 500);
```

Forgetting to clear an interval or remove an event listener you no longer need is a common source of memory leaks and "why is this running twice" bugs — a theme we'll revisit in Chapter 28 (Events).

21.8 Chapter summary

- Synchronous code runs top-to-bottom on the single call stack; nothing else can run while the stack is busy — this is "blocking."
- Asynchronous APIs (timers, I/O, network) hand work off to the runtime and register a callback to run later, without blocking the stack.
- Node.js callbacks conventionally use the **error-first** signature: `(err, result) => {}`.
- Deeply nested dependent callbacks produce **callback hell**, which hurts readability, error handling, and composability — the motivation for promises and `async/await`.
- The **event loop** repeatedly: runs the stack to empty, then fully drains the **microtask queue** (promises, `queueMicrotask`), then runs exactly one **macrotask** (`setTimeout`, I/O, events), then repeats.
- Microtasks always run before the next macrotask — this is why promise callbacks consistently "beat" `setTimeout(fn, 0)`.

21.9 Exercises

1. Before running it, predict the console output order of `examples/21-callbacks-event-loop.js`'s `orderingPuzzle()` function. Then run it and compare.

2. Rewrite the 5-level-deep callback pyramid in this chapter using named functions instead of anonymous inline functions. Does it become easier to read? What's still awkward about it?
3. Add a `console.log` inside a `setInterval` callback and inside a microtask created with `queueMicrotask` in the same tick as the interval fires. Which one prints first, every time? Why?

Chapter 22 — Promises

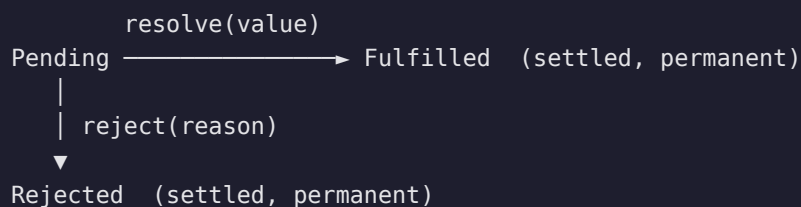
22.1 What is a Promise?

A **Promise** is a built-in JavaScript object that represents the *eventual* result of an asynchronous operation — a placeholder for a value that doesn't exist yet, but will (or will fail to) at some point in the future. Promises were standardized in ES2015 specifically to solve the callback hell problem from Chapter 21.

Every promise exists in exactly one of three states:

- **Pending** — the initial state; the operation hasn't finished yet.
- **Fulfilled** — the operation completed successfully; the promise now holds a resulting value.
- **Rejected** — the operation failed; the promise now holds a reason (usually an `Error`).

Once a promise leaves the pending state, it is **settled**, and it can never change state again — a fulfilled promise stays fulfilled with that same value forever; a rejected promise stays rejected with that same reason forever. This immutability-after-settling is a deliberate design choice that makes promises predictable and safe to hand to multiple consumers.



22.2 Creating a promise

You create a promise with the `Promise` constructor, which takes a single **executor function** with two parameters: `resolve` and `reject`. The executor runs *immediately and synchronously* when the promise is constructed.

```
const promise = new Promise((resolve, reject) => {
  const success = Math.random() > 0.5;
  setTimeout(() => {
    if (success) {
      resolve("Operation succeeded!");
    } else {
      reject(new Error("Operation failed!"));
    }
  }, 1000);
});
```

Calling `resolve(value)` transitions the promise to fulfilled with that value. Calling `reject(reason)` transitions it to rejected. Calling either one after the promise has already settled has no effect — the first call wins and all others are silently ignored.

In practice, **you rarely construct promises by hand**. Most of the time you receive promises from other APIs (`fetch`, `fs.promises.readFile`, database drivers). You mainly write `new Promise(...)` yourself when **wrapping** an older callback-based API — see 22.6.

22.3 Consuming a promise: `.then`, `.catch`, `.finally`

```
promise
  .then((result) => {
    console.log("Success:", result);
  })
  .catch((error) => {
    console.error("Error:", error.message);
  })
  .finally(() => {
    console.log("This always runs, success or failure");
  });
```

- `.then(onFulfilled, onRejected)` registers callbacks for the fulfilled and (optionally) rejected cases.
- `.catch(onRejected)` is exactly equivalent to `.then(undefined, onRejected)` — a shorthand for handling errors.
- `.finally(onSettled)` runs regardless of outcome, receives no argument, and is used for cleanup (hiding a loading spinner, closing a connection).

Crucially, `.then`, `.catch`, and `.finally` all **return a new promise**, which is what makes chaining possible.

22.4 Chaining promises correctly

Each `.then` callback's **return value** becomes the fulfillment value of the *next* promise in the chain. This is the single most important rule for using promises correctly.

```
fetchUser(1)
  .then((user) => {
    console.log("Got user:", user.name);
    return fetchOrders(user.id); // returning a PROMISE here...
  })
  .then((orders) => {
    // ...means this callback waits for it and receives its resolved value
    console.log("Got orders:", orders.length);
    return orders.length;
  })
  .then((count) => {
    console.log("Order count:", count);
  })
  .catch((err) => {
    // catches an error from ANY of the steps above – fetchUser,
    // fetchOrders, or any .then callback that throws
    console.error("Pipeline failed:", err.message);
  });
```

If a `.then` callback returns a **plain value**, the next `.then` receives that value directly. If it returns **another promise**, the chain automatically waits for that promise to settle and unwraps its result — this is what lets you flatten what would have been nested callbacks into a single readable vertical chain.

The classic mistake: forgetting to `return`

```
// BUG: forgot to `return` the inner promise
fetchUser(1).then((user) => {
  fetchOrders(user.id).then((orders) => {
    console.log(orders); // this still works...
  });
  // but nothing is returned here, so the OUTER chain's next .then
  // (if any) receives `undefined` immediately, without waiting
});
```

Always `return` inside a `.then` callback when you want the next link in the chain to wait for an inner asynchronous operation. Forgetting this `return` is one of the most common promise bugs in real codebases — operations silently run "in the background" instead of being awaited by the chain, and error handling downstream never sees their failures.

A single `.catch` handles every prior failure

An error thrown anywhere in a promise chain — inside the executor, inside any `.then` callback, or a promise that rejects — skips straight to the nearest `.catch` down the chain, exactly like a synchronous `try / catch` skips to the `catch` block. You do not need (and generally should not add) a separate

`.catch` after every single `.then`.

22.5 Combinators: running promises together

Real programs frequently need to coordinate *multiple* promises. ES2015 gave us `Promise.all` and `Promise.race`; ES2020/2021 added `Promise.allSettled` and `Promise.any`.

`Promise.all` — wait for all, fail fast on any rejection

```
Promise.all([fetchUser(1), fetchUser(2), fetchUser(3)])
  .then((users) => {
    console.log("All three users:", users); // array, same order as input
  })
  .catch((err) => {
    // fires as soon as ANY of the promises rejects – the others' results
    // (even if they later succeed) are discarded
    console.error("At least one failed:", err.message);
  });
```

Use `Promise.all` when you need *every* result and are fine treating any single failure as a total failure.

`Promise.allSettled` — wait for all, never short-circuits

```
Promise.allSettled([fetchUser(1), fetchUser(-1), fetchUser(3)]).then(
  (results) => {
    results.forEach((result, i) => {
      if (result.status === "fulfilled") {
        console.log(`User ${i}:`, result.value);
      } else {
        console.log(`User ${i} failed:`, result.reason.message);
      }
    });
  }
);
```

Use `Promise.allSettled` when partial failure is acceptable and you want to know the outcome of *every* operation, successful or not — e.g., sending notifications to five users where one bad email shouldn't cancel the other four.

`Promise.race` — settles as soon as the first one settles

```
Promise.race([
  fetchWithTimeout("https://api.example.com/data", 3000),
  new Promise(_, reject) =>
    setTimeout(() => reject(new Error("Global timeout")), 5000)
),
]).then(console.log).catch(console.error);
```

`Promise.race` settles (fulfilled or rejected) as soon as the fastest input promise settles. A very common use is implementing a timeout around a slow operation, as shown above.

`Promise.any` — first success wins, ignores rejections

```
Promise.any([fetchFromMirror1(), fetchFromMirror2(), fetchFromMirror3()])
  .then((firstSuccess) => console.log("Fastest successful mirror:", firstSuccess))
  .catch((aggregateError) => {
    // only rejects if ALL of them reject
    console.error("Every mirror failed:", aggregateError.errors);
  });
```

`Promise.any` is the fulfillment-focused sibling of `Promise.race` — it ignores rejections and only settles (rejects) if every input promise rejects, bundling all the individual errors into an `AggregateError`.

22.6 Promisifying callback-based APIs

Older Node.js APIs and many libraries still use error-first callbacks. "Promisifying" wraps such a function so it returns a promise instead, letting it participate in modern promise chains and, eventually, `async / await`.

```
function readFileCallback(path, callback) {
  // pretend this is fs.readFile
  setTimeout(() => {
    if (path === "missing.txt") {
      callback(new Error("ENOENT: no such file"));
    } else {
      callback(null, "file contents for " + path);
    }
  }, 10);
}

function readFilePromise(path) {
  return new Promise((resolve, reject) => {
    readFileCallback(path, (err, data) => {
      if (err) {
        reject(err);
      } else {
        resolve(data);
      }
    });
  });
}

readFilePromise("notes.txt")
  .then((data) => console.log("Read:", data))
  .catch((err) => console.error("Failed:", err.message));
```

Node.js also ships a built-in helper for exactly this pattern: `util.promisify(fn)`, which works automatically for any function that follows the standard `(err, result) => {}` callback signature.

```
const util = require("util");
const promisifiedReadFile = util.promisify(readFileCallback);
promisifiedReadFile("notes.txt").then(console.log);
```

22.7 The Promise constructor antipattern

A frequent mistake, especially among developers new to promises, is wrapping something that is *already a promise* in a new, unnecessary `Promise` constructor:

```
// ANTIPATTERN – don't do this
function getUser(id) {
  return new Promise((resolve, reject) => {
    fetchUser(id) // fetchUser already returns a promise!
      .then((user) => resolve(user))
      .catch((err) => reject(err));
  });
}

// CORRECT – fetchUser already returns a promise, just return it
function getUser(id) {
  return fetchUser(id);
}
```

If you already have a promise, return it directly (or chain `.then` onto it and return that). Only reach for `new Promise(...)` when you're wrapping something that is **not** already promise-based — a raw callback API, a `setTimeout`, or an event emitter.

22.8 Unhandled promise rejections

If a promise rejects and nothing ever attaches a `.catch` (or the rejected branch of a `.then`) to it, JavaScript considers this an **unhandled promise rejection**. In Node.js, this prints a warning to `stderr` and, in recent versions, can crash the process entirely; in browsers, it logs to the console and fires an `unhandledrejection` event on `window`.

```
function riskyOperation() {
  return new Promise(_, reject) => {
    setTimeout(() => reject(new Error("Something broke")), 10);
  });
}

riskyOperation(); // BUG: no .catch anywhere – this rejection goes unhandled
```

The fix is simple but easy to forget in a large codebase: **every promise chain needs a `.catch` somewhere**, or must eventually be `await`-ed inside a `try / catch` (Chapter 23). As a safety net, you can also listen globally:

```
process.on("unhandledRejection", (reason, promise) => {
  console.error("Unhandled rejection:", reason);
});
```

Treat this global listener as a last-resort logging net, not a substitute for handling errors where they occur.

22.9 Chapter summary

- A promise represents a future value with three states: pending, fulfilled, or rejected; once settled, it never changes again.
- `.then / .catch / .finally` consume a promise and each return a *new* promise, enabling chaining.
- Always `return` a value or promise from inside a `.then` callback if the next step in the chain should wait for it.
- `Promise.all` (fail-fast, all results), `Promise.allSettled` (never fails, all outcomes), `Promise.race` (first to settle), and `Promise.any` (first success) each coordinate multiple promises differently — pick based on what "done" means for your use case.
- `util.promisify` (or a hand-written wrapper) converts error-first callback APIs into promise-returning ones.
- Avoid the `new Promise` antipattern when you already have a promise to return.
- Every promise chain needs error handling, or you risk an unhandled rejection.

22.10 Exercises

1. Run `examples/22-promises.js` and read the output for the `Promise.all` vs `Promise.allSettled` section. Change one of the mock fetches to reject and observe how each combinator's output changes.
2. Find the "forgot to return" bug that's deliberately included in `demonstrateReturnBug()` in the example file, fix it, and confirm the corrected chain now logs the expected order-of-completion message.
3. Write a `withTimeout(promise, ms)` helper using `Promise.race` that rejects with a "Timed out" error if `promise` doesn't settle within `ms` milliseconds. Test it against a mock fetch that takes longer than the timeout.

Chapter 23 — Async/Await

23.1 Async/await is syntax sugar over promises

`async / await` (ES2017) is not a new concurrency model — it's syntax that lets you write promise-based code that *looks* synchronous, while the engine still runs it asynchronously underneath, using the exact same promises and microtask machinery from Chapter 22. Every `async` function you write compiles down, conceptually, to a promise chain.

```
// Promise chain style
function loadUser(id) {
  return fetchUser(id)
    .then((user) => {
      console.log("Got user:", user.name);
      return fetchOrders(user.id);
    })
    .then((orders) => {
      console.log("Got orders:", orders.length);
    });
}

// Equivalent async/await style
async function loadUser(id) {
  const user = await fetchUser(id);
  console.log("Got user:", user.name);
  const orders = await fetchOrders(user.id);
  console.log("Got orders:", orders.length);
}
```

Both versions behave identically at runtime — same event loop, same microtask scheduling, same eventual output order. `async / await` just removes the visual nesting and the `.then` boilerplate, making asynchronous logic read top-to-bottom like ordinary code.

23.2 The `async` keyword: functions that always return a promise

Placing `async` before a function declaration changes two things:

1. You may use `await` inside the function body.
2. The function **always returns a promise**, no matter what you `return` inside it.

```

async function getAnswer() {
  return 42;
}

getAnswer().then((value) => console.log(value)); // 42
console.log(getAnswer()); // Promise { 42 } – NOT the number 42 directly

```

If the function body explicitly `return`s a value, that value becomes the resolved value of the returned promise. If the function body `throw`s, the returned promise rejects with that thrown value. If the function returns another promise, the outer promise adopts its state (fulfills or rejects to match).

`async` works on function declarations, function expressions, arrow functions, object methods, and class methods:

```

const asyncArrow = async () => "arrow result";
const obj = { async method() { return "method result"; } };
class Service { async fetchData() { return "class method result"; } }

```

23.3 The `await` keyword: pausing for a promise

`await` can only be used inside an `async` function (with one exception — top-level `await`, see 23.7). It takes a promise, **pauses the async function's execution** at that point (without blocking the rest of the program — other code keeps running), and resumes the function once the promise settles.

- If the promise **fulfills**, `await` evaluates to the resolved value.
- If the promise **rejects**, `await` **throws** that rejection reason as a regular JavaScript exception, at the point of the `await`.

```

async function loadProfile() {
  const user = await fetchUser(1); // pauses here until fetchUser resolves
  console.log("User loaded:", user.name); // resumes here, with the value
}

```

Crucially, "pauses" only pauses that specific `async` function's own execution — it does not block the call stack or the rest of your program. Other synchronous code, other timers, and other `async` functions continue running normally while one `async` function is waiting at an `await`.

23.4 Error handling with `try/catch`

Because a rejected `await` throws, you handle `async` errors with ordinary `try / catch` — no more `.catch()` chains required:

```

async function loadProfileSafely() {
  try {
    const user = await fetchUser(-1); // this will reject
    console.log("User:", user.name);
  } catch (err) {
    console.error("Failed to load profile:", err.message);
  } finally {
    console.log("Done attempting to load profile");
  }
}

```

A single `try` block can wrap multiple `await` calls, and one `catch` will handle a failure from any of them — this mirrors how a single `.catch` at the end of a promise chain catches failures from any earlier `.then`.

```

async function checkout(cartId) {
  try {
    const cart = await fetchCart(cartId);
    const priced = await priceCart(cart);
    const receipt = await charge(priced);
    return receipt;
  } catch (err) {
    // catches a failure from fetchCart, priceCart, OR charge
    console.error("Checkout failed:", err.message);
    throw err; // re-throw if the caller also needs to know
  }
}

```

23.5 Sequential vs. parallel awaits — the classic mistake

`await` pauses *that line*, which means multiple `await`s written one after another run **sequentially**, even when the operations don't depend on each other at all:

```

// SLOW: sequential – each fetch waits for the previous one to finish,
// even though they're completely independent of each other
async function loadDashboardSlow() {
  const user = await fetchUser(1); // waits ~200ms
  const orders = await fetchOrders(1); // THEN waits another ~150ms
  const invoices = await fetchInvoices(1); // THEN waits another ~180ms
  return { user, orders, invoices }; // total: ~530ms
}

```

If the three fetches don't depend on each other, start them all at once and await the combined result with `Promise.all`:

```
// FAST: parallel – all three requests start at the same time
async function loadDashboardFast() {
  const [user, orders, invoices] = await Promise.all([
    fetchUser(1),
    fetchOrders(1),
    fetchInvoices(1),
  ]);
  return { user, orders, invoices }; // total: ~200ms (the slowest one)
}
```

The rule of thumb: **only `await` sequentially when each step genuinely needs the previous step's result.** Otherwise, start the independent promises first (without `await`), then await them together.

The loop mistake: `await` inside `for`, one item at a time

```
// SLOW: processes ids strictly one at a time
async function fetchAllSlow(ids) {
  const results = [];
  for (const id of ids) {
    const user = await fetchUser(id); // blocks the loop until this finishes
    results.push(user);
  }
  return results;
}

// FAST: fires all requests immediately, waits for all together
async function fetchAllFast(ids) {
  const promises = ids.map((id) => fetchUser(id)); // no await here yet –
  return Promise.all(promises);                  // starts them all at once
}
```

`ids.map((id) => fetchUser(id))` calls `fetchUser` for every id **synchronously and immediately** (each call starts its own async work in the background and returns a promise instantly), building an array of in-flight promises before a single `await` happens. `Promise.all` then waits for all of them concurrently. This is one of the most impactful performance fixes you can make in real-world async code — sequential awaits in a loop are a frequent, easy-to-miss source of slow APIs.

Note: if the operations must run one-at-a-time on purpose (e.g., to avoid overwhelming a rate-limited API, or because each step depends on shared mutable state), the sequential `for` loop with `await` is the *correct* choice — the goal isn't "always parallelize," it's "parallelize independent work."

23.6 Async functions always return promises — chaining and mixing

Since an `async` function returns a promise, you can call it and continue with `.then`, or `await` it from inside another `async` function:

```

async function getTotal() {
  return 100;
}

// Both are valid:
getTotal().then((total) => console.log("via .then:", total));

async function useIt() {
  const total = await getTotal(); // via await
  console.log("via await:", total);
}

```

You'll sometimes see `.then` and `await` mixed together in the same codebase, but doing so **within a single function** tends to hurt readability — pick one style per function and stay consistent. A common, justified exception is fire-and-forget: calling an `async` function and attaching only a `.catch` when you deliberately don't want to wait for it:

```

function logAnalyticsEvent(name) {
  sendAnalytics(name).catch((err) => {
    console.error("Analytics failed (non-critical):", err.message);
  });
  // deliberately NOT awaited — don't make the user wait on analytics
}

```

23.7 Top-level await

Historically, `await` was only legal inside an `async` function. ES2022 introduced **top-level await**, which allows `await` directly in the top level of an ES module (`.mjs` files, or `.js` files in a package with `"type": "module"` in `package.json`) — no wrapping `async` function needed.

```

// top-level in an ES module
const config = await loadConfig();
console.log("Config ready:", config);

```

Top-level await is extremely convenient for scripts and module initialization (e.g., a module that needs to fetch configuration before exporting anything), but it's only available in modules, not in regular CommonJS scripts (`require` -based `.js` files run with plain `node`) — see Chapter 24 for the module systems this depends on.

23.8 A realistic end-to-end example

Below is the kind of pattern you'll write constantly in real applications: fetch from a couple of independent "endpoints" in parallel, transform the combined data, and handle failure gracefully without crashing the whole operation. (See the example file for the full runnable version, using `setTimeout` - based mock endpoints so it's deterministic and needs no network access.)

```
async function loadDashboard(userId) {
  try {
    // independent requests -> run in parallel
    const [profile, notifications] = await Promise.all([
      fetchProfile(userId),
      fetchNotifications(userId),
    ]);

    // this step genuinely depends on `profile` -> must be sequential
    const recommendations = await fetchRecommendations(profile.interests);

    return {
      profile,
      unreadCount: notifications.filter((n) => !n.read).length,
      recommendations,
    };
  } catch (err) {
    console.error("Dashboard failed to load:", err.message);
    // return a degraded-but-usable result instead of crashing the caller
    return { profile: null, unreadCount: 0, recommendations: [] };
  }
}
```

This function demonstrates the full toolkit from this chapter working together: `async / await` for readable control flow, `Promise.all` for the genuinely independent calls, a sequential `await` for the genuinely dependent call, and `try / catch` for turning a failure into a safe fallback instead of an unhandled rejection.

23.9 Chapter summary

- `async / await` is syntax sugar over promises — same event loop, same microtask semantics, more readable syntax.
- An `async` function always returns a promise; `return value` resolves it, `throw` rejects it.
- `await` pauses only the current async function (not the whole program) until its promise settles, and throws if the promise rejects.
- Use `try / catch / finally` around `await` for error handling.
- Independent async operations should be started together and awaited with `Promise.all`, not awaited one-by-one — sequential awaits (especially inside loops) are a common, costly performance mistake.
- Top-level `await` is legal in ES modules without wrapping in an `async` function.

23.10 Exercises

1. Run `examples/23-async-await.js` and note the timing difference printed between `loadDashboardSlow` and `loadDashboardFast`. Explain in your own words why the fast version isn't just "a bit" faster but roughly proportional to the *slowest single request* instead of the *sum* of all requests.
2. Take the `fetchAllSlow` loop example and modify it to process items in batches of 2 at a time (start 2, wait for both, start the next 2) — useful when full parallelism would overwhelm a rate-limited API. Hint: combine `for` with `Promise.all` over small slices of the array.
3. In `loadDashboard`, deliberately make `fetchNotifications` reject and confirm the `catch` block produces the degraded fallback instead of crashing the program.

Chapter 24 — Modules

24.1 The problem modules solve

Every non-trivial program needs to be split across multiple files. Without a module system, JavaScript's only tool for sharing code between files was the **global scope**: you loaded several `<script>` tags in a row, and every variable declared at the top level of any of them became a property of `window`, visible to every other script on the page.

```
<script src="utils.js"></script>
<script src="cart.js"></script>
<script src="app.js"></script>
```

This works for tiny pages, but it breaks down fast in real applications:

- **Naming collisions.** If `utils.js` and `cart.js` both declare a top-level `function total() {}`, the second one silently overwrites the first. There is no error — just a bug that surfaces later.
- **No explicit dependencies.** Nothing in `app.js` states "I need `cart.js`'s `Cart` class." You only find out by reading the `<script>` tag order in the HTML, which is easy to get wrong as the project grows.
- **No privacy.** Every variable is global by default. A helper function meant to be an internal implementation detail of `utils.js` is just as exposed as the public API you intended other files to use.
- **Load-order fragility.** If someone reorders the `<script>` tags, or a build tool concatenates files in a different order, things break in ways that are hard to trace.

24.2 The pre-module workaround: the IIFE pattern

Before ES2015 gave JavaScript real modules, the community's best answer to "global scope pollution" was the **Immediately Invoked Function Expression (IIFE)**, often called the "module pattern." A function creates its own private scope, and you deliberately export only what you want public by attaching it to a single global object:

```
// utils.js – old-school module pattern
var MyApp = MyApp || {};

MyApp.utils = (function () {
  // everything in here is private – not visible outside this IIFE
  function formatCurrency(cents) {
    return "$" + (cents / 100).toFixed(2);
  }

  function slugify(text) {
    return text.toLowerCase().trim().replace(/\s+/g, "-");
  }

  // explicitly expose only what should be public
  return { formatCurrency, slugify };
})();

// another file, loaded after utils.js
console.log(MyApp.utils.formatCurrency(1999)); // "$19.99"
```

This pattern is worth recognizing because you'll still encounter it in older codebases, and because it demonstrates the core idea every later module system formalizes: **wrap code in its own scope, and export an explicit public interface**. What changed over time is *how* that wrapping and exporting happens, and whether the loading of dependencies is handled for you.

24.3 CommonJS (Node.js's original module system)

When Node.js was created in 2009, ES modules didn't exist yet in the language. Node adopted **CommonJS (CJS)**, a synchronous, file-based module system that is still the default in plain `.js` files in Node today (unless your project opts into ES modules — see §24.7).

Exporting with CommonJS

Every CommonJS file has an implicit `module.exports` object (initially `{}`) that represents what the file makes available to other files.

```
// math.js
function add(a, b) {
  return a + b;
}

function multiply(a, b) {
  return a * b;
}

module.exports = { add, multiply };
// or, exporting one thing at a time:
// exports.add = add;
// exports.multiply = multiply;
```

`exports` is just a shorthand reference to `module.exports` — **do not** reassign `exports = {...}` directly (that breaks the reference); always reassign `module.exports` if you want to replace the whole object.

Importing with `require`

```
// main.js
const { add, multiply } = require("./math.js");
console.log(add(2, 3)); // 5
console.log(multiply(2, 3)); // 6
```

`require` is a regular, **synchronous** function call. Node reads the file, executes it top to bottom immediately, caches the resulting `module.exports` object, and returns it. Because it's synchronous, you can call `require` anywhere in your code — inside an `if`, inside a function, conditionally — something ES modules deliberately restrict (see below).

How Node resolves module paths

- `require("./math.js")` or `require("./math")` — relative paths must start with `./` or `../`. Node will also try appending `.js`, `.json`, and `.node`, and will look for an `index.js` if you `require` a directory.
- `require("lodash")` — a **bare specifier** (no `./`) is resolved by walking up the directory tree looking for `node_modules/lodash`.
- `require("fs")` — matches a built-in Node module by name; built-ins always win over anything in `node_modules` with the same name.

The module cache

Node caches modules by their resolved absolute file path. The second time anything `require`s the same file, Node returns the *same* cached exports object instead of re-running the file — this is why modules are a great place to keep singleton state (a shared database connection, a configuration object),

and also why mutating an exported object from one place affects everyone else who imported it.

24.4 ES Modules (the language-native system)

ES2015 introduced `import / export` as part of the JavaScript language itself, not a Node-specific convention. ES Modules (ESM) work in both browsers (`<script type="module">`) and Node.js (with the right file extension or `package.json` setting — §24.7).

Named exports

```
// math.mjs
export function add(a, b) {
  return a + b;
}

export function multiply(a, b) {
  return a * b;
}

export const PI = 3.14159;
```

```
// main.mjs
import { add, multiply, PI } from "./math.mjs";
console.log(add(2, 3), multiply(2, 3), PI);
```

You can also export a whole block at once, and rename on either side:

```
function subtract(a, b) { return a - b; }
function divide(a, b) { return a / b; }
export { subtract, divide as safeDivide };

// elsewhere
import { subtract, safeDivide as divide } from "./math.mjs";
```

Default exports

Each module may additionally have **one** default export — used for "the main thing this file provides":

```
// Logger.mjs
export default class Logger {
  log(message) {
    console.log(`[LOG] ${message}`);
  }
}
```

```
// main.mjs
import Logger from "./Logger.mjs"; // no curly braces, any name you want
const logger = new Logger();
logger.log("hello");
```

You can mix default and named exports in one file, and import both:

```
import Logger, { LOG_LEVELS } from "./Logger.mjs";
```

import.meta

Inside an ES module, `import.meta` gives you metadata about the current module — most commonly `import.meta.url`, the file's own URL (useful for resolving paths relative to the current file, replacing the CommonJS-only `__dirname` / `__filename`, which do not exist in ESM).

```
console.log(import.meta.url);
// file:///home/you/project/main.mjs (Node)
// https://example.com/js/main.mjs (browser)
```

24.5 CommonJS vs. ES Modules: the real differences

These two systems look similar on the surface (`require` / `module.exports` vs `import` / `export`) but differ in ways that matter in practice:

	CommonJS	ES Modules
Loading	Synchronous	Asynchronous (even locally)
When resolved	At <code>require()</code> call time (runtime)	Statically, before code runs
Where allowed	Anywhere (inside <code>if</code> , functions, loops)	Only at the top level of a module
<code>this</code> at top level	<code>module.exports</code>	<code>undefined</code>
Strict mode	Opt-in	Always on
Circular imports	Returns partially-completed <code>exports</code>	Live bindings that update as the cycle resolves
File extension (Node)	<code>.js</code> (default) or <code>.cjs</code>	<code>.mjs</code> , or <code>.js</code> with <code>"type": "module"</code>

The **static** nature of ES module imports is a deliberate design choice: because `import` statements must appear at the top level with a literal string path (`import x from "./y.js"`, never `import x from someVariable`), tools can analyze your dependency graph *without running your code*. This is what enables **tree-shaking** — bundlers like esbuild, Rollup, and webpack can detect that you only used `add` from `math.mjs` and strip out `multiply` from the final bundle entirely, something that's much harder to do safely with CommonJS's dynamic, runtime `require()` calls.

Interop gotchas

Mixing the two systems is one of the most common sources of confusion in real projects:

- A CommonJS module can be loaded from an ES module using `import`, but Node has to guess how to expose it — typically the entire `module.exports` object becomes the **default** export, and named imports only work if Node's interop layer can statically detect named properties (it usually can for simple objects, but not always).
- An ES module **cannot** be loaded with `require()` at all in standard Node.js — attempting `require("./esm-file.mjs")` throws `ERR_REQUIRE_ESM`. You must use dynamic `import()` instead (§24.6).
- Many npm packages ship as "dual packages" (both CJS and ESM builds) to paper over this — that's what the `exports` field with `require / import` conditions in a package's `package.json` is for.

24.6 Dynamic `import()`

Both CommonJS and ESM code can use the **dynamic import expression**, `import("./module.js")`, which returns a **Promise** that resolves to the module's namespace object. Unlike static `import`, it can appear anywhere — inside a function, inside a conditional — which makes it useful for:

- **Lazy loading** — only download/parse a module when it's actually needed (e.g., a heavy chart library, loaded only when the user opens the "Reports" tab).
- **Conditional loading** — pick a module based on runtime conditions.
- **Loading ESM from CommonJS.**

```
async function showReportsTab() {
  const { renderChart } = await import("./chart-library.js");
  renderChart(data);
}

button.addEventListener("click", showReportsTab);
```

24.7 Telling Node.js which system you mean

Because a plain `.js` file is ambiguous, Node uses these rules, in order:

1. A file ending in `.mjs` is always treated as an ES module.
2. A file ending in `.cjs` is always treated as CommonJS.
3. A file ending in plain `.js` looks at the nearest `package.json` going up the directory tree: if it has `"type": "module"`, the file is ESM; otherwise (or if there's no `package.json`) it defaults to CJS.

```
{
  "name": "my-app",
  "type": "module"
}
```

Setting `"type": "module"` changes the default for **every** `.js` file in that project, which is why many teams instead just use explicit `.mjs` / `.cjs` extensions for clarity, especially in mixed codebases.

24.8 Modules in the browser

Adding `type="module"` to a `<script>` tag tells the browser to treat that file (and everything it imports) as an ES module:

```
<script type="module" src="main.js"></script>
```

This has several implications that differ from regular scripts:

- **Deferred by default** — module scripts don't block HTML parsing and always run after the document is parsed, similar to the `defer` attribute on regular scripts.
- **Strict mode automatically** — no need for `"use strict"`.
- **Their own scope** — top-level variables in a module script do **not** leak onto `window`, unlike classic scripts.
- **CORS rules apply** — a module fetched via `import` from another origin needs proper CORS headers, unlike a classic `<script src>`.
- **Only loaded once** — the browser (like Node) caches modules by URL, so importing the same module from multiple files only fetches/executes it once.

24.9 Practical guidance

- For new Node.js projects, prefer ES modules (`"type": "module"`) — it's where the language and ecosystem are heading, and it matches browser syntax, reducing context-switching.
- For existing large CommonJS codebases, there's rarely a good reason to do a risky big-bang migration — dual-package support and dynamic `import()` let you interoperate.
- Keep each module focused on one responsibility, and be deliberate about what you export — an explicit, small public API per file is the entire point of having modules in the first place.

24.10 Chapter summary

- Before real modules, JavaScript relied on the global scope and IIFE patterns to fake privacy and namespacing.

- **CommonJS** (`require / module.exports`) is Node's original, synchronous module system, resolved at runtime, loadable from anywhere in your code.
- **ES Modules** (`import / export`) are the language-native, statically analyzed system that enables tree-shaking, run in both browsers and Node, and are always strict-mode.
- The two systems differ in timing (sync vs. async), where imports are allowed (anywhere vs. top-level only), and how `this` behaves at the top level.
- Dynamic `import()` returns a promise and works in both systems — useful for lazy loading and for loading ESM from CommonJS.
- Node decides CJS vs. ESM per file using `.mjs / .cjs` extensions or the `"type"` field in the nearest `package.json`.

24.11 Exercises

1. Run both example subfolders (`examples/24-modules/cjs` and `examples/24-modules/esm`) with `node` and confirm they produce the same output despite using different module systems.
2. Try changing `module.exports = { add, multiply }` in the CJS example to `exports = { add, multiply }` (reassigning `exports` instead of `module.exports`). Predict what happens in `main.js` , then run it to check.
3. In the ESM example, add a `console.log(import.meta.url)` to `math.mjs` and explain what it prints and why it differs from Node's old `__filename` .

Chapter 25 — JSON and Web Storage

25.1 What JSON actually is

JSON (JavaScript Object Notation) is a text-based data format inspired by JavaScript object literal syntax, but it is important to understand that JSON is a *language-independent* data format, not JavaScript code. Every major programming language has a JSON parser, which is exactly why JSON became the universal data-interchange format for web APIs.

Because JSON is not literally JavaScript, its syntax rules are **stricter** than a JS object literal:

Rule	JS object literal	JSON
Keys	Can be unquoted (<code>{a: 1}</code>)	Must be double-quoted strings (<code>{"a": 1}</code>)
Strings	Single or double quotes	Double quotes only
Trailing commas	Allowed in modern JS	Not allowed
Comments	Allowed	Not allowed
Functions/ <code>undefined</code>	Allowed as values	Not allowed
Keys must be	Identifiers or strings	Always strings

```
{
  "name": "Ada Lovelace",
  "born": 1815,
  "isProgrammer": true,
  "languages": ["Analytical Engine notes"],
  "spouse": null
}
```

This is **valid JSON**. Each of the following is **invalid JSON** and will fail to parse, even though each looks like perfectly normal JavaScript:

```
{ name: "Ada" }           // ❌ unquoted key
{ "name": 'Ada' }         // ❌ single-quoted string
{ "name": "Ada", }        // ❌ trailing comma
{ "greet": function () {} } // ❌ functions aren't JSON values
```

25.2 `JSON.stringify` — turning JS values into JSON text

```
const user = { name: "Ada", born: 1815, active: true };
JSON.stringify(user);
// '{"name":"Ada","born":1815,"active":true}'
```

`JSON.stringify` accepts two additional arguments beyond the value: `JSON.stringify(value, replacer, space)`.

The `space` argument — pretty-printing

```
JSON.stringify(user, null, 2);
// {
//   "name": "Ada",
//   "born": 1815,
//   "active": true
// }
```

`space` can be a number (spaces of indentation, up to 10) or a string (e.g. `"\t"`), and is purely cosmetic — extremely useful for logging, config files, or anything a human will read.

The `replacer` argument — filtering or transforming

`replacer` can be an **array of keys to keep**:

```
JSON.stringify(user, ["name", "active"]);
// '{"name":"Ada","active":true}'
```

...or a **function** called for every key/value pair (including the root object itself, under the key `""`), letting you transform or omit values:

```
JSON.stringify(user, (key, value) => {
  if (key === "born") return undefined; // omit this key entirely
  return value;
});
// '{"name":"Ada","active":true}'
```

What gets skipped or converted automatically

- `undefined`, functions, and Symbols are **omitted** if they're object property values, and become `null` if they're array elements.
- `NaN` and `Infinity` are converted to `null`.
- `Date` objects are converted to ISO 8601 strings automatically (because `Date` has a built-in `toJSON` method — see below).

```
JSON.stringify({ a: undefined, b: () => {}, c: NaN, d: [undefined, 1] });  
// '{"c":null,"d":[null,1]}' -> a and b vanish, undefined in an array becomes null
```

Custom serialization with `toJSON`

If a value has a `.toJSON()` method, `JSON.stringify` calls it and serializes *its* return value instead of the object itself. This is how `Date` produces ISO strings, and you can use the same hook for your own classes:

```
class Money {  
  constructor(cents) {  
    this.cents = cents;  
  }  
  toJSON() {  
    return `$$${(this.cents / 100).toFixed(2)}`;  
  }  
}  
  
JSON.stringify({ price: new Money(1999) });  
// '{"price": "$19.99}"'
```

Circular references throw

`JSON.stringify` walks the object graph recursively, so an object that (directly or indirectly) references itself throws a `TypeError`:

```
const a = { name: "a" };  
a.self = a;  
JSON.stringify(a);  
// TypeError: Converting circular structure to JSON
```

There is no built-in fix — you either write a custom replacer that tracks visited objects and substitutes something like `"[Circular]"`, or use a library (e.g. `flatted`) designed for this.

25.3 `JSON.parse` — turning JSON text into JS values

```
const text = '{"name":"Ada","born":1815,"active":true}';  
const user = JSON.parse(text);  
console.log(user.name); // "Ada"
```

Malformed JSON throws a `SyntaxError` — always wrap untrusted input in a `try/catch`:

```
function safeParse(text, fallback = null) {
  try {
    return JSON.parse(text);
  } catch (err) {
    console.error("Invalid JSON:", err.message);
    return fallback;
  }
}
```

`JSON.parse` also accepts an optional **reviver** function, called bottom-up for every key/value pair, letting you transform values as they're parsed — a common use is reviving ISO date strings back into real `Date` objects:

```
const revived = JSON.parse(text, (key, value) => {
  if (key === "createdAt") return new Date(value);
  return value;
});
```

25.4 Deep cloning with JSON (and why it has limits)

A very common trick for a quick deep clone of "plain data" is:

```
const clone = JSON.parse(JSON.stringify(original));
```

This works, but only for values JSON itself can represent. It **silently loses**: `undefined` values, functions, `Date` objects (become strings, not `Date` instances again, unless you use a `reviver`), `Map` / `Set`, `RegExp`, and it throws on circular references. For real deep-cloning needs, prefer the built-in `structuredClone()` (available in modern browsers and Node 17+), which handles all of these correctly.

25.5 `localStorage` and `sessionStorage`

These are **browser-only** APIs — they do not exist in Node.js at all (`typeof localStorage` in a Node script is `"undefined"`). They let a web page persist small amounts of string data on the user's machine.

```
// Only works in a browser (or a browser-like environment)
localStorage.setItem("theme", "dark");
localStorage.getItem("theme"); // "dark"
localStorage.removeItem("theme");
localStorage.clear(); // wipes everything for this origin
localStorage.length; // number of stored keys
localStorage.key(0); // the first key's name
```

localStorage VS. sessionStorage

	localStorage	sessionStorage
Lifetime	Persists until explicitly cleared	Cleared when the tab/window closes
Shared across tabs?	Yes, for the same origin	No — each tab gets its own
Typical use	"Remember my theme preference"	"Keep this multi-step form's progress"

Storing objects: you must JSON-encode them yourself

Both storages only store **strings** — passing a non-string silently calls `.toString()` on it (so an object becomes the useless string `"[object Object]"`). Always go through `JSON.stringify / JSON.parse`:

```
const prefs = { theme: "dark", fontSize: 14 };
localStorage.setItem("prefs", JSON.stringify(prefs));

const stored = JSON.parse(localStorage.getItem("prefs") ?? "{}");
console.log(stored.theme); // "dark"
```

Limits and scoping

- Storage is **origin-scoped** — `https://a.com` cannot read `https://b.com`'s storage, and even `http://a.com` and `https://a.com` count as different origins.
- Browsers typically cap total storage per origin around **5–10 MB** (varies by browser) — plenty for preferences and cached small datasets, not for large files or "offline databases" (for that, look at IndexedDB, which is outside this book's scope).
- All storage APIs are **synchronous**, which means large read/writes can briefly block the main thread — another reason to keep stored data small.

A brief note on cookies, for contrast

Before `localStorage` existed (it arrived with HTML5 / ES5-era browsers), the only client-side storage was **cookies**. Cookies are still used today, but for a different purpose: unlike `localStorage`, cookies are automatically sent to the server with every matching HTTP request, have a much smaller size limit (~4 KB), and support an expiration date and security flags (`HttpOnly`, `Secure`, `SameSite`) relevant to authentication. Rule of thumb: use cookies for data the *server* needs to see on every request (like a session ID); use `localStorage` / `sessionStorage` for data only the *client* needs.

25.6 Chapter summary

- JSON is a strict, language-independent text format: double-quoted keys and strings only, no trailing commas, no comments, no functions.
- `JSON.stringify(value, replacer, space)` serializes JS values to JSON text; `replacer` filters/transforms, `space` pretty-prints.
- Objects with a `.toJSON()` method control their own serialization (this is how `Date` becomes an ISO string automatically).
- `JSON.stringify` throws on circular references and silently drops `undefined` / functions / Symbols.
- `JSON.parse(text, reviver)` parses JSON text back into JS values, and always throws `SyntaxError` on malformed input — wrap it in `try/catch`.
- `localStorage` / `sessionStorage` are browser-only, string-only, origin-scoped storage APIs — encode objects with `JSON.stringify` before storing them and `JSON.parse` after retrieving them.

25.7 Exercises

1. Run `examples/25-json-and-web-storage.js` and read through each section's output.
 2. Write a `replacer` function that redacts any key literally named `"password"` by replacing its value with `"***"` before stringifying.
 3. Try `JSON.parse("{invalid}")` in a `try/catch` and print a friendly error message instead of letting it crash your program.
-

Trying the storage APIs (browser only): open any web page's DevTools console and run:

```
localStorage.setItem("greeting", JSON.stringify({ hello: "world" }));
console.log(JSON.parse(localStorage.getItem("greeting")));
// { hello: "world" }
```

You can also inspect stored values visually in DevTools under **Application** → **Storage** → **Local Storage** (Chrome) or **Storage** → **Local Storage** (Firefox).

Chapter 26 — Regular Expressions

26.1 What regular expressions are for

A **regular expression** (regex) is a pattern used to match, extract, or replace text. Instead of writing manual loops that check characters one by one, you describe *the shape* of what you're looking for — "a sequence of digits," "an optional plus sign followed by digits," "anything between quotes" — and the regex engine does the searching.

Regexes are extremely powerful for text processing, but they are also famous for becoming unreadable quickly. This chapter teaches the syntax you'll actually use day to day, and is equally honest about when *not* to reach for a regex.

26.2 Creating a regex: literal vs. constructor

```
// Literal syntax – most common, pattern known at write-time
const re1 = /hello/;

// Constructor syntax – needed when building a pattern dynamically
const word = "hello";
const re2 = new RegExp(word);

// Constructor also accepts flags as a second argument
const re3 = new RegExp(word, "gi");
```

Use the literal form whenever the pattern is fixed; use `RegExp(...)` only when you need to build the pattern from a variable at runtime (e.g., searching for user-provided text).

26.3 Flags

Flags go after the closing slash (`/pattern/flags`) and change how the whole regex behaves:

Flag	Meaning
<code>g</code>	Global — find all matches, not just the first
<code>i</code>	Case-insensitive
<code>m</code>	Multiline — <code>^</code> / <code>\$</code> match the start/end of each line, not just the whole string
<code>s</code>	Dotall — <code>.</code> also matches newlines (normally it doesn't)
<code>u</code>	Unicode — treat the pattern/string as full Unicode code points

Flag	Meaning
y	Sticky — match only starting at <code>lastIndex</code> , no scanning ahead

```
/hello/i.test("Hello world"); // true – case-insensitive
"a1 b2 c3".match(/\d/g);      // ["1", "2", "3"] – all matches, not just the first
```

26.4 Character classes and metacharacters

Pattern	Matches
.	Any character except newline (unless <code>s</code> flag)
\d	A digit (<code>[0-9]</code>)
\D	A non-digit
\w	A "word" character (<code>[A-Za-z0-9_]</code>)
\W	A non-word character
\s	Whitespace (space, tab, newline)
\S	Non-whitespace
[abc]	Any one of <code>a</code> , <code>b</code> , or <code>c</code>
[^abc]	Any character except <code>a</code> , <code>b</code> , <code>c</code>
[a-z]	Any character in the range <code>a</code> to <code>z</code>
\. \. \. etc.	An escaped literal metacharacter

```
/\d{3}-\d{4}/.test("555-1234"); // true
/[aeiou]/.test("sky");           // false – no vowels
```

26.5 Quantifiers — how many times

Pattern	Meaning
*	0 or more
+	1 or more
?	0 or 1 (optional)
{n}	Exactly <code>n</code> times
{n,}	<code>n</code> or more times

Pattern	Meaning
<code>{n,m}</code>	Between <code>n</code> and <code>m</code> times

By default quantifiers are **greedy** — they consume as much as possible, then backtrack if needed. Appending `?` makes them **lazy** — they consume as little as possible:

```
"<a><b>".match(/<.+>)/[0]; // "<a><b>" - greedy, spans both tags
"<a><b>".match(/<.+?>)/[0]; // "<a>" - lazy, stops at the first ">"
```

26.6 Anchors and boundaries

```
/^hello/ // matches only at the start of the string (or line, with /m)
/world$/ // matches only at the end of the string (or line, with /m)
\bcat\b/ // \b is a word boundary - matches "cat" but not "category"
```

```
/^cat$/.test("cat"); // true - the entire string is exactly "cat"
/^cat$/.test("concat"); // false - anchors pin start and end
/\bcat\b/.test("concat"); // false - "cat" inside "concat" has no word boundary
/\bcat\b/.test("a cat!"); // true
```

26.7 Groups

Capturing groups `(...)` let you extract sub-matches:

```
const match = "2024-01-15".match(/(\d{4})-(\d{2})-(\d{2})/);
console.log(match[0]); // "2024-01-15" - the whole match
console.log(match[1]); // "2024" - first group
console.log(match[2]); // "01"
console.log(match[3]); // "15"
```

Non-capturing groups `(?:...)` group for alternation/quantifiers without creating a numbered capture:

```
/(?:Mr|Mrs|Ms)\. \w+/.test("Mrs. Smith"); // true, but no extra capture group
```

Named groups `(?<name>...)` are far more readable than numbered ones in complex patterns:

```
const { groups } = "2024-01-15".match(
  /(?<year>\d{4})-(?<month>\d{2})-(?<day>\d{2})/
);
console.log(groups.year, groups.month, groups.day); // 2024 01 15
```

26.8 Matching methods

Method	Called on	Returns
<code>regex.test(str)</code>	RegExp	<code>true</code> / <code>false</code>
<code>regex.exec(str)</code>	RegExp	Match array or <code>null</code> ; with <code>/g</code> , advances <code>lastIndex</code> each call
<code>str.match(regex)</code>	String	Like <code>exec</code> (no <code>/g</code>), or all matches as a plain array (with <code>/g</code> , but no groups)
<code>str.matchAll(regex)</code>	String	An iterator of every match, each with full group info (requires <code>/g</code>)
<code>str.replace(regex, replacement)</code>	String	New string with matches replaced (only first match unless <code>/g</code>)
<code>str.replaceAll(regex, replacement)</code>	String	New string, all matches replaced (regex must have <code>/g</code>)
<code>str.split(regex)</code>	String	Array split on matches

`exec` in a loop (manual global iteration)

```
const re = /\d+/g;
let m;
while ((m = re.exec("a1 b22 c333")) !== null) {
  console.log(m[0], "at index", m.index);
}
// "1" at index 1
// "22" at index 4
// "333" at index 8
```

`matchAll` — the modern, safer alternative

```
for (const m of "a1 b22 c333".matchAll(/\d+/g)) {
  console.log(m[0], "at index", m.index);
}
```

`matchAll` avoids the classic `exec` -in-a-loop bug where forgetting the `g` flag causes an **infinite loop** (because `lastIndex` never advances).

Replacing with capture groups

```
"2024-01-15".replace(/(\d{4})-(\d{2})-(\d{2})/, "$2/$3/$1");
// "01/15/2024" -- $1, $2, $3 refer to capture groups

"2024-01-15".replace(
  /(?<year>\d{4})-(?<month>\d{2})-(?<day>\d{2})/,
  "$<month>/<day>/<year>"
);
// "01/15/2024" -- named group references
```

Replacement can also be a **function**, called for each match:

```
"price: 5, qty: 3".replace(/\d+/g, (match) => String(Number(match) * 2));
// "price: 10, qty: 6"
```

26.9 Practical patterns

```
// Trim whitespace (built-in String.trim() is better, but shown for the pattern):
" hi ".replace(/^\s+|\s+$/g, ""); // "hi"

// Extract all numbers from a string:
"Order #123 has 4 items at $5.99".match(/\d+(\.\d+)?/g);
// ["123", "4", "5.99"]

// Split on multiple delimiters (comma, semicolon, or whitespace):
"a, b; c d".split(/[,\s;]+)/.filter(Boolean);
// ["a", "b", "c", "d"]

// Check if a string is only digits:
/^\d+$/.test("12345"); // true
/^\d+$/.test("123a5"); // false

// Replace multiple spaces with a single space:
"too many spaces".replace(/\s+/g, " ");
```

A word of caution on "validating" emails

A regex like `/^[^\s@]+@[^\s@]+\.[^\s@]+$/` catches obviously malformed input, but the actual email specification (RFC 5322) is far too complex to fully validate with a regex that stays readable. In real applications, use a light regex to catch typos early in the UI, and always confirm real validity by **sending a confirmation email** — that's the only actual proof an address is deliverable.

26.10 When *not* to use a regex

- **Parsing structured formats like HTML, JSON, or CSV with embedded quotes/commas.** These formats have nesting and escaping rules regex cannot reliably handle — use a real parser (`JSON.parse` , a DOM parser, a CSV library).
- **When a simple string method does the job.** `str.includes("foo")` , `str.startsWith("foo")` , and `str.split(",")` are faster to read and faster to run than the equivalent regex.
- **Extremely complex patterns nobody else can read six months later.** If a regex needs a paragraph of comments to explain, consider breaking it into named pieces, using named groups, or writing a small manual parser instead.

Catastrophic backtracking

Certain patterns — typically **nested quantifiers** like `(a+)+b` — can cause the regex engine's backtracking to explode exponentially on certain inputs, freezing your program. This is a real, exploitable denial-of-service vector (ReDoS) if you ever run a regex against untrusted user input. Prefer non-backtracking-friendly patterns (avoid nested `+/*` around overlapping character sets), test with adversarial inputs, or use a regex complexity linter in CI for public-facing input validation.

26.11 Chapter summary

- Regexes describe text patterns using literals (`/pattern/flags`) or `new RegExp(str, flags)` when built dynamically.
- Flags (`g` , `i` , `m` , `s` , `u` , `y`) change matching behavior globally.
- Character classes (`\d` , `\w` , `\s` , `[...]`), quantifiers (`*` , `+` , `?` , `{n,m}`), and anchors (`^` , `$` , `\b`) are the core building blocks.
- Groups can be capturing (`(...)`), non-capturing (`(?:...)`), or named (`(?<name>...)`).
- Prefer `matchAll` over manual `exec` loops to avoid the classic "forgot the `g` flag, infinite loop" bug.
- Regex is powerful but not always the right tool — avoid it for nested/ structured formats, and be careful with untrusted input and catastrophic backtracking.

26.12 Exercises

1. Run `examples/26-regular-expressions.js` and study each section.
2. Write a regex that extracts all hashtags (`#word`) from a string like `"loving #javascript and #regex today"` .
3. Write a regex (with named groups) that parses a simple `"HH:MM"` time string into hours and minutes, and reject strings like `"25:99"` by constraining the digit ranges as tightly as plain regex reasonably allows.

Chapter 27 — The DOM

27.1 What the DOM actually is

When a browser loads an HTML page, it doesn't just display the text of your HTML file — it parses it into an in-memory tree of objects called the **Document Object Model (DOM)**. Every tag becomes a **node** in that tree, and JavaScript can read and mutate that tree through the `document` object, which the browser then re-renders to the screen.

```
<body>
  <div id="app">
    <h1>Hello</h1>
    <p>World</p>
  </div>
</body>
```

becomes, conceptually:

```
document
├─ html
│   └─ body
│       └─ div#app
│           ├─ h1 ("Hello")
│           └─ p ("World")
```

This is why the DOM matters so much: it's the **live bridge** between your JavaScript and what the user actually sees. `document` is not part of the JavaScript language itself — it's a browser-provided API (see §1.4) — so everything in this chapter only works in a browser (or a browser-like environment such as `jsdom` in tests), never in plain Node.js.

27.2 Selecting elements

```
document.getElementById("app");           // single element by id (fastest)
document.querySelector(".card");          // first match of any CSS selector
document.querySelectorAll(".card");        // ALL matches, as a NodeList
document.getElementsByClassName("card");    // live HTMLCollection (older API)
document.getElementsByTagName("li");        // live HTMLCollection
```

`querySelector` / `querySelectorAll` accept **any valid CSS selector**, which makes them the most flexible and commonly used choice in modern code:

```
document.querySelector("ul > li:first-child");
document.querySelectorAll("input[type='checkbox']:checked");
document.querySelector("#app .card.active");
```

`querySelectorAll` returns a **static** `NodeList` (a snapshot at call time);

`getElementsByClassName` / `getElementsByTagName` return a **live** `HTMLCollection` that automatically updates as the DOM changes. This distinction occasionally matters — iterating a live collection while adding/removing matching elements can produce surprising results.

`NodeList` supports `forEach` directly; `HTMLCollection` does not (convert with `Array.from(collection)` first if you need array methods).

27.3 Traversing the tree

```
const app = document.querySelector("#app");

app.parentElement; // the element containing #app
app.children;      // HTMLCollection of direct element children
app.firstElementChild; // first child element (ignores text nodes)
app.lastElementChild;
app.nextElementSibling; // the next element at the same level
app.previousElementSibling;
```

There's a parallel, older set of properties (`.parentNode` , `.childNodes` , `.firstChild` , `.nextSibling` , ...) that operate on **all** nodes, including text nodes (like the whitespace between tags) and comment nodes — almost always prefer the `Element`-suffixed versions above unless you specifically need to see text/comment nodes.

27.4 Reading and writing content

```
const el = document.querySelector("#app");

el.textContent; // all text inside, with tags stripped
el.textContent = "New text"; // replaces content with plain text (safe)

el.innerHTML; // the raw HTML markup inside the element
el.innerHTML = "<b>Bold</b>"; // parses and inserts as real elements
```

`textContent` VS `innerHTML` — a real security boundary

`innerHTML` **parses** the string you assign as HTML. If that string ever contains untrusted user input, this is a classic **XSS (Cross-Site Scripting)** vulnerability — an attacker could set their display name to `<img src=x onerror="stealCookies()">`, and if you ever render it with `innerHTML`, that script

executes in every visitor's browser.

```
// DANGEROUS if `comment` came from user input:
el.innerHTML = comment;

// SAFE – always treated as plain text, never parsed as markup:
el.textContent = comment;
```

Rule of thumb: use `textContent` for any content that includes user input, unless you specifically need to render sanitized HTML (in which case, sanitize it with a dedicated library, never by hand).

27.5 Attributes vs. properties

Elements have both HTML **attributes** (what's written in the markup) and JS **properties** (the live object's fields) — they usually stay in sync, but not always, which trips people up:

```
const input = document.querySelector("input");

input.getAttribute("value"); // the ORIGINAL value from the HTML markup
input.value;                 // the CURRENT value, including live user typing

input.setAttribute("disabled", ""); // add an attribute
input.removeAttribute("disabled"); // remove it
input.hasAttribute("disabled");     // check for it

input.disabled = true; // most boolean attributes also have a matching property
```

For most day-to-day work (`value`, `checked`, `disabled`, `id`, `className`) use the **property**; reach for `getAttribute` / `setAttribute` for custom `data-*` attributes or attributes without a matching property (via `dataset` — see below).

`data-*` attributes and `.dataset`

```
<li data-user-id="42" data-role="admin">Ada</li>
```

```
const li = document.querySelector("li");
li.dataset.userId; // "42" (camelCase automatically from kebab-case)
li.dataset.role;   // "admin"
li.dataset.role = "editor"; // updates the data-role attribute too
```

27.6 Creating, inserting, and removing elements

```
const li = document.createElement("li");
li.textContent = "New item";
li.classList.add("item");

const list = document.querySelector("ul");
list.appendChild(li);           // add as the last child
list.prepend(li);              // add as the first child
list.insertBefore(li, list.firstChild); // insert at a specific position

otherElement.before(li); // insert li immediately before otherElement
otherElement.after(li);  // insert li immediately after otherElement
otherElement.replaceWith(li); // swap otherElement out for li

li.remove(); // remove an element from the DOM entirely
```

27.7 `classList` — the modern way to manage CSS classes

```
const el = document.querySelector(".card");

el.classList.add("active");
el.classList.remove("hidden");
el.classList.toggle("selected"); // add if absent, remove if present
el.classList.toggle("selected", isSel); // force add/remove based on a boolean
el.classList.contains("active"); // true/false
el.className; // the full class string, e.g. "card active"
```

Prefer `classList` over manually building `el.className` strings — it avoids duplicate classes and accidental whitespace bugs.

27.8 Inline styles

```
el.style.color = "red";
el.style.backgroundColor = "black"; // camelCase for hyphenated CSS properties
el.style.setProperty("--accent", "blue"); // for CSS custom properties (variables)

getComputedStyle(el).fontSize; // the ACTUAL rendered value, including CSS from stylesheets
```

`el.style` only reflects styles set **inline** (directly on the element or via JS) — it will not show you styles applied through an external or `<style>` stylesheet. Use `getComputedStyle(el)` when you need the final, rendered value regardless of where it came from. In general, prefer toggling CSS **classes** over setting individual inline styles from JS — it keeps your visual logic in CSS, where it's easier to maintain.

27.9 A note on performance: reflow and batching

Every time you change something that affects layout (adding elements, changing size-affecting styles, reading certain properties like `offsetHeight`), the browser may need to recalculate the page's layout ("reflow") and repaint it. Doing this in a tight loop, one element at a time, can be slow:

```
// Slow: triggers a layout recalculation on every iteration
for (const item of items) {
  list.appendChild(makeListItem(item)); // 1000 separate reflows, worst case
}

// Fast: build everything off-DOM first, then insert once
const fragment = document.createDocumentFragment();
for (const item of items) {
  fragment.appendChild(makeListItem(item));
}
list.appendChild(fragment); // a single reflow
```

`DocumentFragment` is a lightweight, in-memory container that isn't part of the visible page — appending to it doesn't trigger a reflow, and appending the whole fragment to the real DOM does so only once. This pattern (batch changes, then apply once) is the core idea behind why frameworks like React use a virtual DOM: to compute the minimal set of real DOM changes and apply them together instead of one at a time.

27.10 Chapter summary

- The DOM is a live, in-memory tree representation of your HTML that JavaScript can read and mutate through the `document` object.
- `querySelector` / `querySelectorAll` (any CSS selector) are the modern, flexible way to find elements.
- Use `textContent` for user-supplied content to avoid XSS; reserve `innerHTML` for trusted or sanitized markup.
- Attributes (markup) and properties (live object state) usually stay in sync but aren't identical — use `.dataset` for custom `data-*` attributes.
- `createElement` / `appendChild` / `remove` build and tear down the tree; `classList` is the preferred way to manage CSS classes from JS.
- Batch DOM insertions with a `DocumentFragment` to avoid repeated, expensive layout recalculations.

27.11 Exercises

1. Open `examples/27-dom-manipulation.html` directly in a browser and use the working to-do list. Read through the `<script>` section and match each interaction to the API it uses.
2. Modify the demo so completed items get a `line-through` CSS class instead of just a `completed` marker class, using `classList.toggle`.
3. Add a "Clear completed" button that removes every item with the `completed` class in one batch operation.

Chapter 28 — Events

28.1 The event model

The DOM is **event-driven**: instead of your code polling "did the user click yet?", the browser fires **events** — click, keypress, form submit, page load, and hundreds more — and your code subscribes to the ones it cares about with **listeners**.

```
button.addEventListener("click", (event) => {
  console.log("Clicked!", event);
});

button.removeEventListener("click", handlerFunction); // must be the SAME
                                                         // function reference
```

You can attach multiple listeners for the same event on the same element — they all run, in the order they were added. `removeEventListener` only works if you pass the exact same function reference that was added (an inline arrow function passed directly to `addEventListener` can never be removed later, because you have no reference to it).

28.2 The event object

Every listener receives an **event object** describing what happened:

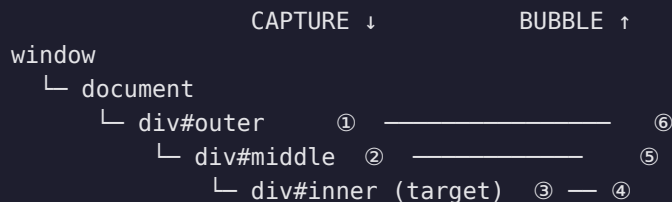
```
element.addEventListener("click", (event) => {
  event.type;           // "click"
  event.target;          // the actual element that triggered the event
  event.currentTarget;   // the element the listener is attached to
  event.clientX;         // mouse coordinates (for mouse events)
  event.key;             // the key pressed (for keyboard events)
  event.preventDefault(); // cancel the default browser behavior
  event.stopPropagation(); // stop the event from bubbling further
});
```

`event.target` vs. `event.currentTarget` is a frequent source of confusion: if you attach a listener to a `<ul>` but the user clicks a `<li>` inside it, `target` is the `<li>` (where the click actually happened) while `currentTarget` is the `<ul>` (where the listener you're inside of is attached). This distinction is the entire basis of **event delegation** (§28.4).

28.3 Bubbling and capturing

When an event fires on an element, it doesn't just run listeners on that element — it travels through the DOM tree in three phases:

1. **Capturing phase** — the event travels *down* from `window` to the target, through every ancestor.
2. **Target phase** — the event reaches the actual element it happened on.
3. **Bubbling phase** — the event travels back *up*, from the target through every ancestor again.



By default, `addEventListener(type, handler)` listens during the **bubbling** phase. Pass `true` (or `{ capture: true }`) as the third argument to listen during **capturing** instead:

```
outer.addEventListener("click", () => console.log("outer bubble"));
outer.addEventListener("click", () => console.log("outer capture"), true);
middle.addEventListener("click", () => console.log("middle bubble"));
inner.addEventListener("click", () => console.log("inner (target)"));

// Clicking #inner logs, in order:
// "outer capture" -> capturing phase reaches outer first
// "inner (target)" -> target phase
// "middle bubble" -> bubbling phase, innermost ancestor first
// "outer bubble" -> bubbling phase continues up
```

Most everyday code only ever uses the default bubbling phase. Capturing is mainly useful when you need to intercept an event **before** a child element's own handler can act on it (e.g., a global "close all dropdowns" handler).

28.4 Event delegation

Because events bubble, you don't need to attach a separate listener to every single child element — you can attach **one** listener to a shared parent and inspect `event.target` to figure out which child was actually clicked. This is called **event delegation**, and it has two major advantages:

- **Performance** — one listener instead of hundreds (e.g., for a table with 1,000 rows).
- **Works for elements that don't exist yet** — if you dynamically add new `<li>` items later, a delegated listener on the parent `<ul>` still catches clicks on them automatically, with zero extra code. A listener attached directly to each `<li>` would need to be re-attached every time you add a new one.

```
list.addEventListener("click", (event) => {
  const item = event.target.closest("li"); // find the nearest <li> ancestor
  if (!item) return; // click landed outside any <li> (e.g. on the <ul> padding)
  console.log("Clicked item:", item.textContent);
});

// Adding a new item later needs NO new listener:
const li = document.createElement("li");
li.textContent = "New item";
list.appendChild(li); // clicking this still triggers the delegated handler
```

`Element.closest(selector)` walks **up** from the clicked element (or the element itself) looking for the nearest ancestor matching the selector — essential for delegation, since `event.target` might be a `<span>` or `<svg>` nested inside the `<li>` you actually care about, not the `<li>` itself.

28.5 `preventDefault` VS. `stopPropagation` VS. `stopImmediatePropagation`

These three methods are frequently confused because they sound similar but control completely different things:

Method	Stops...
<code>event.preventDefault()</code>	The browser's default action for this event (e.g., following a link, submitting a form, checking a checkbox) — does not affect bubbling
<code>event.stopPropagation()</code>	The event from bubbling/capturing to other elements — does not affect the browser's default action, and does not stop other listeners on the <i>same</i> element
<code>event.stopImmediatePropagation()</code>	Both propagation and any other listeners still queued on the <i>same</i> element

```
form.addEventListener("submit", (event) => {
  event.preventDefault(); // stop the page from reloading on submit
  // ...handle the form data with JS instead
});

child.addEventListener("click", (event) => {
  event.stopPropagation(); // parent's click listener will NOT fire
});
```

28.6 Custom events

You can define and dispatch your own events with `CustomEvent`, useful for decoupled communication between unrelated parts of your code (e.g., a cart module notifying a header badge component without either directly referencing the other):

```
const event = new CustomEvent("cart:updated", {
  detail: { itemCount: 3 }, // arbitrary data payload
  bubbles: true,           // whether it bubbles like a native event
});

document.dispatchEvent(event);

document.addEventListener("cart:updated", (event) => {
  console.log("Cart now has", event.detail.itemCount, "items");
});
```

28.7 Common events reference

Event	Fires on	When
<code>click</code>	Any element	Mouse click (or Enter/Space on a focused button)
<code>dblclick</code>	Any element	Double click
<code>submit</code>	<code><form></code>	Form submission (button click or Enter in a field)
<code>input</code>	Form fields	Value changes, fires on every keystroke
<code>change</code>	Form fields	Value changes and the field loses focus (or a select/checkbox toggles)
<code>keydown</code> / <code>keyup</code>	Focused element	A key is pressed / released
<code>focus</code> / <code>blur</code>	Form fields	Element gains / loses focus (do not bubble!)
<code>mouseenter</code> / <code>mouseleave</code>	Any element	Pointer enters/leaves (do not bubble)
<code>mouseover</code> / <code>mouseout</code>	Any element	Pointer enters/leaves (do bubble)
<code>DOMContentLoaded</code>	<code>document</code>	HTML is fully parsed (before images/styles finish loading)
<code>load</code>	<code>window</code>	Everything (images, styles, scripts) has fully loaded

`input` vs. `change` trips people up constantly: use `input` for "respond immediately as the user types" (live search, character counters), and `change` for "respond only once they're done" (validating a field after they tab away).

28.8 Chapter summary

- Events let your code react to user/browser actions via `addEventListener`, which receives an event object describing what happened.
- Events travel through three phases: capturing (down), target, bubbling (up) — most listeners use the default bubbling phase.
- Event delegation exploits bubbling to handle many (including future/dynamic) child elements with a single listener on a shared ancestor, using `event.target.closest(...)`.
- `preventDefault` stops default browser behavior; `stopPropagation` stops bubbling/capturing; they are independent of each other.
- `CustomEvent` lets you build your own event-based communication between unrelated parts of an application.

28.9 Exercises

1. Open `examples/28-events.html` in a browser. Click the nested boxes and read the on-page log to see the exact bubbling/capturing order.
2. In the same file, click "Add item" several times, then click one of the newly-added list items — confirm the delegated listener still catches it without any new code running.
3. Modify the demo to call `stopPropagation()` on the middle box's click listener, and predict (then verify) which log lines disappear.

Chapter 29 — Fetch, AJAX, and HTTP

29.1 A brief history

AJAX (Asynchronous JavaScript and XML) is the general technique of a web page making an HTTP request *after* it has already loaded, without a full page reload — the thing that made Gmail-style single-page apps possible in the mid-2000s. The name is historical; nobody actually sends XML anymore (JSON won), but "AJAX request" stuck as the generic term.

- **XMLHttpRequest (XHR)** — the original browser API for this, from 1999. It works, but its callback-based, event-driven API (`onreadystatechange` , checking `readyState` and `status` manually) is verbose and easy to get wrong.
- **jQuery's `$.ajax()`** — a popular wrapper around XHR that dominated the 2010s, offering a much nicer callback/promise-like API before the platform caught up.
- **`fetch`** — the modern, native, promise-based replacement, available in every modern browser and (since Node 18) in Node.js itself, with no library required.

This chapter focuses on `fetch` , which is what you should reach for in new code.

29.2 The `fetch` API basics

```
const response = await fetch("https://api.example.com/users");
const data = await response.json(); // parses the response body as JSON
console.log(data);
```

`fetch(url, options)` returns a **Promise that resolves to a `Response` object** representing the HTTP response — headers, status code, and a body you read separately using one of several methods:

```
response.json(); // parse body as JSON -> Promise<any>
response.text(); // read body as plain text -> Promise<string>
response.blob(); // read body as binary data -> Promise<Blob>
response.status; // e.g. 200, 404, 500
response.ok;     // true if status is in the 200-299 range
response.headers.get("Content-Type");
```

GET request (the default)

```
const res = await fetch("/api/products");
const products = await res.json();
```

POST request with a JSON body

```
const res = await fetch("/api/products", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ name: "Widget", price: 9.99 }),
});
const created = await res.json();
```

You must set the `Content-Type` header yourself when sending JSON — `fetch` does not infer it from the body — and `body` must be a **string** (or `FormData` / `Blob` /etc.), so a plain object always needs `JSON.stringify` first.

29.3 The gotcha: `fetch` does NOT reject on HTTP errors

This surprises almost everyone the first time: **`fetch`'s promise only rejects on network failure** (DNS failure, no connection, CORS block) — **not** when the server responds with a 404 or 500. A "successful" fetch of a 404 page still resolves normally; you must check `response.ok` yourself:

```
async function getUser(id) {
  const res = await fetch(`/api/users/${id}`);
  if (!res.ok) {
    // This branch runs for 404, 500, etc – fetch did NOT throw for these.
    throw new Error(`Request failed: ${res.status} ${res.statusText}`);
  }
  return res.json();
}

try {
  const user = await getUser(999);
} catch (err) {
  console.error(err.message); // "Request failed: 404 Not Found"
}
```

Forgetting this check is one of the most common real-world bugs in fetch code — a 404 response silently "succeeds" and you get a confusing crash later when you try to read a field from an error page's body instead of the JSON you expected.

29.4 `async / await` with fetch

Because `fetch` is promise-based, it pairs naturally with `async / await` (Chapter 23) for readable sequential code, including proper error handling with `try/catch`:

```

async function loadDashboard(userId) {
  try {
    const [user, orders] = await Promise.all([
      fetch(`/api/users/${userId}`).then((r) => r.json()),
      fetch(`/api/users/${userId}/orders`).then((r) => r.json()),
    ]);
    return { user, orders };
  } catch (err) {
    console.error("Dashboard load failed:", err);
    throw err;
  }
}

```

29.5 Cancelling requests with `AbortController`

Sometimes you need to cancel an in-flight request — the classic example is a search-as-you-type box, where you want to abandon a stale request if the user keeps typing:

```

let controller;

async function search(query) {
  controller?.abort(); // cancel any previous, still-pending search
  controller = new AbortController();

  try {
    const res = await fetch(`/api/search?q=${query}`, {
      signal: controller.signal,
    });
    return await res.json();
  } catch (err) {
    if (err.name === "AbortError") {
      console.log("Previous search cancelled – ignoring");
      return; // not a real error, just an intentional cancellation
    }
    throw err;
  }
}

```

`AbortController` is also the standard way to implement request timeouts:

```

async function fetchWithTimeout(url, ms) {
  const controller = new AbortController();
  const timer = setTimeout(() => controller.abort(), ms);
  try {
    return await fetch(url, { signal: controller.signal });
  } finally {
    clearTimeout(timer);
  }
}

```

29.6 CORS — why you sometimes see errors in the console

Browsers enforce the **same-origin policy**: by default, JavaScript running on `https://siteA.com` cannot read responses from `https://siteB.com` — even though the request itself may actually reach the server. This exists to protect users: without it, any malicious page you accidentally visited could make authenticated requests to your bank using cookies your browser automatically attaches, and read the response.

CORS (Cross-Origin Resource Sharing) is the mechanism that lets a server *opt in* to allowing specific other origins to read its responses, via response headers like:

```
Access-Control-Allow-Origin: https://siteA.com
Access-Control-Allow-Methods: GET, POST
Access-Control-Allow-Headers: Content-Type
```

If those headers are missing or don't match, the browser blocks your JS from reading the response and logs a CORS error in the console — this is a **browser-enforced client-side restriction**, not something you can fix from the requesting page's JavaScript. It must be fixed on the **server** you're calling (by adding the right headers), or worked around with a proxy server you control. This is also why CORS errors never happen in Node.js scripts or `curl` — the same-origin policy is a browser-specific security feature, not an HTTP-level restriction.

For requests with methods/headers beyond simple `GET / POST` with basic headers, browsers first send an automatic **preflight** `OPTIONS` request to ask the server "would you allow this?" before sending the real request — you'll sometimes see this extra `OPTIONS` request in your network tab.

29.7 Node.js's built-in `fetch` vs. browser `fetch`

Since **Node.js 18**, `fetch` is available globally with no import and no extra dependency (like `node-fetch`), and behaves almost identically to the browser version — same `Response / Request / Headers` classes, same promise-based API. The differences that matter in practice:

- **No CORS in Node.js** — CORS is a browser security feature; a Node script can fetch from any server that allows the *connection* itself, regardless of `Access-Control-Allow-Origin` headers.
- **No cookies/credentials by default the same way** — browsers automatically attach same-origin cookies; Node's `fetch` does not have a browser cookie jar at all.
- Both support `AbortController`, streaming bodies, and the same `Response` methods (`.json()`, `.text()`, etc.).

This means you can write `fetch`-based code once and mostly reuse it in both environments — a big part of why libraries increasingly ship "isomorphic" HTTP logic.

29.8 Chapter summary

- `fetch(url, options)` is the modern, native, promise-based way to make HTTP requests, replacing `XMLHttpRequest` and jQuery's `$.ajax`.
- `fetch`'s promise resolves even for 4xx/5xx responses — **always check** `response.ok` before treating a request as successful.
- POST/PUT requests need `JSON.stringify` on the body and an explicit `Content-Type: application/json` header.
- `AbortController` lets you cancel in-flight requests or implement timeouts.
- CORS is a **browser-only**, server-controlled security mechanism governing which origins may *read* a cross-origin response — it cannot be fixed from the calling page's JavaScript.
- Node.js 18+ has a built-in `fetch` with the same core API as browsers, minus browser-specific concerns like CORS and automatic cookies.

29.9 Exercises

1. Run `examples/29-fetch-and-ajax.js`. It starts its own local HTTP server and makes real GET/POST requests against it with Node's built-in `fetch` — no internet connection required.
2. Modify the example to request a route that doesn't exist, and confirm `response.ok` is `false` and `response.status` is `404` even though `fetch` did not throw.
3. Add a request timeout using `AbortController` and `setTimeout`, and test it against an endpoint you make artificially slow.

Chapter 30 — Node.js Fundamentals

30.1 What Node.js actually is

Node.js is not a language, a framework, or a browser — it is a **runtime**: a program that lets you execute JavaScript outside of a web browser. Node was created in 2009 by Ryan Dahl, who took Google's V8 engine (the same engine that powers Chrome) and embedded it inside a C++ program that adds capabilities V8 alone doesn't have: reading files, opening network sockets, spawning processes, and so on.

Node.js is built from three main pieces:

- **V8** — parses and executes your JavaScript, exactly as covered in Chapter 1.
- **libuv** — a C library that provides the event loop, asynchronous I/O, and a thread pool for operations that can't be done non-blockingly at the OS level (like some file system calls and DNS lookups).
- **Node APIs** — a set of built-in modules written partly in C++ and partly in JavaScript that expose libuv's capabilities to your code: `fs`, `http`, `path`, `os`, `events`, `crypto`, `stream`, and dozens more.

This is the same "engine + host environment" split from Chapter 1, just with Node.js as the host instead of a browser.



30.2 The global object: `globalThis`, and what's missing

In a browser, the global object is `window`. In Node.js, the global object is `global` (both are also accessible through the universal `globalThis`, standardized in ES2020 specifically so code can refer to "the global object" without caring which environment it's in).

```
console.log(globalThis === global); // true, in Node.js
```

Node.js does **not** have `window`, `document`, or any DOM API — there is no web page to represent. Instead it exposes server-oriented globals that browsers don't have:

Global	Purpose
<code>process</code>	Information about and control over the current Node.js process
<code>__dirname</code> / <code>__filename</code>	Absolute path of the current file/folder (CommonJS only)
<code>Buffer</code>	Working with raw binary data
<code>require</code> / <code>module.exports</code>	CommonJS module system (see Chapter 24)
<code>setTimeout</code> , <code>setInterval</code> , <code>console</code>	Present in both Node.js and browsers

30.3 `process`: talking to the operating system

`process` is a global object that represents the currently running Node.js program. It's one of the first things you reach for in any real script.

```
console.log(process.platform); // "linux", "darwin", "win32", ...
console.log(process.version);  // "v20.11.0"
console.log(process.pid);      // the OS process ID

// Command-line arguments – argv[0] is the node binary, argv[1] is the
// script path, and everything after that is what the user typed.
console.log(process.argv);
// $ node script.js hello world
// [ '/usr/bin/node', '/path/to/script.js', 'hello', 'world' ]

// Environment variables
console.log(process.env.HOME);
console.log(process.env.NODE_ENV ?? "development");

// Exiting with a status code (0 = success, non-zero = failure).
// Useful in CLI tools and CI scripts.
if (someFatalCondition) {
  console.error("Fatal error, aborting.");
  process.exit(1);
}
```

`process.argv` is how you'll parse command-line arguments for CLI tools throughout this book (see the Todo app in Chapter 35) — no dependency needed for simple cases.

30.4 The `fs` module: reading and writing files

`fs` (file system) is one of the most-used Node.js built-ins. It has three flavors of nearly every function:

1. **Synchronous** (`fs.readFileSync`) — blocks the entire process until done. Fine for CLI scripts and startup code; avoid in servers.
2. **Callback-based** (`fs.readFile`) — the original Node.js async style (see Chapter 21). Non-blocking, but leads to callback nesting.
3. **Promise-based** (`fs/promises`) — modern, works beautifully with `async / await` (see Chapter 23). Prefer this for new code.

```
import { readFileSync, writeFileSync } from "node:fs";
import { readFile, writeFile } from "node:fs/promises";

// Synchronous – simple, but blocks the thread.
writeFileSync("notes.txt", "Hello from Node.js!\n");
const contents = readFileSync("notes.txt", "utf8");
console.log(contents);

// Promise-based – non-blocking, composes with async/await.
async function main() {
  await writeFile("notes-async.txt", "Written asynchronously.\n");
  const text = await readFile("notes-async.txt", "utf8");
  console.log(text);
}
main();
```

A subtle but important detail: without the `"utf8"` encoding argument, `readFileSync / readFile` return a raw `Buffer` (binary data), not a string. Always pass an encoding when you expect text.

30.5 The `path` module: cross-platform file paths

Never build file paths with string concatenation (`dir + "/" + file`) — Windows uses `\` as a separator, macOS/Linux use `/`, and hand-rolled path logic breaks the moment your code runs somewhere else. The `path` module handles this correctly:

```
import path from "node:path";

path.join("data", "users", "42.json");
// "data/users/42.json" (or "data\\users\\42.json" on Windows)

path.resolve("data", "users", "42.json");
// absolute path from the current working directory

path.basename("/home/user/report.pdf"); // "report.pdf"
path.extname("/home/user/report.pdf"); // ".pdf"
path.dirname("/home/user/report.pdf"); // "/home/user"
```

30.6 Building a minimal HTTP server

Frameworks like Express are built on top of Node's core `http` module. It's worth seeing the low-level version once so the framework doesn't feel like magic:

```
import http from "node:http";

const server = http.createServer((req, res) => {
  if (req.url === "/" && req.method === "GET") {
    res.writeHead(200, { "Content-Type": "text/plain" });
    res.end("Hello from a raw Node.js server!\n");
  } else {
    res.writeHead(404, { "Content-Type": "text/plain" });
    res.end("Not found\n");
  }
});

server.listen(3000, () => {
  console.log("Listening on http://localhost:3000");
});
```

Every incoming request runs your callback once, with a `req` (request, readable) and `res` (response, writable) object. There's no routing, no JSON body parsing, no middleware — that's exactly the gap that frameworks like Express fill.

30.7 The `events` module: `EventEmitter`

`EventEmitter` is the class that powers Node's asynchronous APIs internally (HTTP servers, streams, file watchers) and is directly usable in your own code. It implements the **observer pattern** (formalized in Chapter 33): objects can emit named events, and any number of listeners can subscribe to react to them.

```
import { EventEmitter } from "node:events";

class OrderSystem extends EventEmitter {
  placeOrder(item) {
    console.log(`Order placed: ${item}`);
    this.emit("order:placed", { item, timestamp: Date.now() });
  }
}

const orders = new OrderSystem();

orders.on("order:placed", (order) => {
  console.log(`[email] Sending confirmation for ${order.item}`);
});

orders.on("order:placed", (order) => {
  console.log(`[inventory] Decrementing stock for ${order.item}`);
});

orders.placeOrder("Wireless Mouse");
// Order placed: Wireless Mouse
// [email] Sending confirmation for Wireless Mouse
// [inventory] Decrementing stock for Wireless Mouse
```

Both listeners fire synchronously, in the order they were registered, every time `emit` is called. This decoupling — the order system doesn't know or care who's listening — is the same principle behind DOM events (Chapter 28) and is one of the most reusable patterns in JavaScript.

30.8 Modules recap

Chapter 24 covers the module system (`import / export` vs. `require / module.exports`) in depth. The short version for Node.js specifically: a project is treated as ECMAScript Modules (ESM, the `import` syntax) if its `package.json` has `"type": "module"`, or CommonJS (`require`) otherwise. File extensions can also force the choice: `.mjs` is always ESM, `.cjs` is always CommonJS regardless of `package.json`.

30.9 Release lines and LTS

Node.js ships a new major version roughly every six months. Even-numbered majors (18, 20, 22, 24...) become **Long-Term Support (LTS)** releases a few months after launch and are then supported (with security and critical bug fixes) for about 30 months. Odd-numbered majors are "Current" releases meant for experimenting with the newest features, not production. **Rule of thumb: for production services, run the active LTS version.** Tools like `nvm` (Node Version Manager) let you install and switch between multiple Node.js versions on one machine, which matters because different projects on your disk may target different LTS lines.

30.10 Chapter summary

- Node.js pairs the V8 JavaScript engine with libuv (the event loop and async I/O) and a library of built-in APIs (`fs` , `http` , `path` , `os` , `events` , and more) — together, the "Node runtime."
- Node has its own globals (`process` , `Buffer` , `__dirname`) and lacks browser-only globals (`window` , `document`).
- `fs` and `fs/promises` read and write files; prefer the promise-based API for anything beyond quick scripts.
- `path` builds file paths safely across operating systems.
- The core `http` module can create a working server with no dependencies; frameworks like Express add routing and convenience on top of it.
- `EventEmitter` is the built-in implementation of the observer pattern and underlies much of Node's own asynchronous API surface.
- Use the active LTS release for production work.

30.11 Exercises

1. Write a script that reads its own source file (`__filename` or `import.meta.url`) and prints how many lines it contains.
2. Extend the minimal HTTP server to respond with JSON (`Content-Type: application/json`) at a `/time` route that returns the current server time.
3. Create an `EventEmitter` subclass that models a `TrafficLight` emitting `"red"` , `"yellow"` , and `"green"` events, with at least two independent listeners reacting differently to each color.

Chapter 31 — npm and Package Management

31.1 What npm is

npm ("Node Package Manager") is three things bundled under one name:

1. **A command-line tool** (`npm`) installed alongside Node.js, used to install, update, and manage dependencies.
2. **A public registry** at `npmjs.com` hosting well over two million packages — reusable pieces of JavaScript anyone can publish and anyone can install.
3. **A file format**, `package.json`, that describes a project's metadata, scripts, and dependencies in a way both humans and tools can read.

Without a package manager, using a third-party library meant manually downloading a `.js` file and dropping it into your project — no version tracking, no dependency resolution, no easy updates. npm (and its alternatives) solved that problem for the entire JavaScript ecosystem.

31.2 Anatomy of `package.json`

Every npm-managed project has a `package.json` at its root. Here's what the common fields mean (see `examples/31-npm-and-package-management/package.json` for a full working example, and its neighboring `README.md` for a field-by-field walkthrough since JSON has no comment syntax):

```
{
  "name": "my-app",
  "version": "1.2.0",
  "description": "A short description of the package.",
  "main": "index.js",
  "type": "module",
  "scripts": {
    "start": "node index.js",
    "test": "node --test"
  },
  "dependencies": {
    "express": "^4.19.2"
  },
  "devDependencies": {
    "eslint": "^9.0.0"
  },
  "peerDependencies": {
    "react": ">=18.0.0"
  },
  "engines": {
    "node": ">=18"
  }
}
```

- **name** / **version** — required if you intend to publish; version must follow semantic versioning (next section).
- **main** — the entry file `require("my-app")` resolves to (CommonJS). For ESM projects, `exports` is the modern replacement, allowing a package to expose multiple, explicitly defined entry points.
- **type** — `"module"` makes `.js` files in this package ECMAScript Modules (`import` / `export`); omitted or `"commonjs"` keeps `require`.
- **scripts** — named shell commands runnable via `npm run <name>`. `start`, `test`, `build`, and a few others get a shorthand (`npm start`, `npm test`) without needing `run`.
- **dependencies** — packages required at runtime, in production.
- **devDependencies** — packages only needed while developing (test runners, linters, bundlers) — not installed when a consumer of *your* package installs it, but installed when *you* run `npm install` in this repo.
- **peerDependencies** — "this package expects the *consumer's* project to already have a compatible version of X installed" — common for plugins (a React component library declares `react` as a peer dependency rather than bundling its own copy).
- **engines** — documents (and can be enforced to require) a minimum Node.js version.

31.3 Semantic versioning (SemVer)

Package versions follow `MAJOR.MINOR.PATCH`, e.g. `4.19.2` :

- **MAJOR** — incremented for breaking changes (`3.x` → `4.0.0`).
- **MINOR** — incremented for backward-compatible new features (`4.18.x` → `4.19.0`).
- **PATCH** — incremented for backward-compatible bug fixes (`4.19.1` → `4.19.2`).

In `package.json`, version ranges use prefixes to say how much automatic updating is allowed:

Prefix	Meaning	Example allows
<code>^4.19.2</code>	Compatible within the same major version	<code>4.19.3</code> , <code>4.20.0</code> , not <code>5.0.0</code>
<code>~4.19.2</code>	Compatible within the same minor version	<code>4.19.3</code> , not <code>4.20.0</code>
<code>4.19.2</code>	Exact version only	only <code>4.19.2</code>
<code>*</code> or <code>latest</code>	Any version	anything (avoid in real projects)

`^` (caret) is the npm default and the most common choice — it trusts the package author's SemVer promise that minor/patch releases won't break your code. That trust is sometimes violated, which is exactly why lockfiles exist.

31.4 `package-lock.json` and reproducible installs

`package.json` describes *acceptable ranges*; `package-lock.json` records the **exact** version of every package (and every dependency of every dependency) that was actually installed, down to the exact download URL and integrity hash. Two developers running `npm install` against the same `package.json` could, in theory, get different transitive dependency versions if a sub-dependency published a new minor release between their installs — the lockfile eliminates that non-determinism.

Always commit `package-lock.json` to version control. It's what makes "works on my machine" reproducible on your teammate's machine and in CI.

31.5 Core commands

```

npm install           # install everything in package.json (writes/updates the lockfile)
npm install express   # add express to dependencies, install it
npm install -D eslint # add eslint to devDependencies (-D / --save-dev)
npm install -g nodemon # install globally (available as a CLI command everywhere)
npm uninstall express  # remove a dependency
npm update             # update packages within their allowed SemVer ranges
npm run build          # run the "build" script from package.json
npm start              # shorthand for `npm run start`
npm test               # shorthand for `npm run test`
npm ci                 # "clean install" – strictly follows the lockfile, used in CI

```

`npm ci` deletes `node_modules` first and installs *exactly* what `package-lock.json` specifies, failing loudly if `package.json` and the lockfile disagree. It's faster and safer than `npm install` for automated environments (CI pipelines, Docker builds) precisely because it refuses to silently resolve new versions.

31.6 `npx` : running packages without installing them globally

`npx` (bundled with `npm` since v5.2) runs a package's CLI binary, installing a temporary copy if it isn't already available locally — useful for one-off tool usage without polluting your global install list:

```
npx create-react-app my-app
npx cowsay "Hello!"
```

If the package is already listed in your project's `devDependencies`, `npx <command>` will run the local copy from `node_modules/.bin` instead of downloading anything, which is the more common real-world use: running locally-installed CLI tools (`npx eslint .`, `npx tsc`) without needing them on your system `PATH`.

31.7 Alternative package managers: Yarn and pnpm

`npm` isn't the only option compatible with the `package.json` / `npm` registry ecosystem:

- **Yarn** (2016, Facebook) — originally created to fix `npm`'s reliability and speed problems at the time (most have since been fixed in `npm` itself). Yarn 1 is largely `npm`-compatible; Yarn Berry (2+) introduced its own architecture, including "Plug'n'Play" mode that skips `node_modules` entirely.
- **pnpm** — its main selling point is disk efficiency: instead of copying every package into every project's `node_modules` (`npm`/Yarn's historical approach), `pnpm` keeps one global content-addressable store on disk and uses hard links/symlinks per project, saving significant space when you have many projects. It also enforces stricter dependency isolation (a package can't accidentally `require` a dependency it never declared, a common class of bug called "phantom dependencies").

Both Yarn and `pnpm` have first-class **workspaces** support — managing multiple related packages (a "monorepo") from one repository root, sharing a single lockfile and hoisted dependencies. `npm` added workspaces support in v7+ as well. Choosing between them is mostly a team/tooling preference today; the concepts in this chapter (SemVer, lockfiles, `dependencies` vs `devDependencies`) apply to all three.

31.8 Publishing a package (conceptual overview)

Publishing your own package to the npm registry, at a high level:

1. Create a `package.json` with a unique `name` (or a scoped name like `@yourusername/package-name` to avoid collisions) and a starting `version` (conventionally `1.0.0` or `0.1.0`).
2. Run `npm login` once to authenticate the CLI with your npm account.
3. Run `npm publish` from the package root. npm uploads everything not excluded by `.npmignore` (or, if absent, `.gitignore`) or the `files` field in `package.json`.
4. For subsequent releases, bump the version (`npm version patch|minor|major`, which also updates `package.json` and creates a git tag) and `npm publish` again — npm will not let you re-publish the same version number, which is a deliberate safeguard for reproducibility.

This book doesn't walk through publishing hands-on since it requires a registry account, but the mental model above covers what happens when you `npm install` any public package: someone ran exactly this process.

31.9 Security: supply-chain awareness

Every `dependency` you add is code you did not write, running with the same permissions as your own code — including transitive dependencies you never explicitly chose. A few practical habits:

- **`npm audit`** scans your installed dependency tree against a database of known vulnerabilities and reports affected packages, often with a suggested fix version (`npm audit fix`).
- **Be deliberate about new dependencies.** Before installing a package for a small task, consider whether a few lines of your own code (or a built-in like `fs`, `crypto`, or `Array` methods) would do the job without adding a new piece of the supply chain to trust.
- **`postinstall` scripts** are arbitrary shell commands a package can run automatically the moment it's installed — this has been the entry point for real supply-chain attacks (a compromised or typo-squatted package exfiltrating environment variables during `npm install`, before you've even imported it). Review unfamiliar packages, especially ones with install scripts, before adding them to a project with real secrets in its environment.
- **Pin and review lockfile diffs.** A lockfile diff that changes far more packages than you'd expect from the one dependency you just bumped is worth a second look before merging.

None of this means avoid npm packages — the ecosystem's size is one of JavaScript's biggest practical advantages. It means treating "add a dependency" as a decision with a real (if usually small) cost, not a free action.

31.10 Chapter summary

- npm is a CLI, a public registry, and the `package.json` file format together.

- `dependencies` ship to production; `devDependencies` are for local tooling only; `peerDependencies` declare an expectation on the consumer's own install.
- SemVer (`MAJOR.MINOR.PATCH`) with `^` / `~` range prefixes controls how much a version is allowed to drift on install/update.
- `package-lock.json` pins exact versions for reproducibility — always commit it; use `npm ci` in CI/automated environments.
- `npx` runs a package's CLI without a permanent global install.
- Yarn and pnpm are npm-compatible alternatives; pnpm in particular optimizes disk usage and dependency isolation.
- Treat every new dependency as a small trust decision; use `npm audit` and review lockfile diffs.

31.11 Exercises

1. Read `examples/31-npm-and-package-management/package.json` and its `README.md`, then write, by hand, what version range `^2.1.0` vs `~2.1.0` would each allow between `2.0.0` and `3.0.0`.
2. Without running `npm install`, explain in your own words the difference in what happens on your machine when a teammate runs `npm install` versus `npm ci` right after cloning a repository.
3. Find (by reading, not installing) one real npm package's `package.json` on `npmjs.com` and identify its `dependencies`, `devDependencies`, and any `peerDependencies`.

Chapter 32 — Testing JavaScript

32.1 Why automated testing matters

Manual testing — clicking around an app, or running a script and eyeballing the output — doesn't scale. It's slow, it's easy to forget a case, and it gives you no protection against *regressions*: a change to file A silently breaking behavior in file B that nobody thought to re-check. Automated tests are code that verifies your code, and unlike a human, they run in milliseconds, every time, exactly the same way, forever. The payoff isn't just "fewer bugs" — it's the confidence to refactor and change code quickly, because a broken assumption gets caught in seconds instead of surfacing as a production incident days later.

32.2 The testing pyramid: unit, integration, end-to-end

Tests are commonly grouped by scope:

- **Unit tests** — test one small, isolated piece of code (a single function, a single class) with everything around it faked or ignored. Fast (milliseconds), numerous, cheap to write, precise about what broke when they fail.
- **Integration tests** — test multiple pieces working together (a function that reads from a real, temporary database; an API route actually calling its real service layer). Slower, fewer of them, catch bugs unit tests structurally cannot see (two correct pieces that don't fit together correctly).
- **End-to-end (E2E) tests** — test the whole system the way a real user would (a browser automation script clicking through a signup flow against a running app). Slowest, most realistic, most brittle (a CSS change can break a test that has nothing to do with styling), typically the fewest in number.

The "pyramid" shape is a common piece of advice: have *many* fast unit tests, a *moderate* number of integration tests, and a *few* critical-path E2E tests — because the fast, cheap tests at the bottom catch most bugs for the least cost, while the slow, expensive tests at the top exist to catch the specific class of bug ("do these pieces actually work together in a real environment") that only they can find.

32.3 Anatomy of a test: Arrange, Act, Assert

Nearly every good test follows the same three-part shape, often called **AAA**:

1. **Arrange** — set up the inputs and any necessary state.
2. **Act** — call the code under test.
3. **Assert** — check that the result matches what you expected.

```
test("adds two positive numbers", () => {
  // Arrange
  const a = 2;
  const b = 3;

  // Act
  const result = add(a, b);

  // Assert
  assert.strictEqual(result, 5);
});
```

A good test name describes *behavior*, not implementation — "adds two positive numbers", not "test1" or "calls add()". When a test fails months from now, its name alone should tell you what broke.

32.4 Test doubles: mocks, stubs, and spies

Unit tests isolate the code under test by replacing its real dependencies with fake ones, collectively called **test doubles**:

- **Stub** — a fake that returns a canned value, replacing a real dependency so the test doesn't depend on it (e.g., a fake "currentTime" that always returns a fixed date, so a test doesn't become flaky based on when it happens to run).
- **Mock** — like a stub, but the test also asserts *how* it was called (was `sendEmail` called exactly once, with this address?). Mocks verify interactions, not just outcomes.
- **Spy** — wraps a *real* function, letting it still run normally while recording how it was called, so you can assert on the calls afterward without changing the function's behavior.

The underlying motivation for all three is the same: a unit test for function A shouldn't fail because function B (which A happens to call) has a bug, is slow, costs money to call (a real payment API), or is non-deterministic (the current time, a random number, the network). Doubles let you test A's logic in isolation.

32.5 Popular frameworks: a conceptual comparison

You'll encounter three names constantly in the JavaScript ecosystem:

Framework	Notes
Jest	Created at Facebook; long the default choice for React and general JS projects. Batteries-included: test runner, assertion library, and mocking all in one, zero-config for most projects.

Framework	Notes
Vitest	Built for projects using Vite; Jest-compatible API, much faster in Vite-based projects because it reuses Vite's module transformation pipeline instead of its own.
Mocha	Older, unopinionated test runner — you pick your own assertion library (often paired with <code>chai</code>) and mocking library (often <code>sinon</code>) separately. Maximum flexibility, more setup.

All three share the same conceptual vocabulary this chapter teaches (`describe / test` or `it` blocks, `expect / assert` style assertions, `beforeEach / afterEach` hooks for setup and teardown) — learning one makes the others easy to pick up. This book uses Node.js's **built-in** test runner for hands-on examples specifically so every example in this repo runs with zero installed dependencies.

32.6 Hands-on: Node's built-in test runner

Since Node.js 18, a test runner and assertion library ship with Node.js itself — no installation required. Import from `node:test` and `node:assert`:

```
// sum.js
export function sum(a, b) {
  return a + b;
}
```

```
// sum.test.js
import test from "node:test";
import assert from "node:assert/strict";
import { sum } from "./sum.js";

test("sum adds two numbers", () => {
  assert.strictEqual(sum(2, 3), 5);
});

test("sum handles negative numbers", () => {
  assert.strictEqual(sum(-1, -1), -2);
});
```

Run it with:

```
node --test examples/32-testing-javascript/
```

`node:test` supports the same conceptual building blocks as Jest/Mocha:

```
import test, { describe, it, beforeEach } from "node:test";
import assert from "node:assert/strict";

describe("a counter", () => {
  let count;

  beforeEach(() => {
    count = 0; // fresh state before every test in this block
  });

  it("starts at zero", () => {
    assert.strictEqual(count, 0);
  });

  it("increments", () => {
    count += 1;
    assert.strictEqual(count, 1);
  });
});
```

`assert/strict` uses strict equality (`===`-like comparisons, and deep strict equality for objects/arrays) — prefer it over the legacy `assert` module, whose default comparisons use the loose `==` semantics from Chapter 4.

32.7 A worked example: red, green, refactor

A common workflow, **Test-Driven Development (TDD)**, writes the test *before* the implementation:

1. **Red** — write a test for behavior that doesn't exist yet; run it, watch it fail (there's nothing to pass yet).
2. **Green** — write the simplest possible code that makes the test pass.
3. **Refactor** — clean up the implementation (and/or the test) with the safety net of a passing test catching any regression.

```
// Step 1 (red): write the test first.
test("clamp restricts a value to a range", () => {
  assert.strictEqual(clamp(15, 0, 10), 10);
  assert.strictEqual(clamp(-5, 0, 10), 0);
  assert.strictEqual(clamp(5, 0, 10), 5);
});
// Running this now fails: ReferenceError: clamp is not defined.

// Step 2 (green): the simplest implementation that passes.
function clamp(value, min, max) {
  if (value < min) return min;
  if (value > max) return max;
  return value;
}
// Running the test now: all three assertions pass.

// Step 3 (refactor): same behavior, using Math.min/Math.max instead.
function clamp(value, min, max) {
  return Math.min(Math.max(value, min), max);
}
// Re-run the test – still passes, confirming the refactor didn't
// change behavior.
```

TDD isn't mandatory for every line of code you write, but the discipline of "can I describe the expected behavior as a test before I write the code" is a genuinely useful design tool even when used selectively.

32.8 Code coverage: a signal, not a goal

Coverage tools (like `c8`, which works with `node --test`) report what percentage of your code's lines/branches were executed while your test suite ran. High coverage numbers feel reassuring, but coverage measures *execution*, not *correctness* — a test that calls a function without asserting anything meaningful about its output "covers" that function while verifying nothing. Use coverage to find code with **no** tests at all (a genuinely useful signal), not as a target to game by inflating a percentage.

32.9 Chapter summary

- Automated tests catch regressions fast and give you the confidence to refactor.
- Unit, integration, and end-to-end tests trade off speed and cost against realism; use many of the fast, cheap ones and fewer of the slow, realistic ones.
- Arrange/Act/Assert is the standard shape of an individual test.
- Stubs, mocks, and spies isolate the code under test from its dependencies.
- Jest, Vitest, and Mocha are the most common frameworks; Node.js also ships a built-in `node:test` + `node:assert` runner requiring zero installation.
- TDD (red, green, refactor) is a workflow where tests are written before the implementation.
- Code coverage tells you what's *untested*, not what's *correct*.

32.10 Exercises

1. Run `node --test examples/32-testing-javascript/` and confirm all tests pass, then intentionally break `sum.js` (e.g., change `+` to `-`) and re-run to see a failing test's output.
2. Using TDD, write a test first for a `isPalindrome(str)` function, watch it fail, then implement the function to make it pass.
3. Identify one place in `examples/30-nodejs-fundamentals.js`'s `EventEmitter` example from Chapter 30 where a spy would let you test "was the `order:placed` listener called with the right item" without relying on `console.log` output.

Chapter 33 — Design Patterns

33.1 What patterns are for

A design pattern is a named, reusable solution to a problem that shows up repeatedly in software design. Patterns aren't a language feature or a library — they're vocabulary. Saying "just use an observer here" instantly communicates a whole design to another engineer who knows the term, the way "checkmate" communicates a complete chess position without describing every piece. The value of learning patterns is almost entirely this shared vocabulary plus recognizing the *shape* of a problem you've already seen solved before — not memorizing rigid recipes to apply everywhere.

That said, patterns have a well-earned reputation for being overused. A pattern applied where the problem doesn't call for it adds indirection and ceremony without adding value — more on this in section 33.7. Learn them so you can *recognize* when one fits, not so you can find an excuse to use one in your next pull request.

33.2 The module pattern

Before ES2015 gave JavaScript real modules (Chapter 24), developers simulated private state and public APIs using closures (Chapter 14) — the **module pattern**:

```
const counterModule = (function () {
  let count = 0; // private – inaccessible from outside this function

  function increment() {
    count += 1;
    return count;
  }

  function reset() {
    count = 0;
  }

  return { increment, reset }; // public API – the only way in
})();

counterModule.increment(); // 1
counterModule.increment(); // 2
console.log(counterModule.count); // undefined – truly private
```

An Immediately Invoked Function Expression (IIFE) creates a scope, runs once, and returns an object exposing only the functions meant to be public. `count` is only reachable through the closures `increment` and `reset` retain over it. Native ES modules (each file already has its own private scope,

with `export` marking what's public) have made the IIFE version largely obsolete for top-level module boundaries, but the *underlying idea* — closures hiding state, exposing a small deliberate API — is still exactly how you'd design a single object or factory-returned instance today.

33.3 Singleton

A singleton ensures a class or module has exactly one instance, shared everywhere it's used — typically for something genuinely global and expensive to duplicate: a database connection pool, an application-wide configuration object, a logging service.

```
class Logger {
  static #instance;

  constructor() {
    if (Logger.#instance) {
      return Logger.#instance; // hand back the existing instance
    }
    this.logs = [];
    Logger.#instance = this;
  }

  log(message) {
    this.logs.push(message);
    console.log(`[LOG] ${message}`);
  }
}

const a = new Logger();
const b = new Logger();
console.log(a === b); // true — same instance
```

In JavaScript specifically, singletons are often achieved even more simply, without a class at all: a module's top-level state is already a singleton, because `import` ing the same module from anywhere gives you the same already-evaluated module object. Reach for the class-based version only when you specifically need the "first call creates it, later calls reuse it" *lazy* initialization behavior shown above.

33.4 Factory

A factory is a function (or method) that creates and returns objects, encapsulating creation logic that the caller shouldn't need to know about — useful when object creation involves branching logic, or when you want to create objects that share a shape without exposing which concrete "subtype" gets built:

```
function createUser(type, name) {
  const base = { name, createdAt: new Date() };

  switch (type) {
    case "admin":
      return { ...base, role: "admin", permissions: ["read", "write", "delete"] };
    case "editor":
      return { ...base, role: "editor", permissions: ["read", "write"] };
    default:
      return { ...base, role: "viewer", permissions: ["read"] };
  }
}

const alice = createUser("admin", "Alice");
const bob = createUser("viewer", "Bob");
```

The caller of `createUser` never needs to know the permission lists or how they're derived from `type` — that logic lives in exactly one place instead of being duplicated at every call site that needs a new user object.

33.5 Observer (and pub/sub)

The observer pattern — introduced conceptually in Chapter 30 alongside Node's `EventEmitter`, and used constantly in the DOM (Chapter 28) — lets one object (the "subject") notify a list of dependents (the "observers") whenever something happens, without the subject knowing anything about who its observers are.

```

class Subject {
  #observers = [];

  subscribe(observerFn) {
    this.#observers.push(observerFn);
    return () => { // returns an "unsubscribe" function
      this.#observers = this.#observers.filter((fn) => fn !== observerFn);
    };
  }

  notify(data) {
    for (const observerFn of this.#observers) {
      observerFn(data);
    }
  }
}

const priceFeed = new Subject();

const unsubscribe = priceFeed.subscribe((price) => {
  console.log(`Price updated: ${price}`);
});

priceFeed.notify(42.5); // Price updated: $42.5
unsubscribe();
priceFeed.notify(43.0); // (nothing – listener was removed)

```

Observer vs. pub/sub — a subtle but real distinction: in the classic observer pattern, the subject holds direct references to its observers and calls them directly, as above. In a **publish/subscribe** system, publishers and subscribers don't reference each other at all — they both talk to an intermediary (a message broker, an event bus, or in the browser, something like `window` as a shared event target) that routes messages between them. Node's `EventEmitter` sits close to pure observer (you call `.on()` directly on the specific emitter instance you care about); a message queue like Redis pub/sub or a browser `CustomEvent` dispatched on a shared bus is closer to true pub/sub, where publishers and subscribers can be added or removed without either side knowing the other exists.

33.6 Decorator (the JavaScript-idiomatic way)

The decorator pattern wraps an object or function to add behavior without modifying its original source. JavaScript's first-class functions make this especially natural — no special language feature required (though a real `@decorator` syntax for classes is also progressing through TC39):

```
function withLogging(fn) {
  return function (...args) {
    console.log(`Calling ${fn.name} with`, args);
    const result = fn(...args);
    console.log(`${fn.name} returned`, result);
    return result;
  };
}

function add(a, b) {
  return a + b;
}

const loggedAdd = withLogging(add);
loggedAdd(2, 3);
// Calling add with [ 2, 3 ]
// add returned 5
```

`loggedAdd` behaves exactly like `add`, plus logging — `add` itself is untouched, and you can compose multiple decorators (logging, timing, caching) by wrapping a function in more than one at once. This is precisely how many real-world tools work: Express middleware, React higher-order components, and Redux middleware are all decorators by another name.

33.7 Strategy

The strategy pattern defines a family of interchangeable algorithms and lets the caller pick which one to use at runtime, instead of hard-coding a single algorithm (or a long `if / else` chain choosing between them) inside the consuming code:

```
const strategies = {
  cheapest: (options) => options.sort((a, b) => a.price - b.price)[0],
  fastest: (options) => options.sort((a, b) => a.etaMinutes - b.etaMinutes)[0],
  greenest: (options) => options.sort((a, b) => a.co2Grams - b.co2Grams)[0],
};

function chooseShippingOption(options, strategyName) {
  const strategy = strategies[strategyName] ?? strategies.cheapest;
  return strategy(options);
}
```

Because strategies here are just functions stored in an object, "picking a strategy" is just looking up a key — no class hierarchy required. This is a recurring theme in JavaScript: many classic object-oriented patterns, originally described in a language (like C++ or Java) with mandatory classes and no first-class functions, become simpler in JavaScript because functions and objects can carry the same weight a whole class would in those languages.

33.8 Don't over-engineer: patterns are tools, not goals

Every pattern in this chapter adds a layer of indirection in exchange for some benefit (decoupling, flexibility, testability). That trade is worth making when the benefit is real and needed *now* — not because a problem *could theoretically* need that flexibility someday. Signs a pattern is being reached for reflexively rather than usefully:

- A "factory" that only ever constructs one kind of object, with no branching logic — it's just a constructor with extra steps.
- A "strategy" object with exactly one strategy that never changes — it's just a function.
- An "observer" system built for a UI element that only ever has one listener — a plain callback would do.
- A singleton used for something that isn't actually global (most classes that "could" be a singleton don't need to be).

The best JavaScript engineers reach for patterns *after* noticing the same shape of problem recur, not before writing the first version of something. When in doubt, write the plain, direct version first; reach for the named pattern once the code is telling you, through actual duplication or actual coupling pain, that it wants one.

33.9 Chapter summary

- Patterns are shared vocabulary for recurring problem shapes, not mandatory scaffolding.
- The module pattern uses closures to hide private state behind a small public API — largely superseded by ES modules for top-level code, but the underlying idea remains everywhere.
- Singletons guarantee one shared instance; in JavaScript, a module's top-level state is often already a singleton without any extra code.
- Factories centralize object-creation branching logic in one place.
- Observer lets a subject notify subscribed listeners without knowing who they are; pub/sub goes further, routing through a shared intermediary so publishers and subscribers never reference each other.
- Decorators wrap functions/objects to add behavior without modifying the original — natural in JavaScript thanks to first-class functions.
- Strategy swaps interchangeable algorithms via a lookup instead of a branching chain.
- Apply patterns when a problem's shape calls for them, not by default.

33.10 Exercises

1. Refactor the `withLogging` decorator into a `withTiming` decorator that logs how long `fn` took to run (see `console.time / console.timeEnd`, also used in Chapter 34), then compose both decorators around the same function.
2. Take the `Subject` /observer example and rewrite it as a small pub/sub "event bus" object that many unrelated publishers and subscribers could share, instead of each subject managing its own observer list.
3. Find one place in a personal or work project where a `switch` statement choosing behavior could be replaced with the strategy pattern's lookup-object approach, and judge honestly whether doing so would actually improve the code or just add indirection.

Chapter 34 — Performance and Best Practices

34.1 Readability is a performance concern too

"Performance" usually means "how fast does this run," but there's a second kind of performance that determines a codebase's long-term health: how fast can a human understand and safely change this code? Code that's clever but opaque costs every future reader (including you, in six months) real time and real risk of introducing bugs while modifying something they don't fully understand. Most of the advice in this chapter optimizes for both kinds of performance at once, because they're rarely in tension — clear code is usually also easy to make fast, because you can actually see what it's doing.

34.2 Avoid premature optimization

Donald Knuth's famous line — "premature optimization is the root of all evil" — is frequently misquoted as "never optimize," which isn't the point. The point is *sequencing*: write clear, correct code first; measure where the actual bottleneck is (using `console.time / console.timeEnd`, a profiler, or a benchmarking tool); *then* optimize the specific hot path that measurement identified. Optimizing code that isn't actually a bottleneck makes it harder to read for zero real-world benefit — the classic trap is spending an afternoon shaving microseconds off a function that runs once at startup, while a genuinely slow database query three files away goes unnoticed.

```
console.time("expensive operation");
const result = doExpensiveThing();
console.timeEnd("expensive operation"); // "expensive operation: 842.315ms"
```

Measure first. Guessing where code is slow is wrong more often than intuition suggests, even for experienced engineers.

34.3 Common real performance issues

A few patterns show up repeatedly in real, measurable slowdowns:

- **Unnecessary work inside loops (N+1-style problems).** Fetching data one item at a time inside a loop — a network request, a database query, or even a repeated expensive DOM query — instead of fetching everything needed in one batched call before the loop starts. This is the single most common source of real-world slowness in web applications, and it scales *badly*: a page that's fine with 10 items becomes unusable at 10,000.

```
// Slow: one query per user, N+1 total round trips.
for (const id of userIds) {
  const user = await fetchUser(id);
  results.push(user);
}

// Fast: one batched round trip.
const results = await fetchUsers(userIds);
```

- **Unbounded closures and memory leaks.** A closure (Chapter 14) that captures a large object and is itself stored somewhere long-lived (a global array, an event listener never removed) keeps that object alive in memory forever, even after it's logically no longer needed. In browsers, a very common leak is an event listener attached to a DOM node that never gets removed even after the node is removed from the page — the listener's closure keeps the whole node (and anything it closed over) alive.

```
function attachHandler(largeDataset) {
  button.addEventListener("click", () => {
    console.log(largeDataset.length); // keeps largeDataset alive forever
  });
}
// If `button` is later removed from the page but the listener was never
// removed, `largeDataset` still can't be garbage collected.
```

The fix is usually explicit cleanup: `removeEventListener` when a component/element is torn down, or using `AbortController` (Chapter 28) to remove multiple listeners at once.

- **Unnecessary re-computation.** Re-deriving the same expensive result every time it's needed instead of computing it once and reusing it (see memoization, section 34.5, and the discussion of UI frameworks re-rendering more than necessary when state changes they don't actually depend on — a conceptual cousin of this same problem, though the fix in that case is usually framework-specific, like React's `useMemo` or `React.memo`).

34.4 Debouncing and throttling

Both techniques limit how often a function runs in response to high-frequency events (typing, scrolling, resizing, mouse movement) — but they solve slightly different problems.

Debounce: wait for a pause in activity before running — useful for a search box that shouldn't fire an API call on every single keystroke, only once the user stops typing:

```
function debounce(fn, delayMs) {
  let timeoutId;
  return function (...args) {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => fn.apply(this, args), delayMs);
  };
}

const debouncedSearch = debounce((query) => {
  console.log(`Searching for: ${query}`);
}, 300);

// Rapid calls...
debouncedSearch("h");
debouncedSearch("he");
debouncedSearch("hel");
debouncedSearch("hell");
debouncedSearch("hello");
// ...only the LAST call actually runs, 300ms after the last keystroke.
```

Throttle: run at most once per fixed interval, no matter how often the event fires — useful for a scroll handler that needs to react continuously, but not on literally every pixel of scroll movement:

```
function throttle(fn, intervalMs) {
  let isWaiting = false;
  return function (...args) {
    if (isWaiting) return;
    fn.apply(this, args);
    isWaiting = true;
    setTimeout(() => { isWaiting = false; }, intervalMs);
  };
}

const throttledLog = throttle(() => {
  console.log("Scroll position handled");
}, 200);

// Even if this fires 100 times a second, throttledLog's body runs
// at most once every 200ms.
```

The rule of thumb: **debounce** when you only care about the *final* state after activity settles (search-as-you-type, form validation, window resize handling); **throttle** when you need *regular* updates throughout continuous activity (scroll position, drag events, mousemove-driven effects).

34.5 Memoization, revisited

Chapter 19 introduced memoization as a functional-programming technique; here it is again explicitly as a performance tool. Memoization caches a pure function's return value by its input, so repeated calls with the same arguments skip the computation entirely:

```
function memoize(fn) {
  const cache = new Map();
  return function (n) {
    if (cache.has(n)) return cache.get(n);
    const result = fn(n);
    cache.set(n, result);
    return result;
  };
}

function fibonacci(n) {
  if (n <= 1) return n;
  return fibonacci(n - 1) + fibonacci(n - 2);
}

const memoFibonacci = memoize(function (n) {
  if (n <= 1) return n;
  return memoFibonacci(n - 1) + memoFibonacci(n - 2);
});
```

Naive recursive `fibonacci(40)` re-computes the same sub-results millions of times (exponential time complexity); the memoized version computes each unique input exactly once (linear time), trading a small amount of memory for an enormous amount of repeated work avoided. `examples/34-performance-and-best-practices.js` benchmarks both versions directly with `console.time` so you can see the difference in real milliseconds, not just theory.

Memoization only works correctly for **pure functions** — same input always produces the same output, with no dependency on external mutable state. Memoizing a function that depends on the current time, random numbers, or external state that can change between calls will silently return stale, wrong results.

34.6 `const` by default, and avoiding globals

Two small habits with outsized benefit to both readability and correctness (both covered in depth in Chapter 3 and Chapter 14, restated here as performance-adjacent style rules):

- **Default to `const` ; use `let` only when a binding genuinely needs reassignment; avoid `var` entirely in new code.** A `const` binding tells every future reader "this never changes" without them having to trace the code to verify it — that's a real cognitive performance win, and it also lets the engine make certain assumptions during optimization.
- **Avoid global variables.** Every global is a piece of shared mutable state any part of the program (including third-party code and browser extensions, in a browser context) can read or overwrite unexpectedly, making bugs dependent on *execution order* rather than local reasoning. Prefer module-scoped state (Chapter 24) or function parameters/closures over attaching data to `window/global`.

34.7 Linting and formatting tools

Style consistency stops being a matter of taste once a codebase has more than one contributor — tools automate it so nobody has to argue about it in code review:

- **ESLint** — analyzes your code for both style issues and *actual bugs* (unused variables, unreachable code, missing `await`, accidental `==` instead of `===`) via configurable rules.
- **Prettier** — an opinionated code *formatter* (indentation, quote style, line length) that removes formatting from human judgment entirely — you save a file, it's reformatted, the debate is over.

Most professional JavaScript projects run both: Prettier for formatting, ESLint for everything formatting can't catch (real bugs and code-quality rules), often wired into a pre-commit hook or CI check so violations never reach the main branch.

34.8 A JavaScript style checklist

- **Name things by what they represent**, not by their type or how they're used (`activeUsers`, not `arr` or `data2`).
- **Keep functions short and focused on one job**. If you struggle to name a function without using "and," it's probably doing two things and should be two functions.
- **Prefer early returns over deep nesting**.

```
// Nested – harder to follow the happy path.
function processOrder(order) {
  if (order) {
    if (order.items.length > 0) {
      if (order.paid) {
        return ship(order);
      }
    }
  }
  return null;
}

// Early returns – the happy path reads top to bottom.
function processOrder(order) {
  if (!order) return null;
  if (order.items.length === 0) return null;
  if (!order.paid) return null;
  return ship(order);
}
```

- **Avoid deep nesting generally** (more than 2-3 levels of indentation is a signal to extract a helper function).

- **Don't mutate function parameters** unless that's the function's explicit, documented purpose — surprise mutation is a classic source of action-at-a-distance bugs.
- **Handle errors where you have enough context to do something useful** with them (see Chapter 18) — don't swallow errors silently, and don't catch errors you have no meaningful response to just to make a warning disappear.

34.9 Chapter summary

- Readable code is a performance win for the humans maintaining it, not just a nicety.
- Measure before optimizing; don't guess where the bottleneck is.
- The most common real-world slowdowns are unbatched work inside loops and memory leaks from un-cleaned-up closures/listeners.
- Debounce waits for a pause before acting (search boxes); throttle limits action to a fixed rate during continuous activity (scroll handlers).
- Memoization trades memory for skipped recomputation, but only works correctly for pure functions.
- Default to `const`, avoid `var`, and minimize global state.
- ESLint (bugs and style rules) and Prettier (formatting) are standard tooling in professional JavaScript codebases.

34.10 Exercises

1. Run `examples/34-performance-and-best-practices.js` and compare the `console.time` output for naive vs. memoized `fibonacci(35)`. Try `fibonacci(40)` naively and predict what happens to the timing before running it (consider stopping it if it takes too long — that's the point).
2. Take the deeply nested `processOrder` example and identify one more real function (from a personal project, or one you've seen) that could be flattened using early returns.
3. Implement a `throttle`-based scroll position logger and a `debounce`-based search handler in the same file, and describe in one sentence each why the *other* technique would be the wrong choice for that use case.

Chapter 35 — Building Real Projects

Every previous chapter taught one concept in isolation. Real software requires combining dozens of them into something that actually works end to end. This chapter builds two complete, working projects from scratch, narrating the design decisions along the way — not just the final code, but *why* it ended up looking that way. Both projects deliberately avoid any external dependency, so everything here runs with the Node.js and browser built-ins you already have.

35.1 Project 1: a command-line Todo app

35.1.1 What we're building

A CLI tool, run with `node todo.js <command> [args]`, that:

- `add "<text>"` — adds a new todo item
- `list` — prints all todos, showing completion status
- `complete <id>` — marks a todo as done
- `remove <id>` — deletes a todo
- persists everything to a JSON file on disk, so todos survive between runs

```
$ node todo.js add "Buy milk"
Added #1: Buy milk

$ node todo.js add "Write chapter 35"
Added #2: Write chapter 35

$ node todo.js list
[ ] #1 Buy milk
[ ] #2 Write chapter 35

$ node todo.js complete 1
Completed #1: Buy milk

$ node todo.js list
[x] #1 Buy milk
[ ] #2 Write chapter 35

$ node todo.js remove 2
Removed #2: Write chapter 35
```

35.1.2 Design decisions

Storage: a single JSON file, read and rewritten on every command. A real production task manager would use a database, but for a small CLI tool run occasionally by one person, a JSON file is simpler, human-inspectable, and requires zero setup — exactly the trade-off that matters for a project this size. This is `fs` (Chapter 30) doing the heavy lifting: `readFileSync` / `writeFileSync` for the whole file, since a todo list is small enough that loading it entirely into memory on every run is not a real performance concern (see Chapter 34: measure before optimizing — for data this small, there is nothing to measure).

Argument parsing: `process.argv`, by hand, no dependency. Real-world CLI tools often use a library (`commander` , `yargs`) for argument parsing with flags, help text, and validation. For four commands and simple positional arguments, hand-rolling it keeps the example dependency-free and makes the entire mechanism visible rather than hidden behind a library API you'd have to look up.

IDs: a simple incrementing counter, computed from the current max ID. No UUID library needed — the todos array itself is the source of truth for "what's the next available ID."

Structure: one file, organized top-to-bottom as data layer → command handlers → dispatcher. A larger CLI tool would split these into separate modules (Chapter 24), but at this size, splitting would add navigation overhead (jumping between files) without adding clarity.

35.1.3 Walking through the code

The full, runnable source is in `examples/projects/cli-todo/todo.js`. The key pieces:

Reading and writing the data file, defaulting to an empty list if the file doesn't exist yet (first run):

```
function loadTodos() {
  if (!existsSync(DATA_FILE)) return [];
  const raw = readFileSync(DATA_FILE, "utf8");
  return JSON.parse(raw);
}

function saveTodos(todos) {
  writeFileSync(DATA_FILE, JSON.stringify(todos, null, 2), "utf8");
}
```

`JSON.stringify(todos, null, 2)` (Chapter 25) pretty-prints the file with 2-space indentation — a small touch that makes the data file readable if you ever open it directly, at essentially no cost.

Command handlers, each a small pure-ish function taking the current todos array and the relevant arguments, returning the updated array:

```
function addTodo(todos, text) {
  const nextId = todos.reduce((max, t) => Math.max(max, t.id), 0) + 1;
  const todo = { id: nextId, text, done: false };
  console.log(`Added #${todo.id}: ${todo.text}`);
  return [...todos, todo];
}

function completeTodo(todos, id) {
  const numericId = Number(id);
  const todo = todos.find((t) => t.id === numericId);
  if (!todo) {
    console.log(`No todo with id ${id}`);
    return todos;
  }
  todo.done = true;
  console.log(`Completed #${todo.id}: ${todo.text}`);
  return todos;
}
```

The dispatcher, mapping the first CLI argument to a handler — this is the strategy pattern from Chapter 33, applied to command-line commands instead of shipping options:

```
const [, , command, ...args] = process.argv;
let todos = loadTodos();

switch (command) {
  case "add":
    todos = addTodo(todos, args.join(" "));
    break;
  case "list":
    listTodos(todos);
    break;
  case "complete":
    todos = completeTodo(todos, args[0]);
    break;
  case "remove":
    todos = removeTodo(todos, args[0]);
    break;
  default:
    printUsage();
    process.exit(1);
}

saveTodos(todos);
```

Note `list` is the one command that doesn't need to save anything (it only reads) — but calling `saveTodos` after every command unconditionally is harmless (it just rewrites the same data) and keeps the dispatcher uniform, which is a reasonable simplicity/purity trade-off for a project this size.

35.1.4 What you'd add for production use

This CLI intentionally stays minimal. Real-world hardening would include: input validation (what if `add` is called with no text?), a `--help` flag with real usage text, colored terminal output, and tests (Chapter 32) for each command handler — all straightforward extensions of exactly the functions already shown, not a redesign.

35.2 Project 2: an in-browser calculator

35.2.1 What we're building

A working four-function calculator (add, subtract, multiply, divide) with a clickable button grid, a running display, a clear button, and full keyboard support — a single `index.html` file with embedded CSS and JavaScript, no build step, no dependencies. Open it directly in a browser.

35.2.2 Design decisions

One HTML file, not a bundled app. Real front-end projects almost always use a framework and a build tool (Chapter 27 discusses the DOM these frameworks sit on top of), but a calculator's entire UI is small enough that a plain HTML file with a `<style>` and `<script>` tag is the simplest thing that could possibly work — and it demonstrates the raw DOM APIs from Chapters 27-28 directly, without a framework's abstractions in the way.

State: a small plain object, not scattered variables. Rather than tracking the calculator's state as several independent top-level variables (`currentValue`, `previousValue`, `operator`, ...), the example keeps a single `state` object. This mirrors the "single source of truth" principle that UI frameworks like React formalize — even without a framework, keeping all your mutable state in one identifiable place makes the program easier to reason about, since there's exactly one thing to inspect to answer "what is this app currently doing."

Rendering: one `render()` function, called after every state change, rather than manually updating the display text in ten different places. Any time a button handler changes `state`, it calls `render()` once at the end. This is a small-scale version of the same idea that makes declarative UI frameworks predictable: describe *what the UI should show given the current state*, and call that description function every time the state changes, instead of hand-tracking every individual DOM mutation.

Keyboard support via a single `keydown` listener on `document`, mapped to the same handler functions the buttons call — so there is exactly one implementation of "what happens when you press equals," reachable from either a mouse click or an Enter key press (Chapter 28 covers event delegation and listener design in depth).

35.2.3 Walking through the code

The full source is in `examples/projects/calculator/index.html`. The state and core operation logic:

```
const state = {
  display: "0",
  firstOperand: null,
  operator: null,
  waitingForSecondOperand: false,
};

function inputDigit(digit) {
  if (state.waitingForSecondOperand) {
    state.display = digit;
    state.waitingForSecondOperand = false;
  } else {
    state.display = state.display === "0" ? digit : state.display + digit;
  }
}

function handleOperator(nextOperator) {
  const inputValue = parseFloat(state.display);

  if (state.operator && state.waitingForSecondOperand) {
    state.operator = nextOperator; // just changing operator, e.g. + then *
    return;
  }

  if (state.firstOperand === null) {
    state.firstOperand = inputValue;
  } else if (state.operator) {
    const result = calculate(state.firstOperand, inputValue, state.operator);
    state.display = String(result);
    state.firstOperand = result;
  }

  state.waitingForSecondOperand = true;
  state.operator = nextOperator;
}

function calculate(first, second, operator) {
  switch (operator) {
    case "+": return first + second;
    case "-": return first - second;
    case "x": return first * second;
    case "÷": return second === 0 ? NaN : first / second;
    default: return second;
  }
}
```

This `firstOperand / operator / waitingForSecondOperand` state machine is the classic shape of a four-function calculator's logic — every button press is really just a state transition, and `calculate` (a small, pure function, easy to unit test per Chapter 32) is the only place arithmetic actually happens.

Notice `÷` explicitly guards division by zero rather than silently displaying `Infinity`, a small correctness decision that's easy to skip and easy to regret.

Divide-by-zero and `NaN` results are rendered as `"Error"` in `render()` rather than raw `NaN / Infinity` text, which is a deliberate choice about what a *user* should see versus what the *program's internal state* is — worth noticing as a general UI principle: internal state and displayed state don't have to be the same string.

35.2.4 Keyboard support

```
document.addEventListener("keydown", (event) => {
  if (event.key >= "0" && event.key <= "9") inputDigit(event.key);
  else if (event.key === ".") inputDecimal();
  else if (["+","-"].includes(event.key)) handleOperator(event.key);
  else if (event.key === "*") handleOperator("×");
  else if (event.key === "/") { event.preventDefault(); handleOperator("÷"); }
  else if (event.key === "Enter" || event.key === "=") handleEquals();
  else if (event.key === "Escape") resetCalculator();
  else if (event.key === "Backspace") backspace();
  else return; // don't re-render on keys we don't handle

  render();
});
```

`event.preventDefault()` on `/` stops the browser's default "quick find" behavior some browsers bind to that key; returning early (and skipping `render()`) for unhandled keys avoids unnecessary re-renders — a tiny, concrete instance of the "don't do unnecessary work" performance principle from Chapter 34, applied at UI scale rather than algorithmic scale.

35.2.5 Trying it

Open `examples/projects/calculator/index.html` directly in any browser (no server required — it's a static file) and try both clicking buttons and typing on your keyboard; both paths go through the exact same state- transition functions.

35.3 What these two projects have in common

Despite targeting completely different environments (a terminal vs. a browser tab), both projects follow the same shape, which is worth noticing explicitly: **a clear separation between state, the logic that changes state, and the code that displays state**. The CLI todo app reads JSON into an array, runs a pure-ish handler function against it, and writes the result back out. The calculator keeps a state object, runs handler functions that mutate it in response to input, and calls one `render()` function to reflect it

on screen. This "state in, transformation, state out, then display" shape scales — it's the same underlying structure behind Redux, React's `useState`, and most database-backed web applications you'll build professionally. Small projects are where this habit is cheapest to build.

35.4 Chapter summary

- The CLI todo app demonstrates `fs` for persistence, hand-rolled `process.argv` parsing, and the strategy/dispatcher pattern for routing commands — all dependency-free.
- The browser calculator demonstrates DOM manipulation, event handling (mouse and keyboard, unified through shared handler functions), and a single-source-of-truth state object rendered by one `render()` function.
- Both projects separate "state," "logic that changes state," and "display of state" into distinct, clearly named pieces — a structure that scales far beyond these two small examples.
- Neither project needed a single external dependency; the built-in platform (Node.js's `fs` / `process`, the browser's DOM/event APIs) was sufficient for the whole thing.

35.5 Exercises

1. Extend the CLI todo app with an `edit <id> "<new text>"` command that changes a todo's text in place.
2. Add a `priority` field to each todo (`low` / `medium` / `high`, defaulting to `medium`) and update `list` to sort by priority.
3. Add a percentage (`%`) button to the calculator that converts the current display value to a percentage of the first operand (a common real calculator feature), and wire it into both the button grid and the keyboard handler.
4. Add a `node --test`-based test file (Chapter 32) for the calculator's `calculate` function, covering all four operators and the divide-by-zero case, by extracting `calculate` into its own module the HTML file can `import`.

Chapter 36 — Appendix: Cheat Sheet

A dense, scannable reference for everything covered in this book. Use it to jog your memory, not to learn a concept for the first time — each topic here is explained properly in its own chapter.

36.1 Array method quick reference

Method	Mutates?	Returns	Purpose
<code>push(x)</code> / <code>pop()</code>	Yes	new length / removed item	Add/remove at the end
<code>unshift(x)</code> / <code>shift()</code>	Yes	new length / removed item	Add/remove at the start
<code>splice(start, count, ...items)</code>	Yes	removed items	Insert/remove/replace anywhere
<code>sort(compareFn)</code>	Yes	the array	Sort in place (default: string order!)
<code>reverse()</code>	Yes	the array	Reverse in place
<code>fill(value)</code>	Yes	the array	Fill all/part with a value
<code>map(fn)</code>	No	new array	Transform every item
<code>filter(fn)</code>	No	new array	Keep items matching a condition
<code>reduce(fn, init)</code>	No	any accumulated value	Fold the array into one value
<code>find(fn)</code> / <code>findIndex(fn)</code>	No	item / index	First match, or <code>undefined</code> / <code>-1</code>
<code>findLast(fn)</code> / <code>findLastIndex(fn)</code>	No	item / index	Last match
<code>some(fn)</code> / <code>every(fn)</code>	No	boolean	Does any/every item match?
<code>includes(x)</code>	No	boolean	Does the array contain <code>x</code> ?
<code>indexOf(x)</code> / <code>lastIndexOf(x)</code>	No	index or <code>-1</code>	Find by strict equality
<code>slice(start, end)</code>	No	new array	Extract a portion (non-destructive)
<code>concat(...arrs)</code>	No	new array	Combine arrays
<code>join(sep)</code>	No	string	Combine into a delimited string
<code>flat(depth)</code>	No	new array	Flatten nested arrays
<code>flatMap(fn)</code>	No	new array	<code>map</code> then <code>flat(1)</code>

Method	Mutates?	Returns	Purpose
<code>at(index)</code>	No	item	Supports negative indices (<code>at(-1)</code>)
<code>Array.from(iterable)</code>	—	new array	Build an array from any iterable/array-like
<code>Array.isArray(x)</code>	—	boolean	Reliable array type check

36.2 String method quick reference

Method	Purpose
<code>slice(start, end)</code>	Extract a substring (accepts negative indices)
<code>substring(start, end)</code>	Like <code>slice</code> but clamps negatives to 0
<code>split(sep)</code>	String → array
<code>trim()</code> / <code>trimStart()</code> / <code>trimEnd()</code>	Remove whitespace
<code>toUpperCase()</code> / <code>toLowerCase()</code>	Case conversion
<code>includes(sub)</code> / <code>startsWith(sub)</code> / <code>endsWith(sub)</code>	Substring checks
<code>indexOf(sub)</code> / <code>lastIndexOf(sub)</code>	Find a substring's position
<code>replace(pattern, repl)</code> / <code>replaceAll(pattern, repl)</code>	Substitute text
<code>padStart(len, ch)</code> / <code>padEnd(len, ch)</code>	Pad to a fixed length
<code>repeat(n)</code>	Repeat the string
<code>concat(...strs)</code>	Combine strings (template literals are usually clearer)
<code>charAt(i)</code> / <code>[i]</code>	Get one character
<code>codePointAt(i)</code>	Unicode-aware char code
<code>match(regex)</code> / <code>matchAll(regex)</code>	Regex matching
<code>at(index)</code>	Supports negative indices

36.3 Equality: `==` vs `===`

Comparison	<code>==</code> (loose)	<code>===</code> (strict)
<code>1 == "1"</code>	<code>true</code> (coerces)	<code>false</code>
<code>0 == false</code>	<code>true</code> (coerces)	<code>false</code>

Comparison	== (loose)	=== (strict)
<code>null == undefined</code>	<code>true</code> (special case)	<code>false</code>
<code>NaN == NaN</code>	<code>false</code>	<code>false</code> (use <code>Number.isNaN()</code>)
<code>[] == false</code>	<code>true</code> (coerces both sides)	<code>false</code>
<code>{ } === { }</code>	<code>false</code> (different references)	<code>false</code>

Rule: always use `===` / `!==` unless you have a specific, deliberate reason to allow coercion (the most common legitimate case is checking for both `null` and `undefined` at once with `x == null`).

36.4 Type coercion quick table

Value	Number(value)	String(value)	Boolean(value)
<code>""</code>	<code>0</code>	<code>""</code>	<code>false</code>
<code>"0"</code>	<code>0</code>	<code>"0"</code>	<code>true</code> (non-empty string!)
<code>" "</code>	<code>0</code>	<code>" "</code>	<code>true</code>
<code>0</code>	<code>0</code>	<code>"0"</code>	<code>false</code>
<code>null</code>	<code>0</code>	<code>"null"</code>	<code>false</code>
<code>undefined</code>	<code>NaN</code>	<code>"undefined"</code>	<code>false</code>
<code>[]</code>	<code>0</code>	<code>""</code>	<code>true</code>
<code>[5]</code>	<code>5</code>	<code>"5"</code>	<code>true</code>
<code>[1,2]</code>	<code>NaN</code>	<code>"1,2"</code>	<code>true</code>
<code>{ }</code>	<code>NaN</code>	<code>"[object Object]"</code>	<code>true</code>
<code>NaN</code>	<code>NaN</code>	<code>"NaN"</code>	<code>false</code>

Every value is truthy **except**: `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, and `NaN` — memorize this exact list ("the falsy eight") since everything else, including `[]` and `{ }`, is truthy.

36.5 Async patterns quick reference

```
// Callback (oldest style)
fs.readFile("file.txt", "utf8", (err, data) => { ... });

// Promise
fetch(url).then((res) => res.json()).then((data) => { ... }).catch((err) => { ... });

// async/await (built on promises, reads like sync code)
async function load() {
  try {
    const res = await fetch(url);
    const data = await res.json();
  } catch (err) { ... }
}

// Run promises concurrently, wait for all
const [a, b] = await Promise.all([fetchA(), fetchB()]);

// Run concurrently, don't fail fast on one rejection
const results = await Promise.allSettled([fetchA(), fetchB()]);

// First to settle wins
const first = await Promise.race([fetchA(), timeout(3000)]);

// First to FULFILL wins (ignores rejections unless all reject)
const firstOk = await Promise.any([fetchA(), fetchB()]);
```

36.6 Common `Object` static methods

Method	Purpose
<code>Object.keys(obj)</code>	Array of own enumerable keys
<code>Object.values(obj)</code>	Array of own enumerable values
<code>Object.entries(obj)</code>	Array of <code>[key, value]</code> pairs
<code>Object.fromEntries(pairs)</code>	Build an object from <code>[key, value]</code> pairs
<code>Object.assign(target, ...sources)</code>	Shallow-merge sources into target
<code>Object.freeze(obj)</code>	Prevent all mutation (shallow)
<code>Object.isFrozen(obj)</code>	Check freeze status
<code>Object.create(proto)</code>	Create an object with a given prototype
<code>Object.getPrototypeOf(obj)</code>	Read an object's prototype
<code>Object.defineProperty(obj, key, desc)</code>	Fine-grained property control

36.7 Common `Math` and `Number` statics

Expression	Purpose
<code>Math.max(...nums) / Math.min(...nums)</code>	Largest/smallest
<code>Math.round(x) / Math.floor(x) / Math.ceil(x) / Math.trunc(x)</code>	Rounding
<code>Math.random()</code>	Float in <code>[0, 1)</code>
<code>Math.abs(x) / Math.sign(x)</code>	Absolute value / sign
<code>Math.pow(x, y) / x ** y</code>	Exponentiation
<code>Number.isInteger(x)</code>	True integer check
<code>Number.isNaN(x)</code>	Reliable NaN check (unlike global <code>isNaN</code>)
<code>Number.parseFloat(s) / Number.parseInt(s, radix)</code>	String → number, always pass a radix to <code>parseInt</code>
<code>Number.MAX_SAFE_INTEGER</code>	$2^{53} - 1$, the largest exactly-representable integer

36.8 Common array/object idioms

```
// Deduplicate an array
const unique = [...new Set(array)];

// Shallow copy an array or object
const arrCopy = [...array];
const objCopy = { ...obj };

// Deep clone (structured data, no functions/DOM nodes)
const deepCopy = structuredClone(obj);

// Merge objects, later keys win
const merged = { ...defaults, ...overrides };

// Destructure with defaults and renaming
const { name: userName = "Anonymous", age } = user;

// Group array items into an object of arrays
const byRole = array.reduce((acc, item) => {
  (acc[item.role] ??= []).push(item);
  return acc;
}, {});

// Safe nested access
const city = user?.address?.city ?? "Unknown";

// Convert array-like/iterable to a real array
const chars = Array.from("hello");
```

36.9 Glossary

- **Closure** — a function that "remembers" the variables from the scope it was created in, even after that outer scope has finished executing.
- **Hoisting** — `var` and function declarations are conceptually moved to the top of their scope before code runs; `let` / `const` are hoisted too but remain in a "temporal dead zone" until their declaration line runs.
- **Prototype** — the object a given object automatically delegates property/method lookups to when a property isn't found on the object itself.
- **Event loop** — the mechanism that lets single-threaded JavaScript handle asynchronous work: it repeatedly checks whether the call stack is empty and, if so, pushes the next queued callback onto it.
- **Microtask / macrotask** — promise callbacks (microtasks) run before timer/I/O callbacks (macrotasks) at each turn of the event loop.
- **Promise** — an object representing the eventual result (or failure) of an asynchronous operation, in one of three states: pending, fulfilled, or rejected.
- **Callback** — a function passed into another function to be called later, typically once some operation completes.
- **this** — a keyword whose value is determined by *how a function is called*, not where it's defined (except for arrow functions, which inherit `this` from their enclosing scope).
- **Truthy / falsy** — how a non-boolean value behaves in a boolean context (an `if` condition); see the falsy list in section 36.4.
- **Pure function** — a function whose output depends only on its inputs, with no side effects and no reliance on external mutable state.
- **Immutability** — the practice of not modifying data in place, instead producing new values (`[...arr, x]` instead of `arr.push(x)`).
- **Module** — a file with its own private scope, exposing selected bindings via `export`, consumed elsewhere via `import`.
- **Coercion** — automatic conversion between types, e.g. `"5" - 1` coerces `"5"` to the number `5`.
- **Event delegation** — attaching one listener to a parent element instead of many listeners to its children, using event bubbling to handle events from any of them.
- **Debounce / throttle** — rate-limiting techniques for high-frequency events; debounce waits for a pause, throttle enforces a fixed interval.
- **Memoization** — caching a pure function's results by input to avoid recomputation.

36.10 Further learning

This book is an end-to-end tour, not the final word on any single topic. For deeper dives:

- **developer.mozilla.org (MDN Web Docs)** — the canonical, most accurate reference for every JavaScript, DOM, and Web API method mentioned in this book.
- **javascript.info** — a free, thorough, example-heavy online course covering the language and browser APIs in more depth than any single chapter here.
- **tc39.es** — the official home of the ECMAScript specification and the proposals process; browse it to see what's coming in future JavaScript versions and read the exact rationale behind language decisions.
- **The "You Don't Know JS" book series** (Kyle Simpson) — a deep, spec-adjacent exploration of the language's trickier corners: scope, closures, `this`, prototypes, and `async` — a natural next step after this book for readers who want to go even deeper on the "engine room" material from Part III.
- **Node.js's own official documentation** — the authoritative reference for every built-in module used throughout Part VI (`fs` , `http` , `events` , `path` , `node:test` , and more).

36.11 Closing note

You've now covered JavaScript from `let x = 1` to shipping tested, documented, dependency-free projects in both the browser and Node.js. The fastest way to make any of this permanent knowledge is the same as with any skill: build something real with it. Take one of the two capstone projects from Chapter 35, extend it with one of its suggested exercises, and let the next bug you hit teach you the one thing this book couldn't — what *your* specific mistakes look like, and how to debug them yourself.